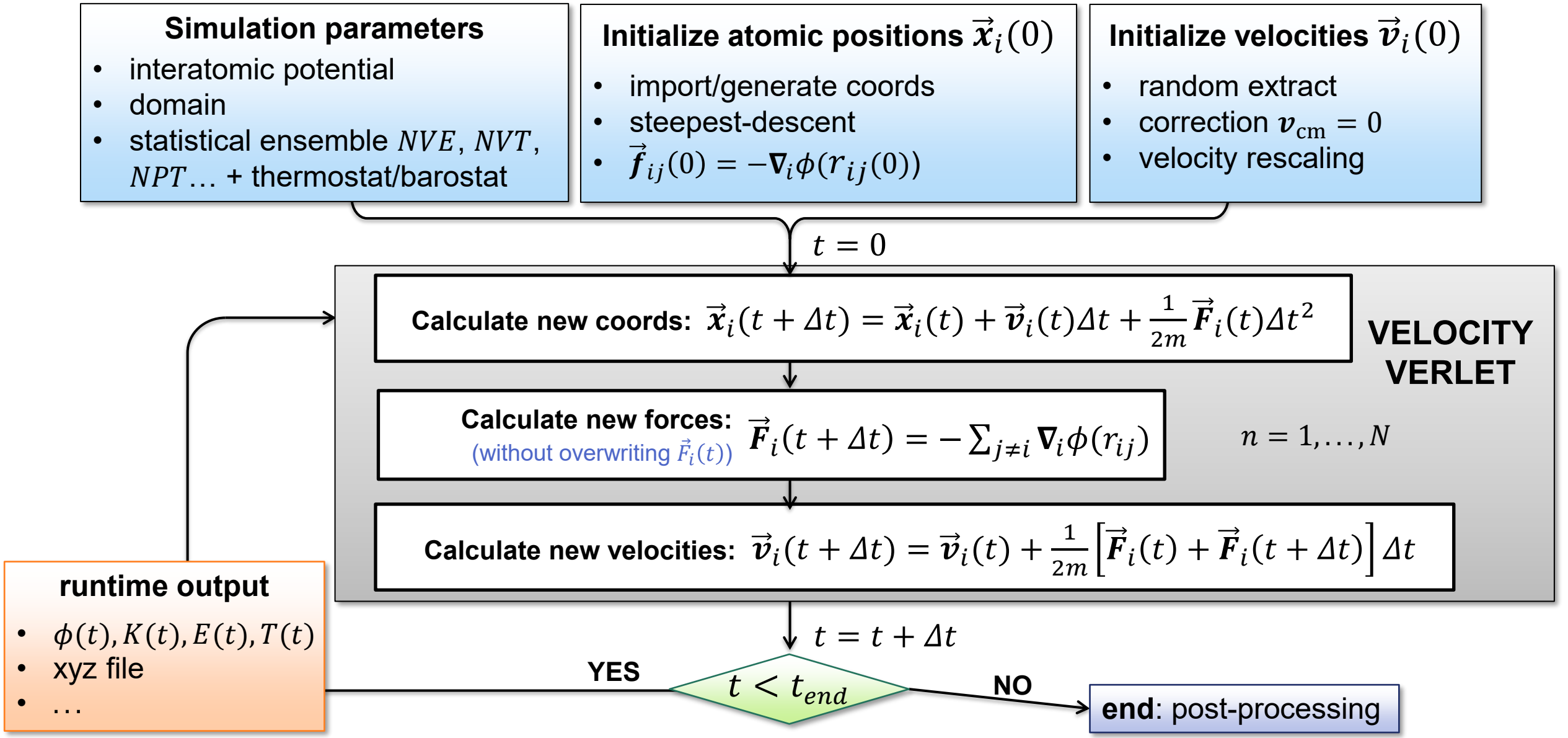
MD simulations

Roberto Bergamaschini

roberto.bergamaschini@unimib.it

MD simulation workflow



Results of MD simulations



- **system configuration**: list of atomic positions $\{\vec{x}_i(t)\}$ and velocities $\{\vec{v}_i(t)\}$
- **scalar outputs**, i.e. global descriptors to be monitored over time, e.g. $\phi(t), K(t), E(t), T(t)$

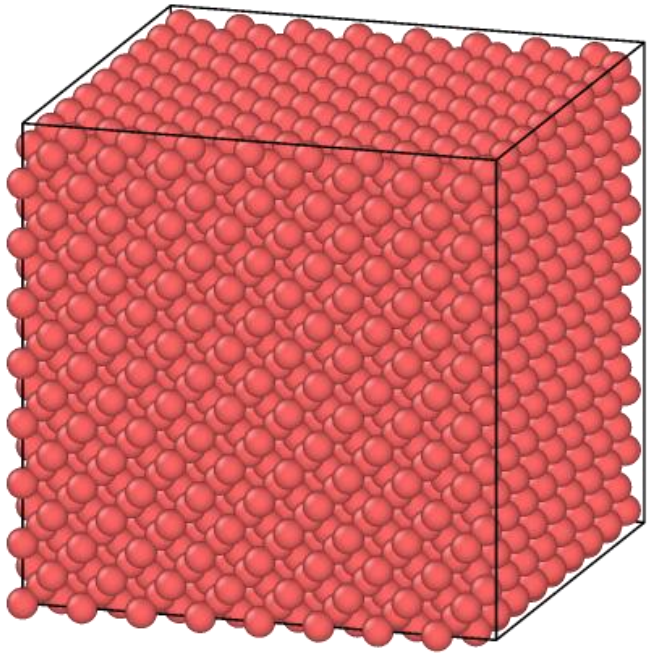
post-processing

- Qualitative changes in the system state e.g. new configurations, critical temperatures, rate of transitions...
- Statistical quantities (mean square displacement, pressure, correlation functions, etc.)

Running simulations costs a lot of time and resources... → Do not perform the analysis of results within the simulation code but save the relevant output in files and then process them later using independent scripts (you can do any analysis without having to repeat the heavy simulations, provided that you save all required data).

An example: MD simulation of a crystal bulk

Here we study a portion of an *infinite bulk* of Si crystal at different *temperature*.

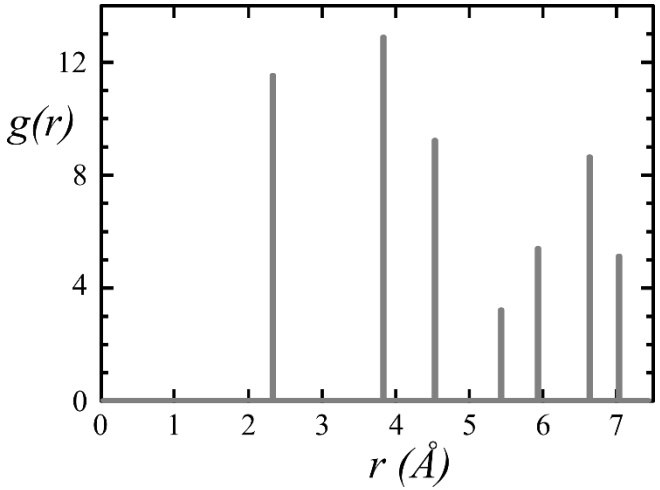


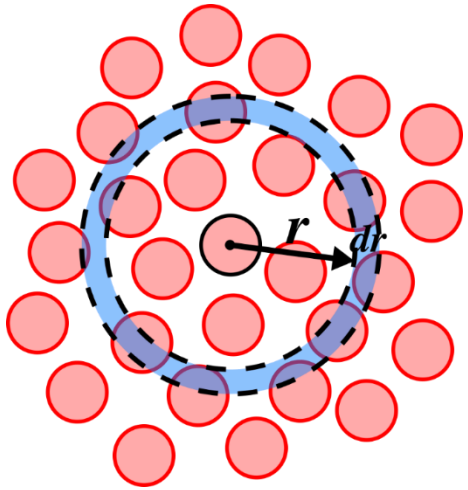
8x8x8 Si unit cells (2048 atoms)

Pair correlation function $g(r)$

$$N(R) = \rho_0 \int_0^R 4\pi r^2 g(r) dr$$

$\rho_0 = N/V$ (avg. atomic density)

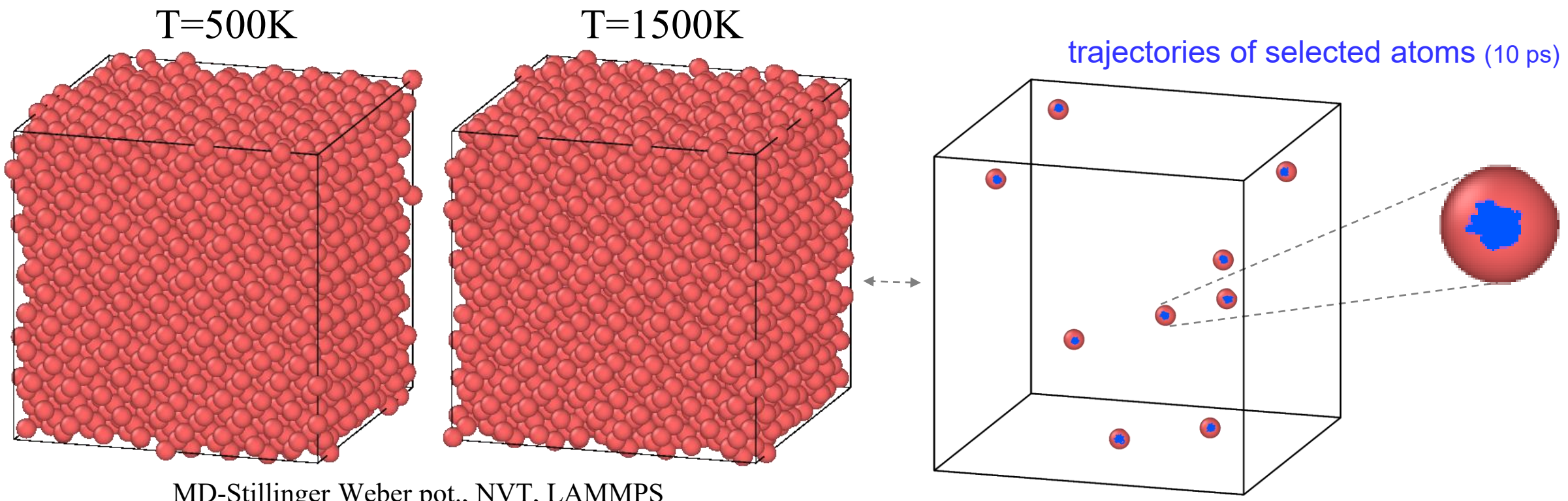




Peaks in $g(r)$ correspond to 1st neighbors, 2nd neighbors, 3rd neighbors….

An example: MD simulation of a crystal bulk – lattice vibrations

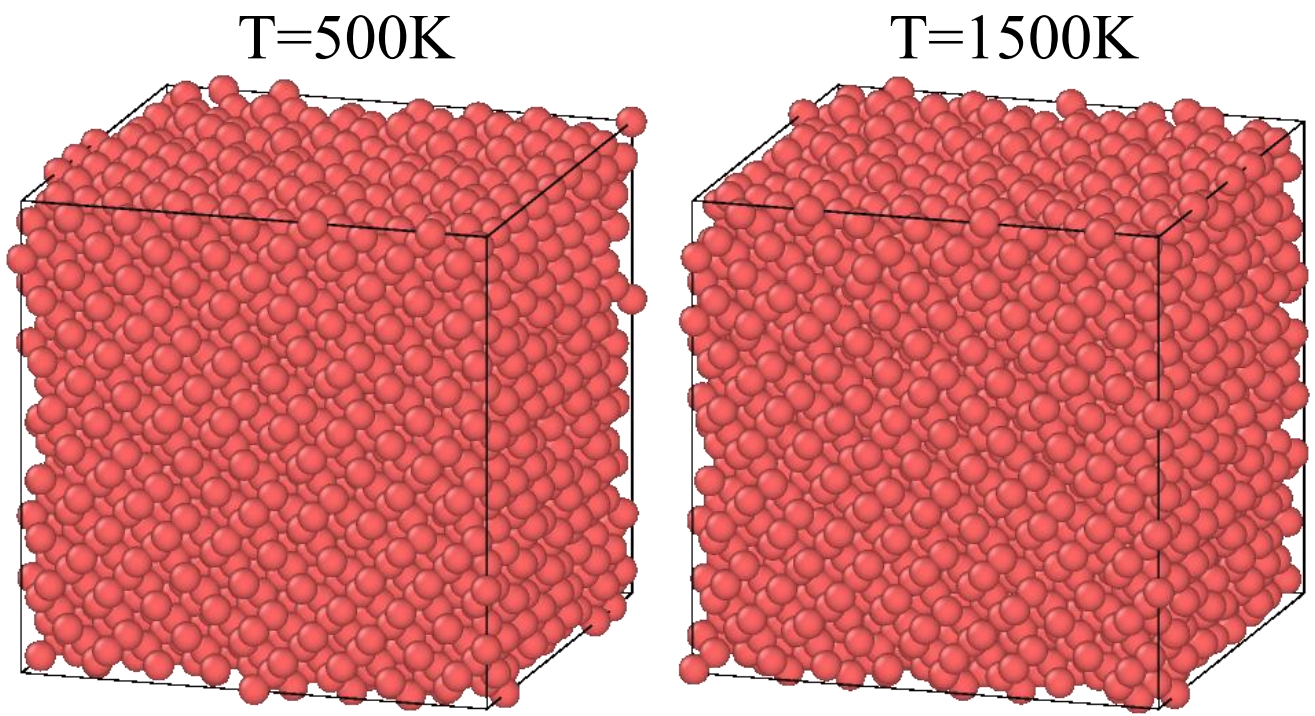
Here we study a portion of an *infinite bulk* of Si crystal at different *temperature*.



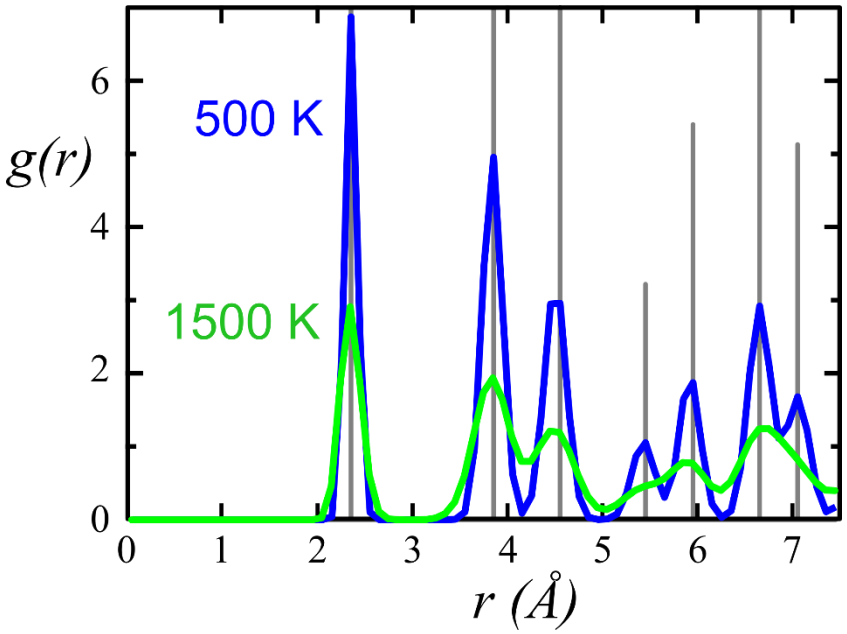
$T < T_m \rightarrow$ vibrational motion (phonons) of the crystalline phase

An example: MD simulation of a crystal bulk – lattice vibrations

Here we study a portion of an *infinite bulk* of Si crystal at different *temperature*.



MD-Stillinger Weber pot., NVT, LAMMPS

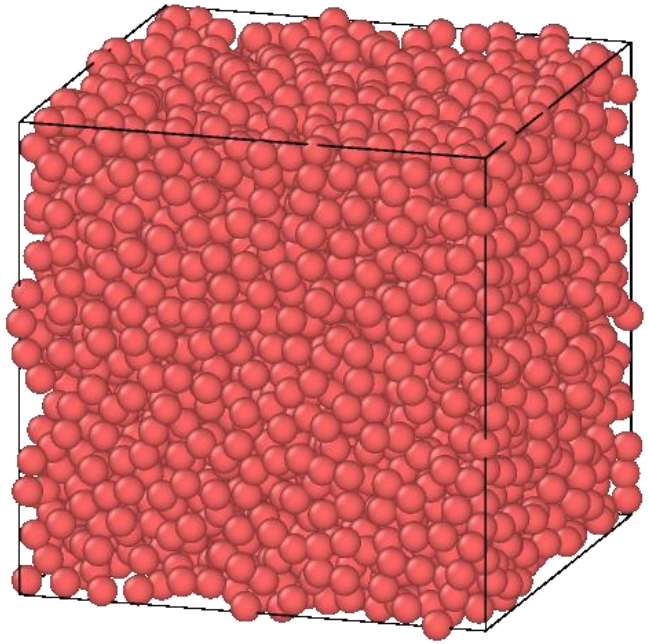


Thermal vibrations broaden the $g(r)$ peaks but we can still recognize the different coordination shells of the diamond structure.

An example: MD simulation of a bulk liquid phase

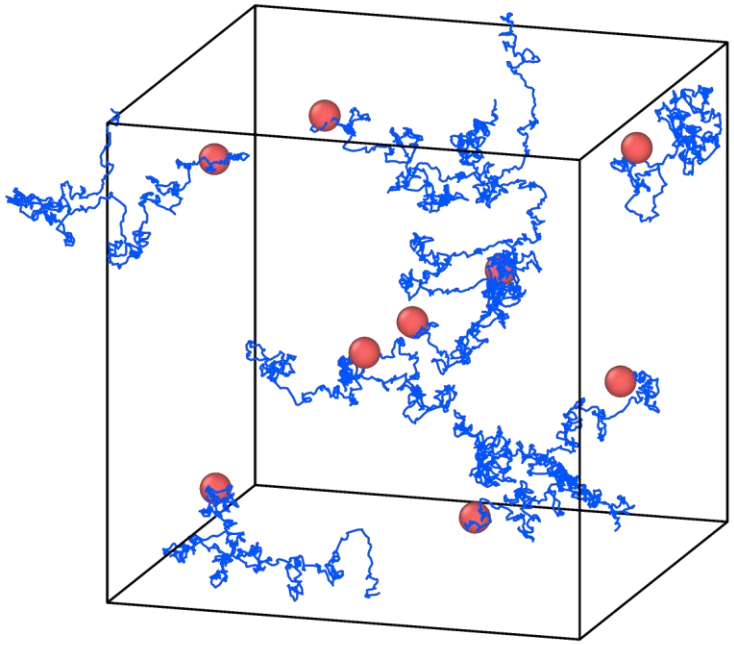
Here we study a portion of an *infinite bulk* of Si crystal at different *temperature*.
If we go above the melting temperature we actually obtain the liquid phase!

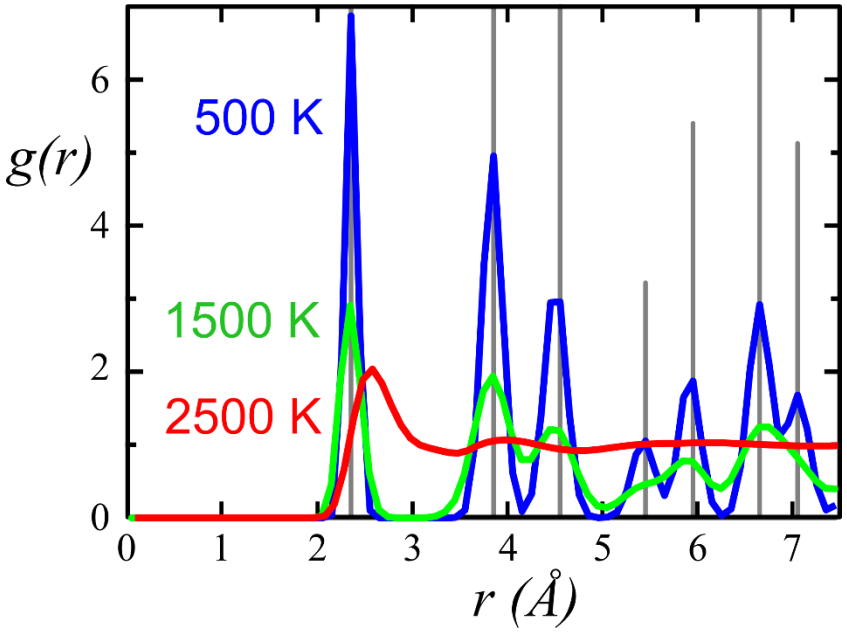
T=2500K



MD-Stillinger Weber pot., NVT, LAMMPS

trajectories of selected atoms (10 ps)



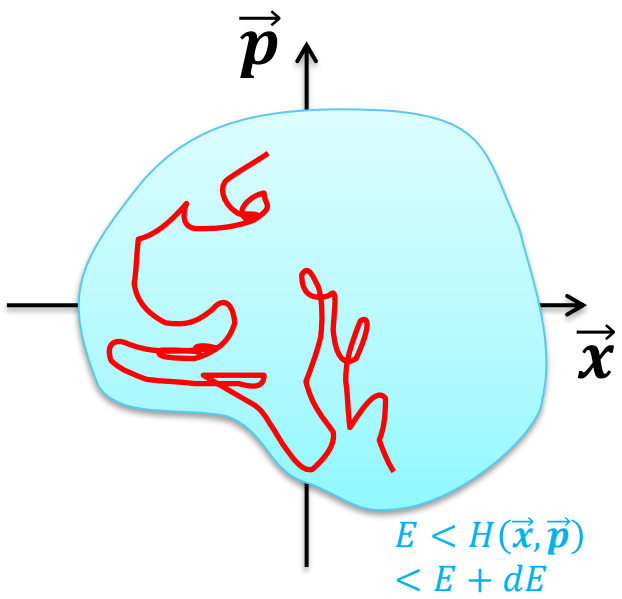


Diffusion dynamics

The liquid phase is not a random distribution of atoms [that would give constant $g(r) = 1$]. It has its own structure with short-range ordering reflecting bond interactions.

Time average ↔ NVE ensemble average

MD sets a certain trajectory within the phase-space, which for NVE is confined on the iso-energetic hypersurface $E = H(\vec{x}, \vec{p})$. For any observable A we can evaluate its average value along the trajectory:



$$\langle A \rangle_{MD} \approx \langle A \rangle_t = \lim_{t\to\infty} \frac{1}{t} \int_0^t A(\vec{x}(\mathrm{t}), \vec{p}(\mathrm{t}))\, d\mathrm{t}$$

temporal average

ergodic theorem

(a long enough trajectory spans the entire accessible phase-space)

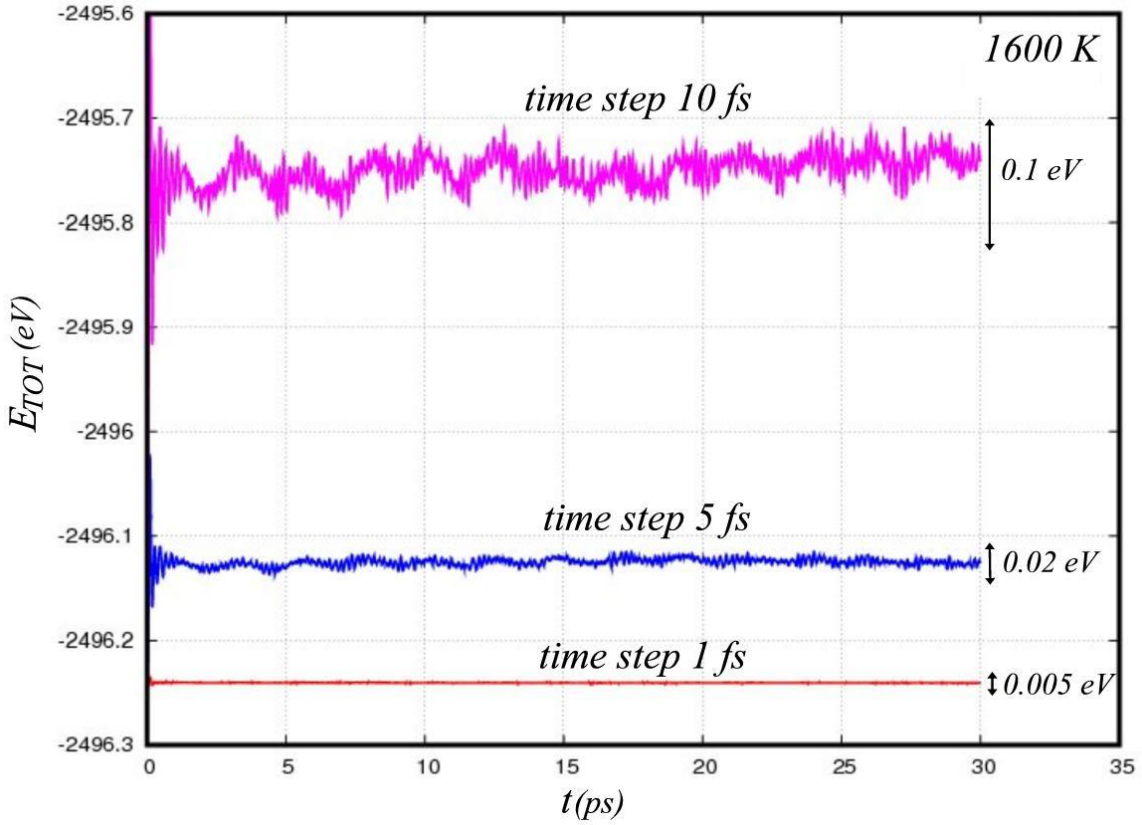
$$\langle A \rangle_{NVE} = \frac{\int_{E<H(\vec{x},\vec{p})<E+dE} A(\vec{x}, \vec{p})\, d\vec{x} d\vec{p}}{\int_{E<H(\vec{x},\vec{p})<E+dE} d\vec{x} d\vec{p}}$$

ensemble average

NVE MD:

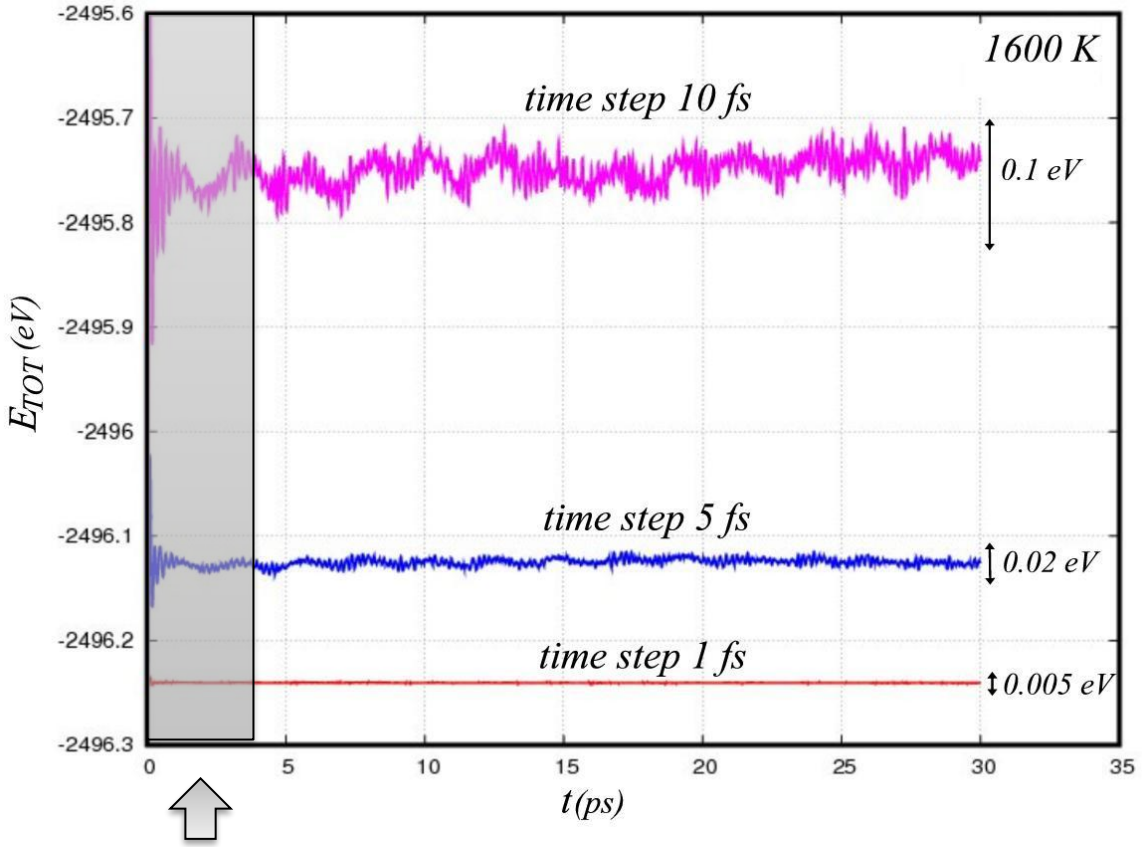
- very convenient as directly translating Newton law
- correct description of isolated systems

Evaluating averages and fluctuations



To compute meaningful time averages $\langle A \rangle_t$ we need to run ‘‘long enough’’ simulations (i.e. such that results are the same if we run for longer times)

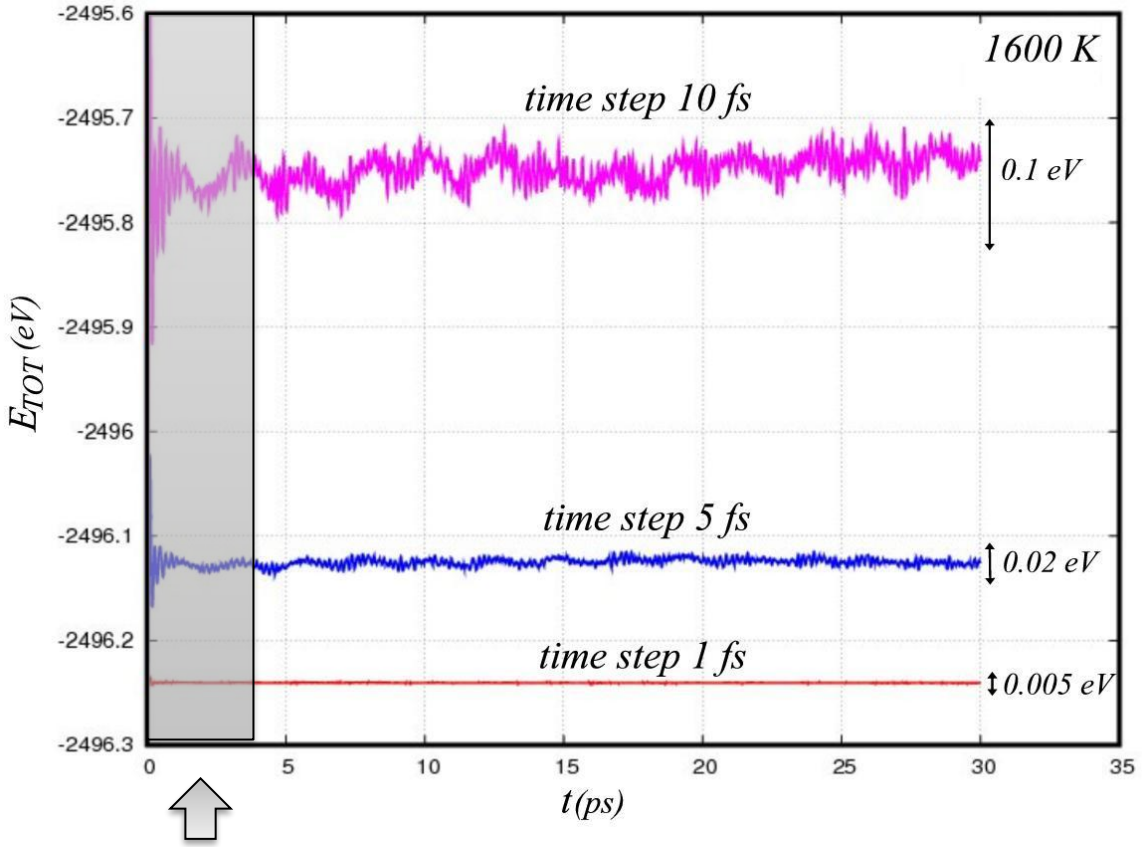
Evaluating averages and fluctuations



Thermalization: the initial state is not at equilibrium so that the system must first relax to it through vibrations. As vibrations happen on a scale of (fractions) of ps, the first stages, t_{th} in the order of tens ps, are to be excluded from averaging of equilibrium properties.

To compute meaningful time averages $\langle A \rangle_t$ we need to run ‘‘long enough’’ simulations (i.e. such that results are the same if we run for longer times)

Evaluating averages and fluctuations



Thermalization: the initial state is not at equilibrium so that the system must first relax to it through vibrations. As vibrations happen on a scale of (fractions) of ps, the first stages, t_{th} in the order of tens ps, are to be excluded from averaging of equilibrium properties.

To compute meaningful time averages $\langle A \rangle_t$ we need to run "long enough" simulations (i.e. such that results are the same if we run for longer times)

Time average of an observable A (e.g. E_{TOT} or T):

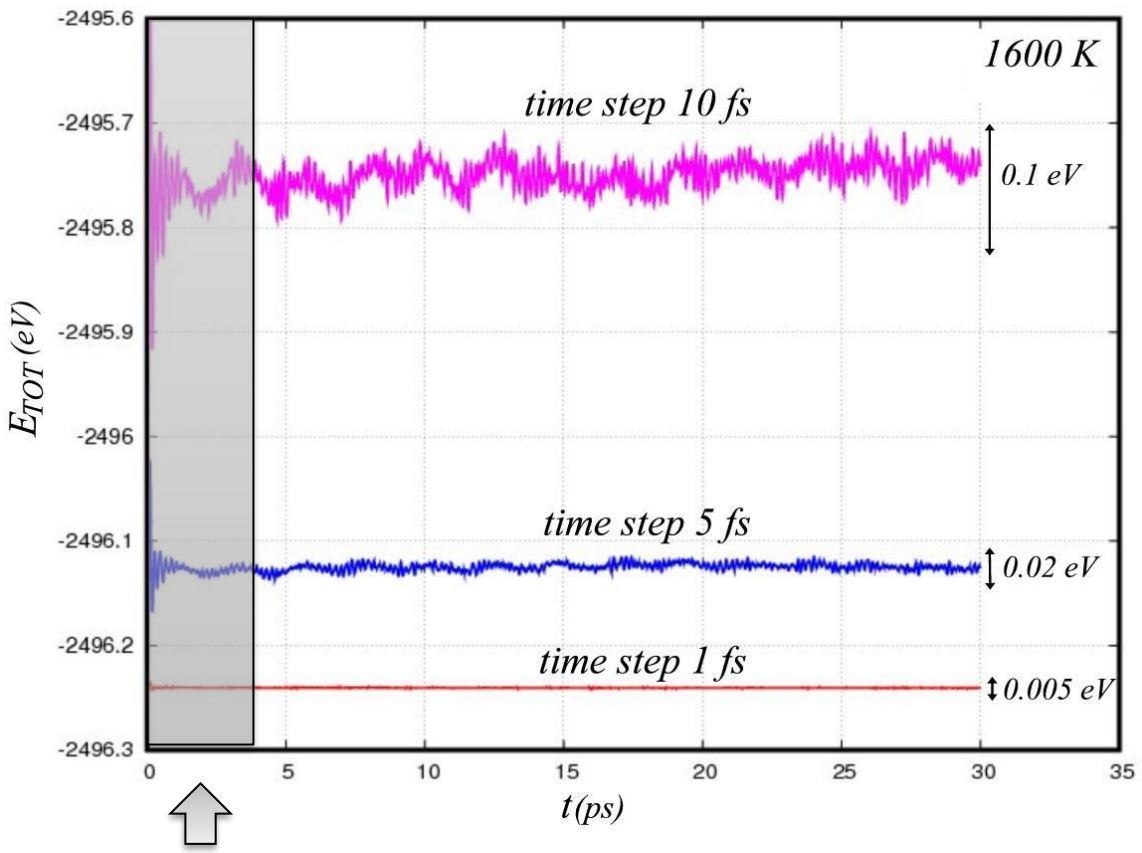
$$\langle A \rangle_t = \frac{1}{t - t_{th}} \int_{t_{th}}^{t} A(t')dt' \approx \frac{1}{t - t_{th}} \sum_{t'=t_{th}}^{t} A_{t'} \Delta t = \frac{1}{N} \sum_{i=i_{th}}^{N} A_i$$

number of MD steps

timestep

thermalization

Evaluating averages and fluctuations



Thermalization

To compute meaningful time averages $\langle A \rangle_t$ we need to run "long enough" simulations (i.e. such that results are the same if we run for longer times)

Time average of an observable A (e.g. E_{TOT} or T):

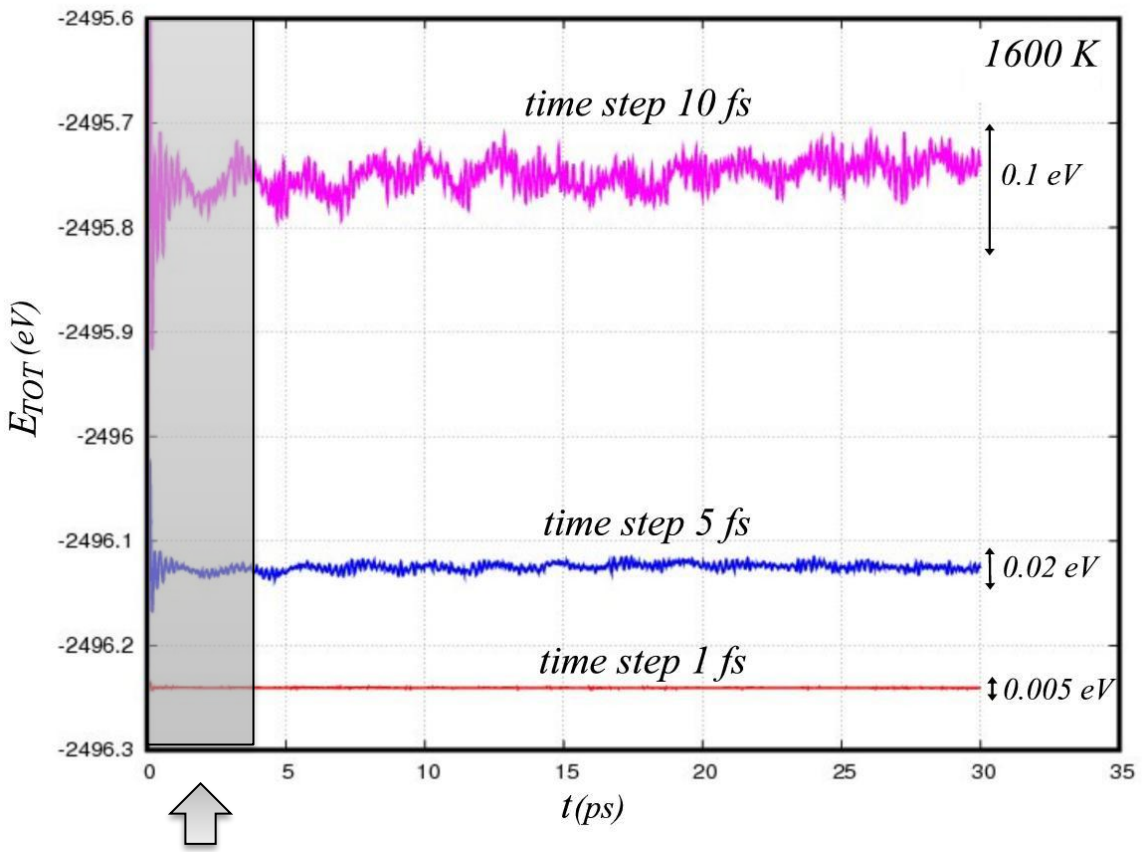
$$\langle A \rangle_t = \frac{1}{t - t_{th}} \int_{t_{th}}^{t} A(t')dt' \approx \frac{1}{t - t_{th}} \sum_{t'=t_{th}}^{t} A_{t'} \Delta t = \frac{1}{N} \sum_{i=i_{th}}^{N} A_i$$

Temporal fluctuations:

$$\delta A = \sqrt{\langle (A - \langle A \rangle)^2 \rangle} = \sqrt{\frac{1}{N} \sum_{i=i_{th}}^{N} (A_i - \langle A \rangle)^2} = \sqrt{\langle A^2 \rangle - \langle A \rangle^2}$$

$$\langle (A - \langle A \rangle)^2 \rangle = \langle A^2 - 2A\langle A \rangle + \langle A \rangle^2 \rangle =$$
$$= \langle A^2 \rangle - 2\langle A \rangle \langle A \rangle + \langle A \rangle^2 = \langle A^2 \rangle - \langle A \rangle^2$$

Evaluating averages and fluctuations



Thermalization

To compute meaningful time averages $\langle A \rangle_t$ we need to run "long enough" simulations (i.e. such that results are the same if we run for longer times)

Time average of an observable A (e.g. E_{TOT} or T):

$$\langle A \rangle_t = \frac{1}{t - t_{th}} \int_{t_{th}}^{t} A(t')dt' \approx \frac{1}{t - t_{th}} \sum_{t'=t_{th}}^{t} A_{t'} \Delta t = \frac{1}{N} \sum_{i=i_{th}}^{N} A_i$$

Temporal fluctuations:

$$\delta A = \sqrt{\langle (A - \langle A \rangle)^2 \rangle} = \sqrt{\frac{1}{N} \sum_{i=i_{th}}^{N} (A_i - \langle A \rangle)^2} = \sqrt{\langle A^2 \rangle - \langle A \rangle^2}$$

Remember that all average quantities and fluctuations are meaningful only after thermalization!

The simulation temperature is not the one you initialize T_{ini} but the one that you get at equilibrium, i.e. $\langle T \rangle \pm \delta T$

Fluctuations vs. Numerical errors in NVE simulations

E should be conserved exactly (we are solving Newton law for an isolated system)
... but numerical integration introduces truncation errors: Velocity Verlet $err(\vec{x}) \sim o(\Delta t^4)$, $err(\vec{v}) \sim o(\Delta t^3)$

δE is unphysical and must be kept as small as possible by reducing the time-step Δt

T is not a conserved quantity → it has to fluctuate over timea

δT fluctations are physical and do not depend on the numerical integration aspects, i.e. do not decrease with time step

Fluctuations in a statistical ensemble depend on the size of the system. The larger the number of atoms the lower the fluctuations as

$$\frac{\delta T}{\langle T \rangle} \approx \frac{1}{\sqrt{N}}$$

Thermodynamic limit $N \to \infty$: all ensembles coincide → no fluctuations of any observable ($\delta T \to 0$)

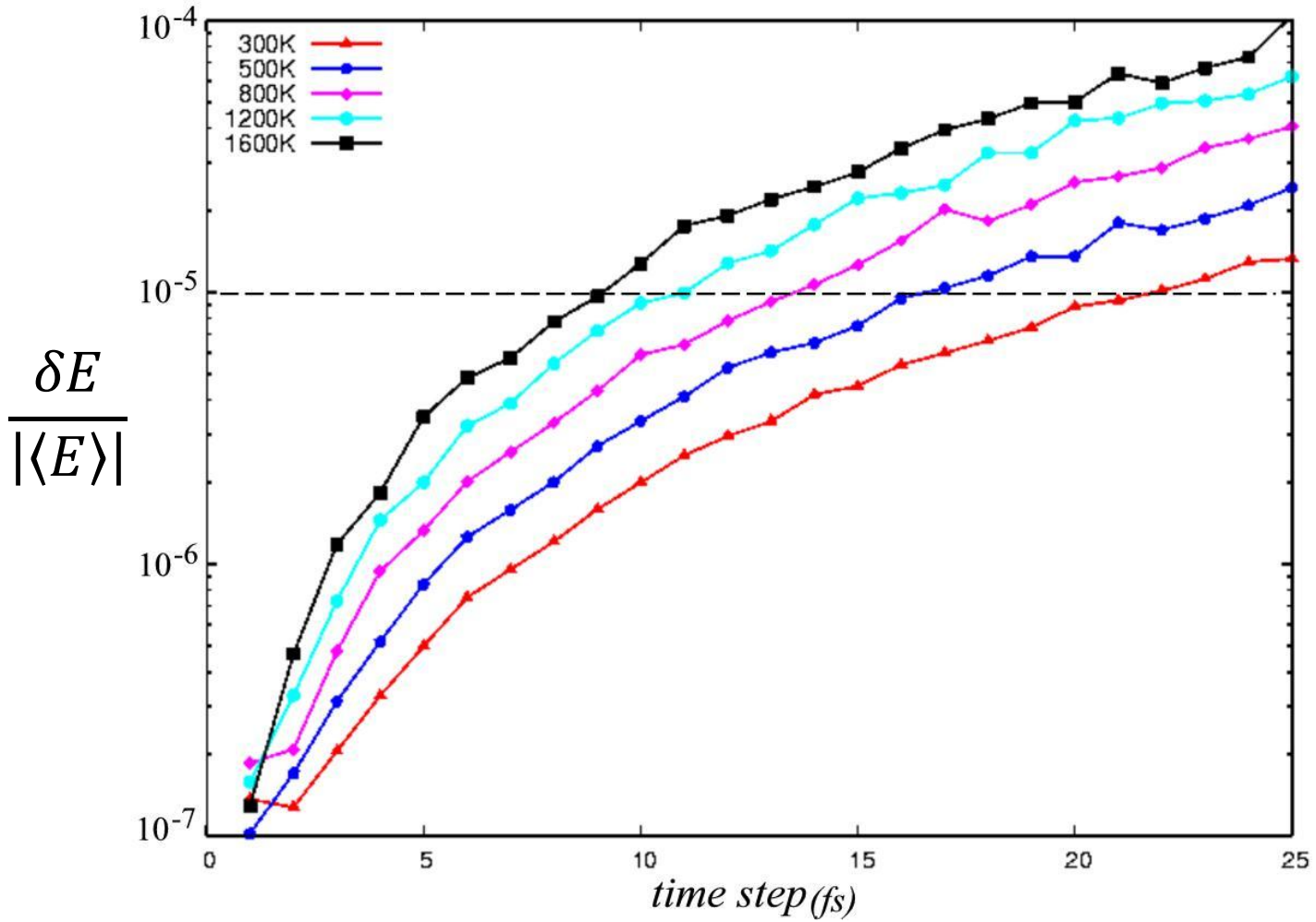
Optimal timestep

In order to perform reliable MD simulations we need to set a timestep such to

- keep truncation errors as low as possible
- allow to reach long enough times

Heuristic criterion: the optimal timestep is the largest one which fulfills the energy conservation criterion

$$\frac{\delta E}{|\langle E \rangle|} \leq 10^{-5}$$



Managing output and analysis of results

Input/output files

```
fout = open('test.txt','w')
fout.write('write this string and move to newline\n')
fout.write(f'This is the value in var: {var} \n')
fout.write('This is the value in var:' + str(var) + '\n')
fout.close()
```

Same as **`print`** but writing in a file rather than on screen

- write data in external files for post-processing (any time needed)
- write on screen every `nPrint` (=100,1000,…) steps to monitor what's going on (writing any output on the screen is a computationally expensive!)

```
fout = open('output.txt','w')
...
for iStep in range(0,totStep):      % MD loop
    ##############     OUTPUT
   fout.write(f'{time} \t {eTot} \t {ePot} \t {eKin} \t {Temp}')
    if( iStep % nPrint ==0 ):
       print(f"Time={time} - Energy={eTot}\n")      % check that code running
       # visualize the configuration (if needed)
       # write coordinates of all atoms in a file
    ...
fout.close()
```

Statistical analysis of output data



We want to evaluate averages and fluctuations of quantities that we have reported at each time step in the `output.txt` file. Lets write an independent script to do it (if you don't like Python you can import the file in any other software (Matlab, Excel, Origin...)

```
# statistics.py
...
data=np.loadtxt('output.txt')
time=data[:,0]
eTot=data[:,1]
```

STANDARD APPROACH

```
n=0
avgE=0
for t in Time:
  if(t>thTime):
    n=n+1
    avgE=avgE+eTot[t]
    ...
end
avgE=avgE/n
```

Use built-in numpy function but you need to exclude the data before thermalization time

"COMPACT" APPROACH

```
for iTh in Time:
  if(time[iTh]>thTime):
    break

avgE=np.mean(eTot[iTh:-1])
```

Output atomic positions and visualize the cluster



We want to observe the motion of atoms. At each time we know the (x, y, z) coordinates of all of them.

1. Lets write them into progressively numbered files (WARNING: DO NOT OUTPUT THE ATOMIC POSITION EVERY MD STEP! (atomic motion from one timestep to the next is very small + long simulations mean a lot of files that can exhaust the PC memory))

```
fname = 'conf' + str(iOut) + '.xyz')
```

[Fixed number **N** of digits? `str(iOut).zfill(N)`]

STANDARD APPROACH

```
ff=open(fname,'w')
for i in range(0, nAt):
   ff.write(f'{pos[i,0]} \t {pos[i,1]} \t {pos[i,2]} \n')
ff.close()
```

COMPACT APPROACH

```
np.savetxt(fname, pos)
```

Notice: `pos=np.loadtxt("fcc100a256.txt")`

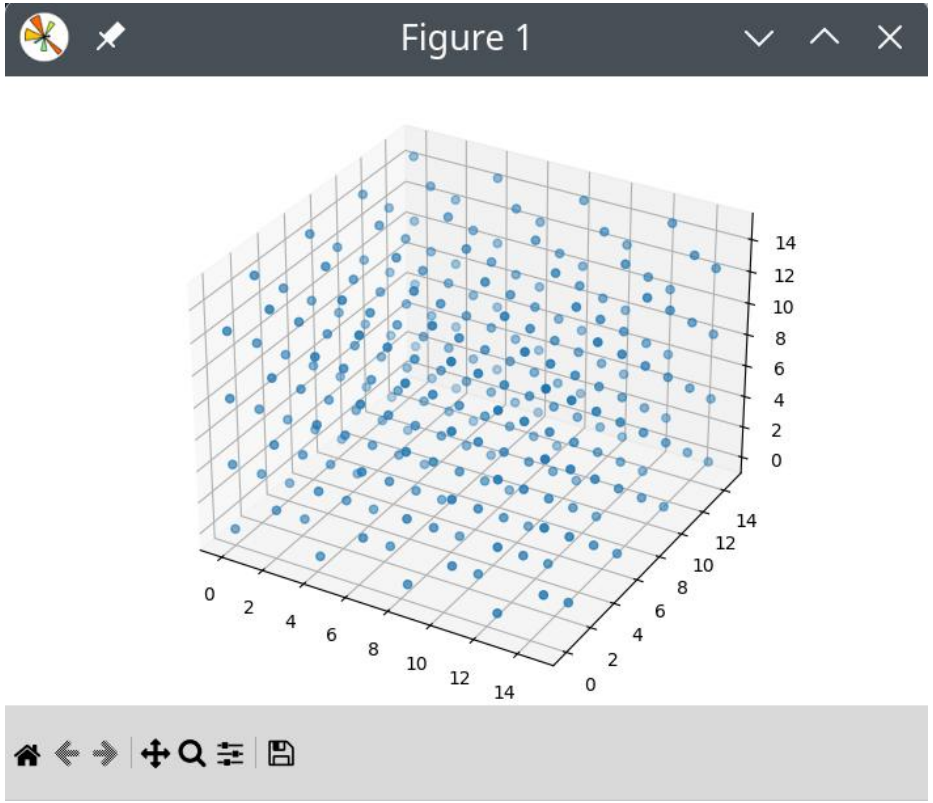
Output atomic positions and visualize the cluster

We want to observe the motion of atoms. At each time we know the (x, y, z) coordinates of all of them.

2. Visualization of atomic configurations

Matplotlib: write your own script for visualizing atomic positions

```
from mpl_toolkits import mplot3d
...
ax = plt.axes(projection='3d')
ax.scatter(pos[:,0],pos[:,1],pos[:,2])
plt.savefig('test.png')
plt.show()
```



Output atomic positions and visualize the cluster

We want to observe the motion of atoms. At each time we know the (x, y, z) coordinates of all of them.

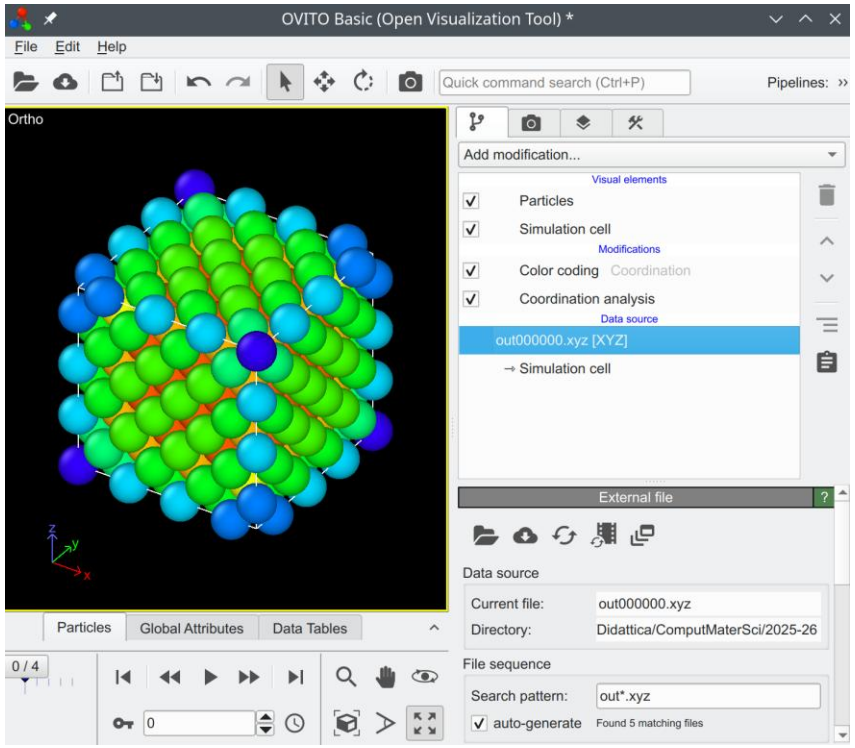
2. Visualization of atomic configurations

Third-party visualization codes: OVITO, VMD…

Here we will use **OVITO**. It can directly open your .xyz files (and make animation out of them) but you have to include 2 lines at the top

Line 1: number of atoms `nAt`
Line 2: anything you like (e.g. `time, eTot, Temp`)

Line 3 to `nAt+2`: x y z coordinates (you can also add other columns, e.g. velocities)



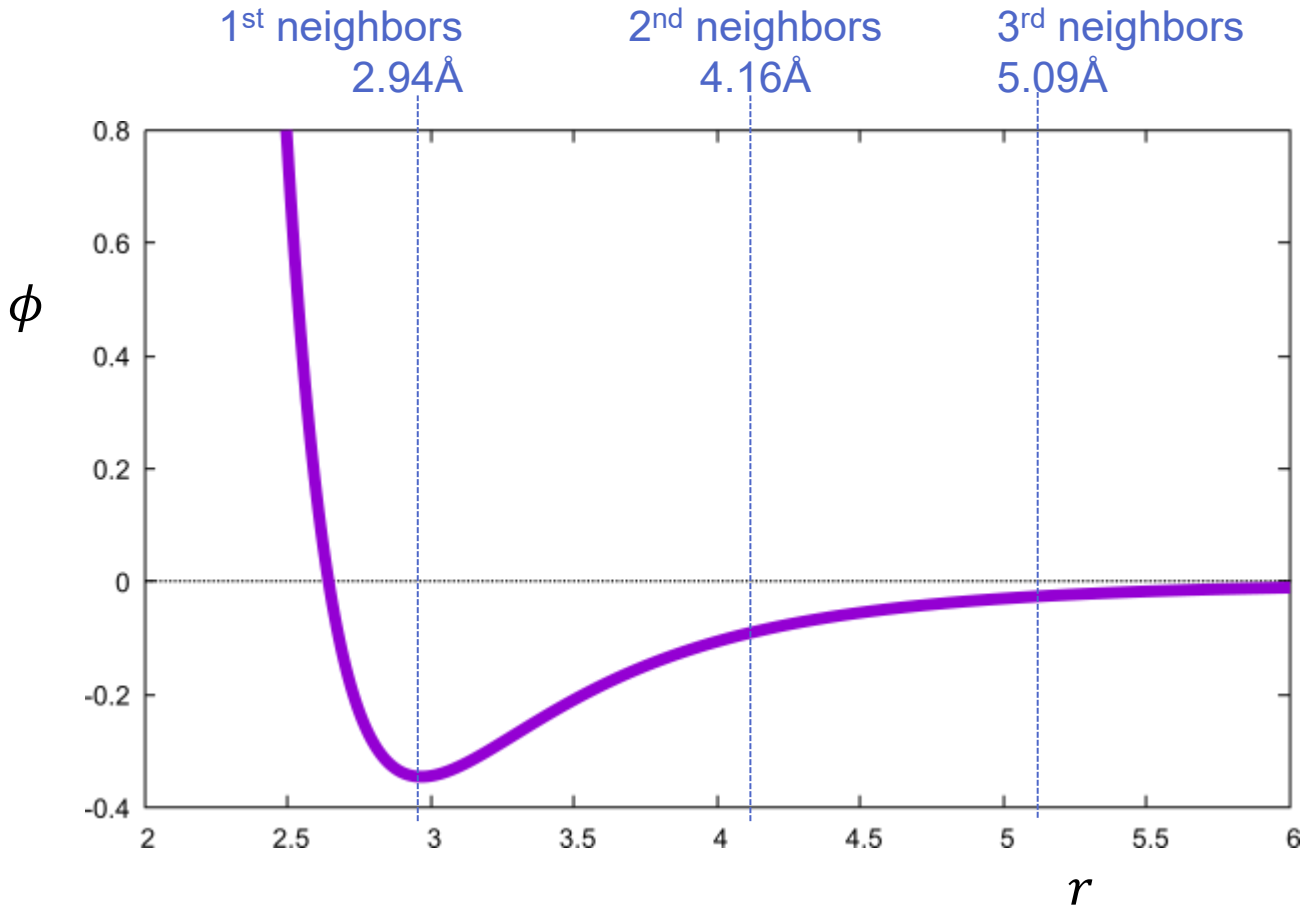
This can be achieved by using:

```
np.savetxt(fname, pos, header='f{nAt}\n time={time}', comments='')
```

For importing these files (ignoring the first 2 lines): `np.loadtxt(fname, skiprows=2)`

Lennard-Jones potential with polynomial junction

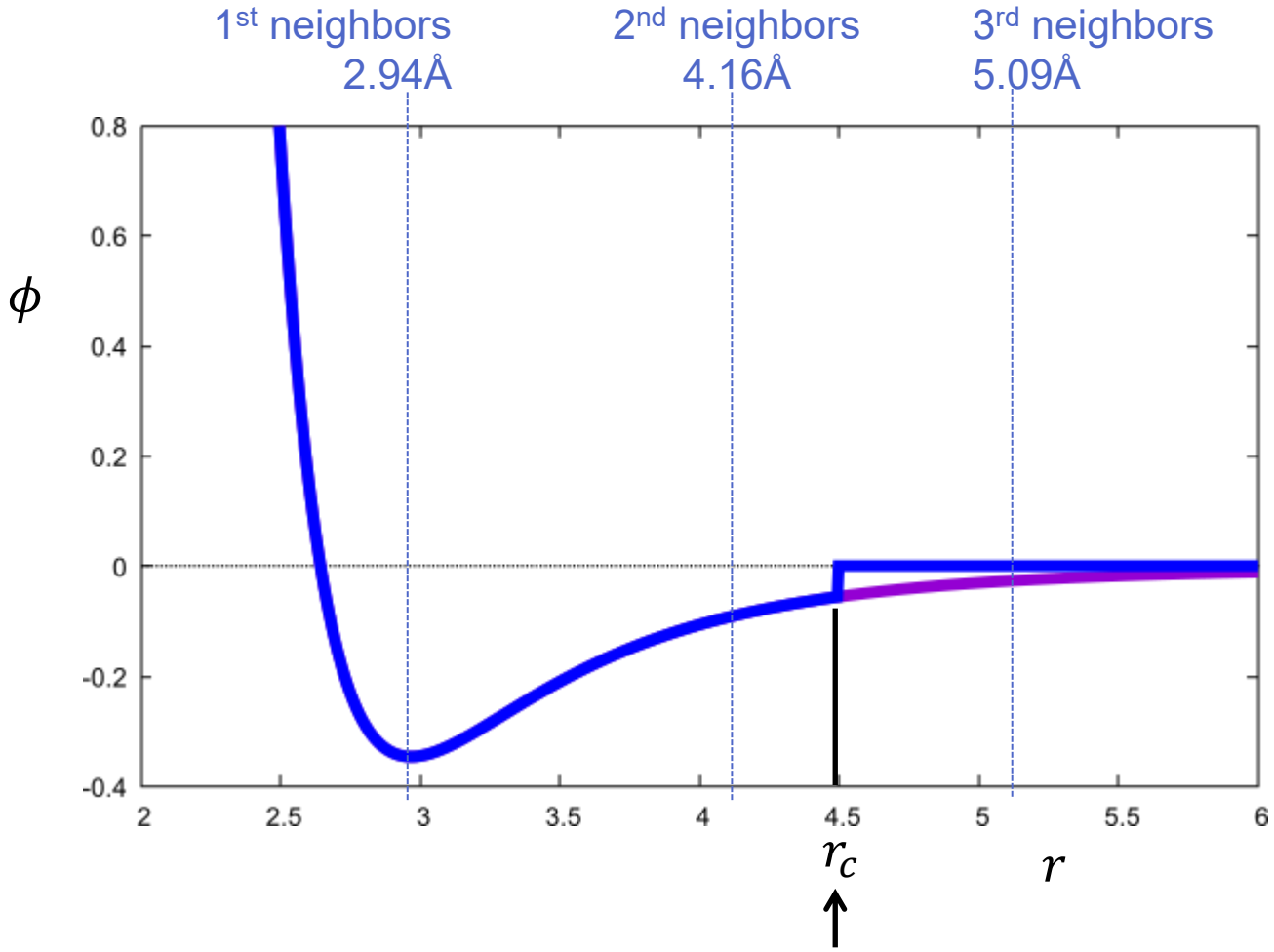
LJ potential



$$\varepsilon = 0.345\ eV \quad - \quad \sigma = \ 2.644\ \text{Å}$$

$$\phi_{LJ}(r) = 4\varepsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right]$$

LJ potential with sharp cutoff



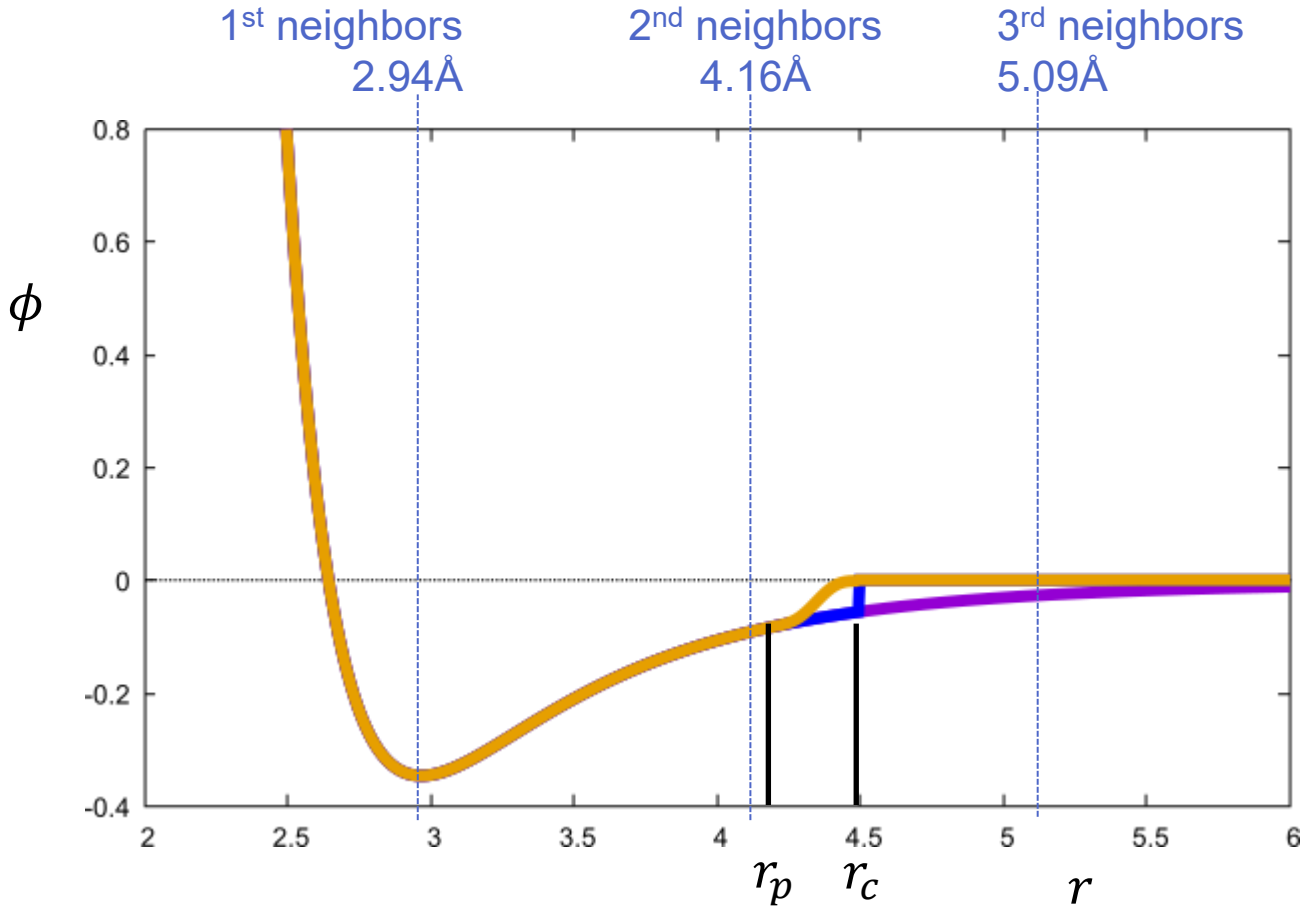
Discontinuity in energy and singularity in forces!!!

$$\varepsilon = 0.345\ eV \quad - \quad \sigma = 2.644\ \text{Å}$$
$$r_c = 4.5\ \text{Å}$$

$$\phi_{LJ}(r) = 4\varepsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right]$$

$$\phi = \begin{cases} \phi_{LJ}(r) & r \le r_c \\ 0 & r > r_c \end{cases}$$

LJ potential with polynomial junction



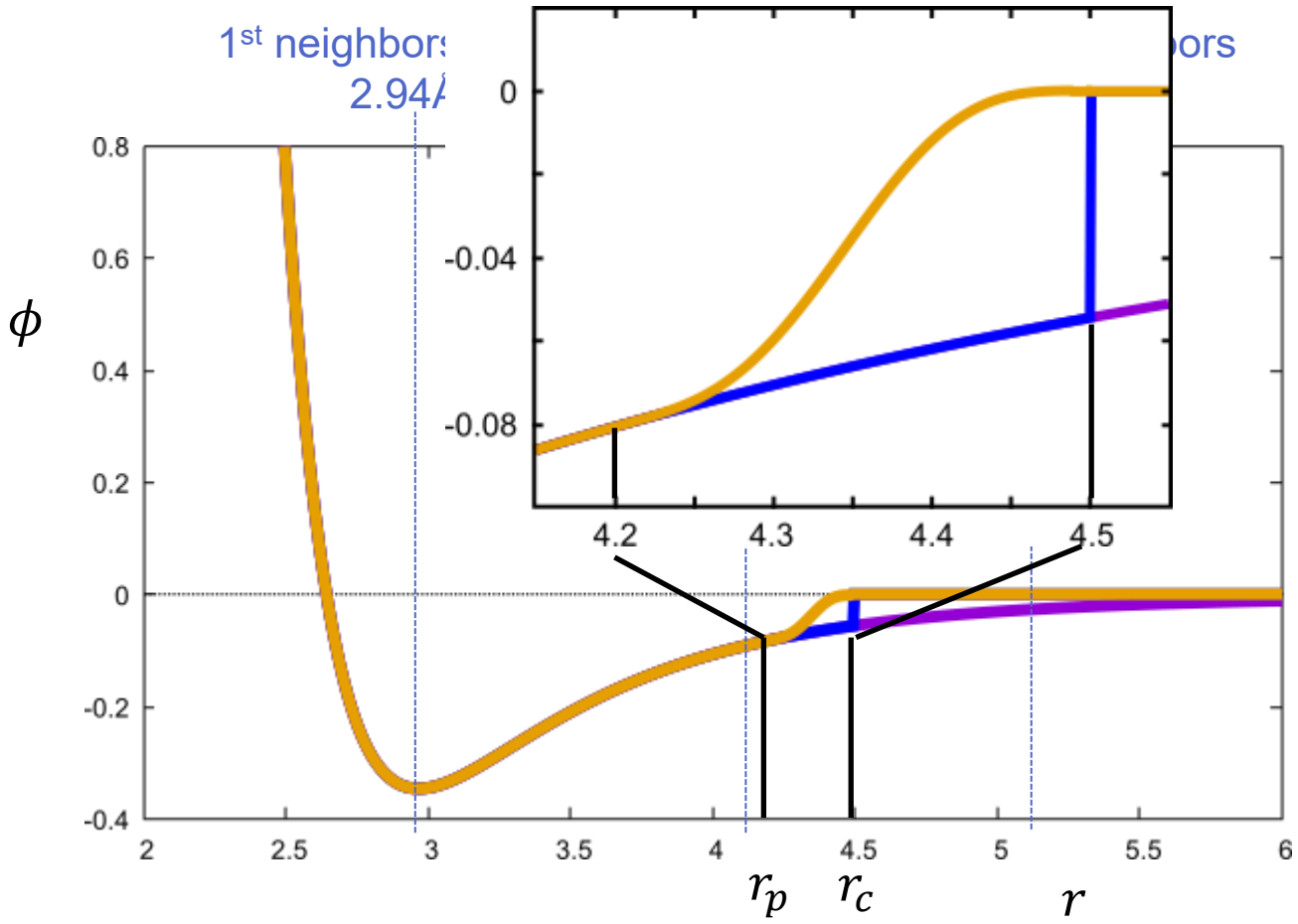
$$\varepsilon = 0.345\ eV \quad - \quad \sigma = 2.644\ \text{Å}$$
$$r_p = 4.2\ \text{Å} \quad - \quad r_c = 4.5\ \text{Å}$$

$$\phi_{LJ}(r) = 4\varepsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right]$$

$$\phi = \begin{cases} \phi_{LJ}(r) & r \le r_p \\ P_7(r) & r_p < r \le r_c \\ 0 & r > r_c \end{cases}$$

$$P_7(r) = A + Br + Cr^2 + Dr^3 + Er^4 + Fr^5 + Gr^6 + Hr^7$$

LJ potential with polynomial junction



$$\varepsilon = 0.345\ eV \quad - \quad \sigma = 2.644\ \text{Å}$$
$$r_p = 4.2\ \text{Å} \quad - \quad r_c = 4.5\ \text{Å}$$

$$\phi_{LJ}(r) = 4\varepsilon\left[\left(\frac{\sigma}{r}\right)^{12} - \left(\frac{\sigma}{r}\right)^{6}\right]$$

$$\phi = \begin{cases} \phi_{LJ}(r) & r \le r_p \\ P_7(r) & r_p < r \le r_c \\ 0 & r > r_c \end{cases}$$

$$P_7(r) = A + Br + Cr^2 + Dr^3 + Er^4 + Fr^5 + Gr^6 + Hr^7$$

How to determine the 8 polynomia coefficients?

Continuity of ϕ

$$P_7(r_p) = \phi_{LJ}(r_p)$$
$$P_7(r_c) = 0$$

Continuity of forces

$$P_7'(r_p) = \phi_{LJ}'(r_p)$$
$$P_7'(r_c) = 0$$

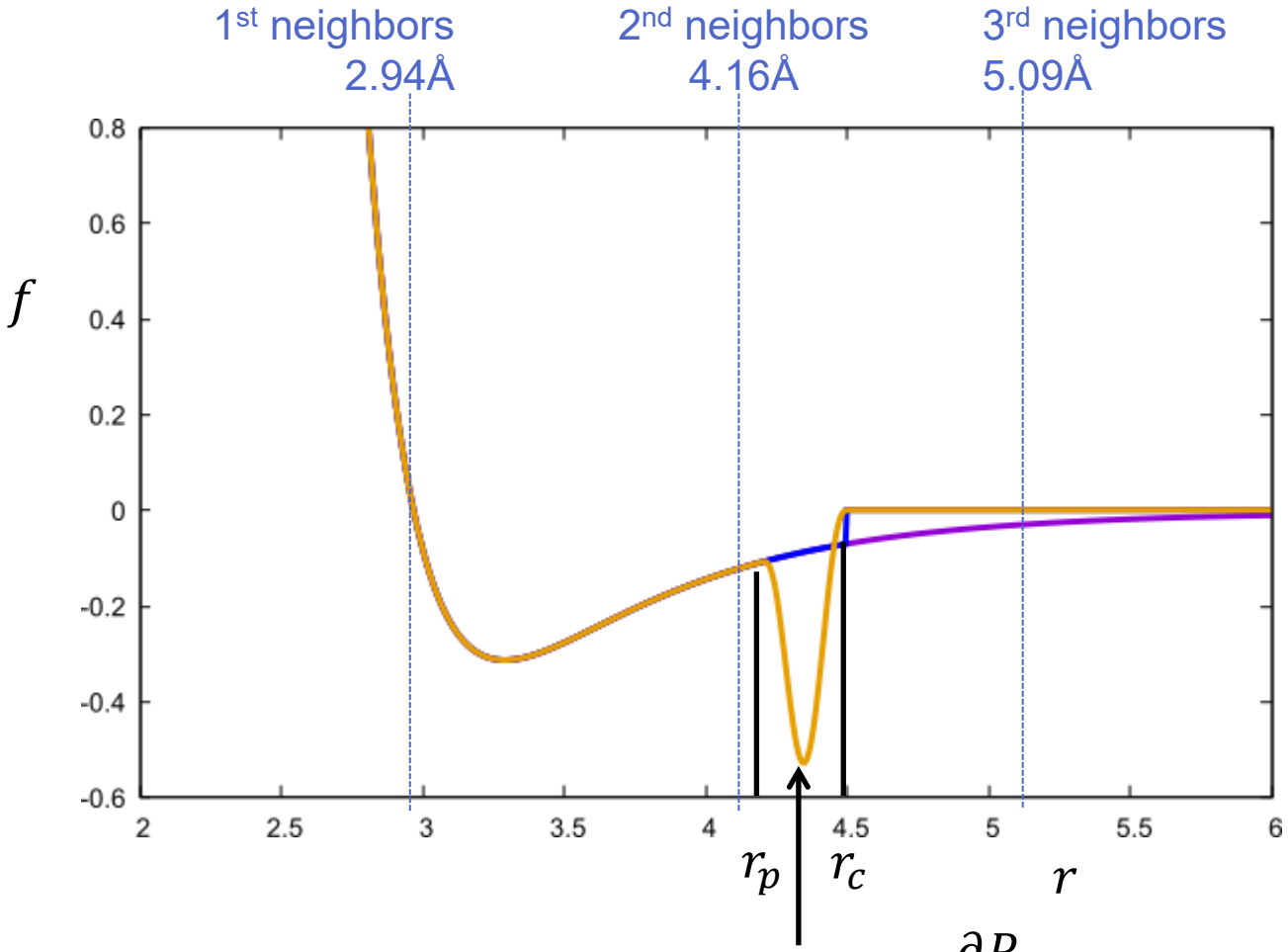
1st derivative of forces

$$P_7''(r_p) = \phi_{LJ}''(r_p)$$
$$P_7''(r_c) = 0$$

2nd derivative of forces

$$P_7'''(r_p) = \phi_{LJ}'''(r_p)$$
$$P_7'''(r_c) = 0$$

Force by LJ potential with polynomial junction



$$\varepsilon = 0.345\ eV \quad - \quad \sigma = 2.644\ \text{Å}$$
$$r_p = 4.2\ \text{Å} \quad - \quad r_c = 4.5\ \text{Å}$$

$$f = \begin{cases} f_{LJ}(r) & r \le r_p \\ -P_7'(r) & r_p < r \le r_c \\ 0 & r > r_c \end{cases}$$

$$P_7'(r) = \frac{\partial P_7}{\partial r} = B + 2Cr + 3Dr^2 + 4Er^3 + 5Fr^4 + 6Gr^5 + 7Hr^6$$

This is the expression for the force within the polynomial junction for a single-pair (1D)

Evaluation of the forces for the 3D cluster of Nat atoms



From the previous lessons you know that, for whatever definition of a pair potential ϕ

$$\vec{\boldsymbol{F}}(k) = \sum_{j \neq k} \vec{\boldsymbol{f}}_j(k) = \sum_{j \neq k} \left[-\boldsymbol{\nabla}_k \phi(r_{kj}) \right] \quad \leftrightarrow \quad F_x(k) = \sum_{j \neq k} f_{x,j}(k) = \sum_{j \neq k} \left[-\frac{\partial}{\partial x(k)} \phi(r_{kj}) \right]$$

same for F_y and F_z

force exerted by atom j on atom k

gradient with respect to coordinates of atom k

$$r_{kj} = \sqrt{\left(x(k) - x(j)\right)^2 + \left(y(k) - y(j)\right)^2 + \left(z(k) - z(j)\right)^2}$$

$$f_{x,j}(k) = -\frac{\partial}{\partial x(k)} \phi(r_{kj}) = -\frac{\partial \phi(r_{kj})}{\partial r_{kj}} \cdot \frac{\partial r_{kj}}{\partial x(k)} = -\frac{\partial \phi(r_{kj})}{\partial r_{kj}} \cdot \frac{x(k) - x(j)}{r_{kj}}$$

$r_{kj} < r_p \rightarrow$ LJ potential $\phi_{LJ}(r)$ $\rightarrow$ $f_{x,j}^{LJ}(k) = -\frac{\partial \phi_{LJ}(r_{kj})}{\partial r_{kj}} \cdot \frac{x(k)-x(j)}{r_{kj}} = 24\varepsilon\sigma^6 \frac{2\sigma^6/r_{kj}^6 - 1}{r_{kj}^7} \cdot \frac{x(k)-x(j)}{r_{kj}}$

$r_p < r_{kj} < r_c \rightarrow$ Polynomial junction $P_7(r)$ $\rightarrow$ $f_{x,j}^{P_7}(k) = -\frac{\partial P_7(r_{kj})}{\partial r_{kj}} \cdot \frac{x(k)-x(j)}{r_{kj}} = -P_7'(r_{kj}) \cdot \frac{x(k)-x(j)}{r_{kj}}$

Implementation of polynomial junction



For convenience, the 8 polynomia coefficients are already computed and available in the file `poly7.py`
They are defined as members of the `poly7` **class** (container of variables and methods). To use them:

```
import poly7
...
LJpoly=poly7(eps,sig,rcut,rpoly)      #initialize class poly7 with LJ parameters
                                      # after this, each coefficient is simply accessible as  LJpoly.A, LJpoly.B, ...

ePot = calcEnergy(..., LJpoly.A, LJpoly.B, ...)      # Transfer coefficients as function arguments
```

Add polynomial junction in the function computing potential energy

```
def calcEnergy(...,polyA,polyB,...):
    ...
    ePot=0
    for i in range(0,nAt):
      for j in range(0,nNbrs[i]):
        dist=calcDist(pos[i],pos[whichNbrs[j] ] )
        if(dist<rpoli):
          ePot=ePot + LJ(dist)
        else:
          ePot=ePot + Poly(dist)
    return ePot
```

Add polynomial junction in the function computing forces

```
def function calcForces(...,polyA,polyB,...):
    ...
    for i in range(0,nAt):
      for j in range(0,nNbrs[i]):
        dx=pos[i]-pos[whichNbrs[j] ]
        dist=calcDist(pos[i], pos[whichNbrs[j] ] )
        if(dist<rpoli):
          force[i]=force[i] + fLJ(dist) * dx
        else:
          force[i]=force[i] + fPoly(dist) * dx
    return forces
```

Steepest-descent algorithm

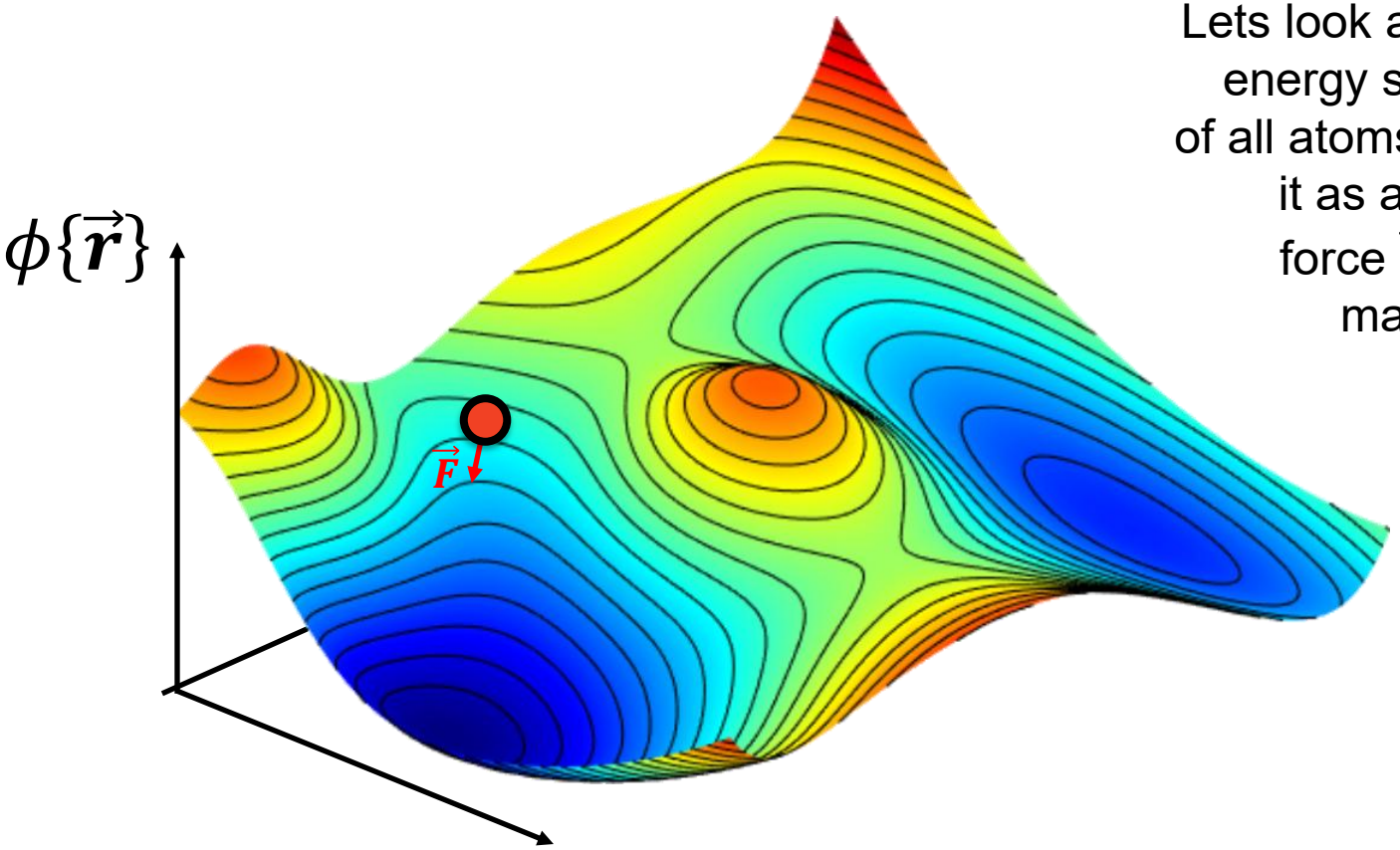
Local minimization: steepest-descent algorithm

Lets consider a configuration of N atoms at coordinates $\{\vec{r}_i\}$.

$$\vec{F}(k) = \sum_{j \neq k} [-\nabla_k \phi(r_{kj})]$$

If a configuration corresponds to a minimum of potential energy → equilibrium: $\vec{F}(k) = 0 \ \ \forall k$

For a generic configuration (such as the initial configuration of the MD simulation) the atoms are subject to non-zero forces. Forces are always directed opposite to the potential energy gradient so that they move atoms toward the (closer) energy minimum.



Lets look at one atom sitting at some point $\vec{r}_k$ on the potential energy surface (PES) determined by the (relative) positions of all atoms (it should be a $3N$ hypersurface but here we draw it as a 2D surface for simplicity). The atom is subject to a force $\vec{F}(k)$ directed toward the closest minimum and with magnitude proportional to how steep is the PES at $\vec{r}_k$.

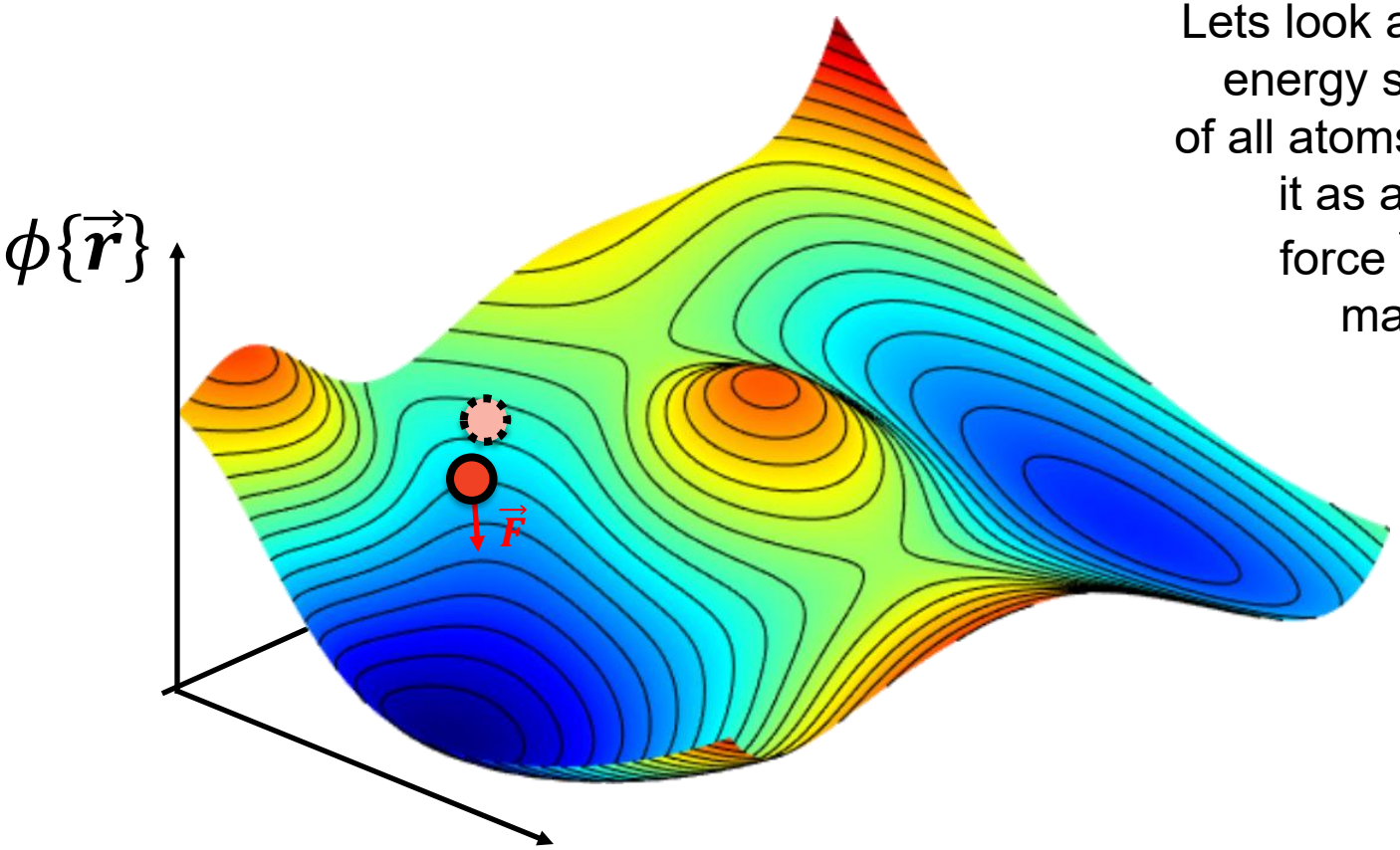
Local minimization: steepest-descent algorithm

Lets consider a configuration of N atoms at coordinates $\{\vec{r}_i\}$.

$$\vec{F}(k) = \sum_{j \neq k} [-\nabla_k \phi(r_{kj})]$$

If a configuration corresponds to a minimum of potential energy → equilibrium: $\vec{F}(k) = 0 \ \ \forall k$

For a generic configuration (such as the initial configuration of the MD simulation) the atoms are subject to non-zero forces. Forces are always directed opposite to the potential energy gradient so that they move atoms toward the (closer) energy minimum.



Lets look at one atom sitting at some point $\vec{r}_k$ on the potential energy surface (PES) determined by the (relative) positions of all atoms (it should be a $3N$ hypersurface but here we draw it as a 2D surface for simplicity). The atom is subject to a force $\vec{F}(k)$ directed toward the closest minimum and with magnitude proportional to how steep is the PES at $\vec{r}_k$.

Lets move "a bit" the atom along the direction of $\vec{F}$ (and do the same for all other atoms): we reach a point that is closer to the energy minimum!

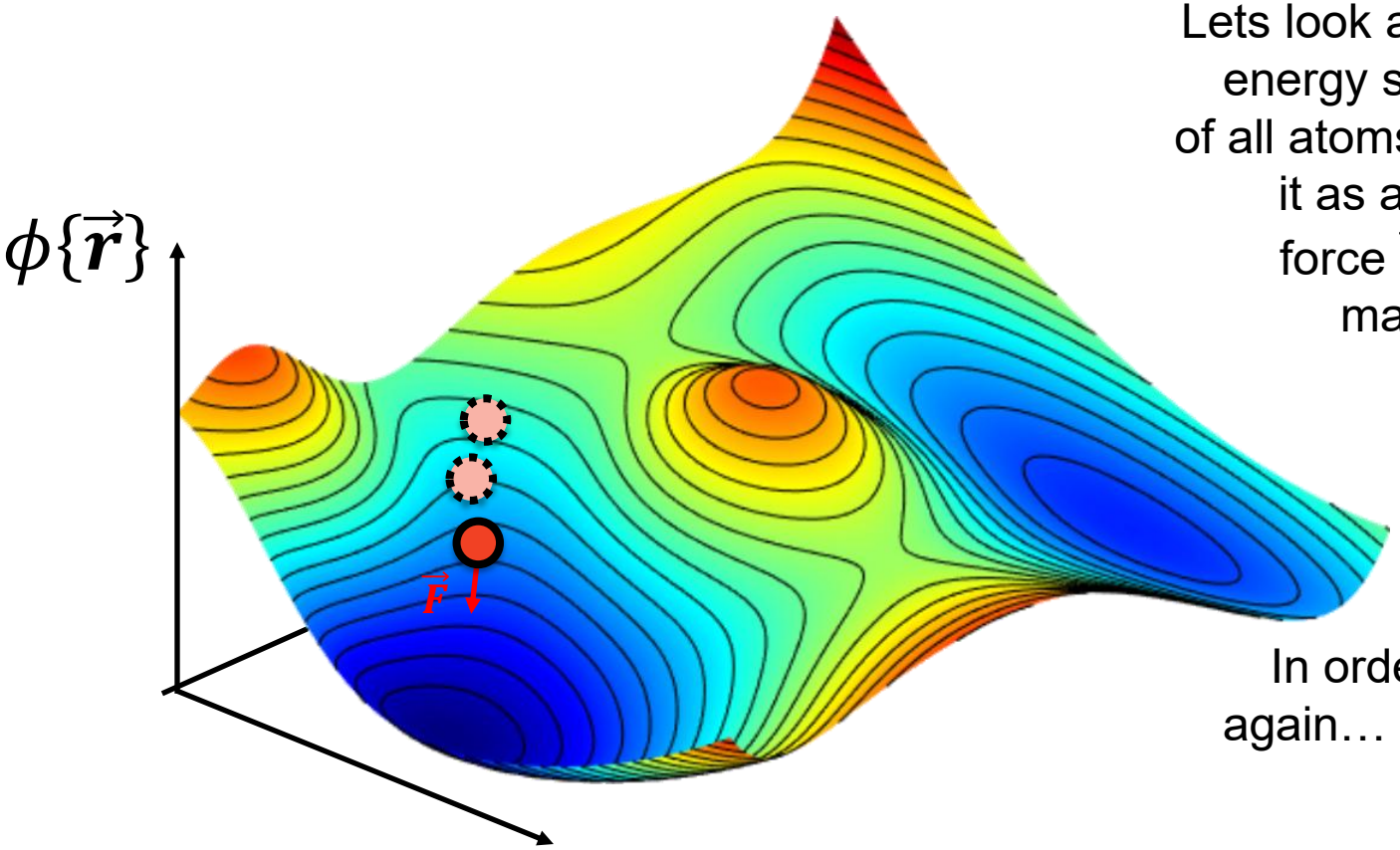
Local minimization: steepest-descent algorithm

Lets consider a configuration of N atoms at coordinates $\{\vec{r}_i\}$.

$$\vec{F}(k) = \sum_{j \neq k} [-\nabla_k \phi(r_{kj})]$$

If a configuration corresponds to a minimum of potential energy → equilibrium: $\vec{F}(k) = 0 \ \ \forall k$

For a generic configuration (such as the initial configuration of the MD simulation) the atoms are subject to non-zero forces. Forces are always directed opposite to the potential energy gradient so that they move atoms toward the (closer) energy minimum.



Lets look at one atom sitting at some point $\vec{r}_k$ on the potential energy surface (PES) determined by the (relative) positions of all atoms (it should be a $3N$ hypersurface but here we draw it as a 2D surface for simplicity). The atom is subject to a force $\vec{F}(k)$ directed toward the closest minimum and with magnitude proportional to how steep is the PES at $\vec{r}_k$.

Lets move "a bit" the atom along the direction of $\vec{F}$ (and do the same for all other atoms): we reach a point that is closer to the energy minimum!

In order to reach the minimum we just have to repeat this again…

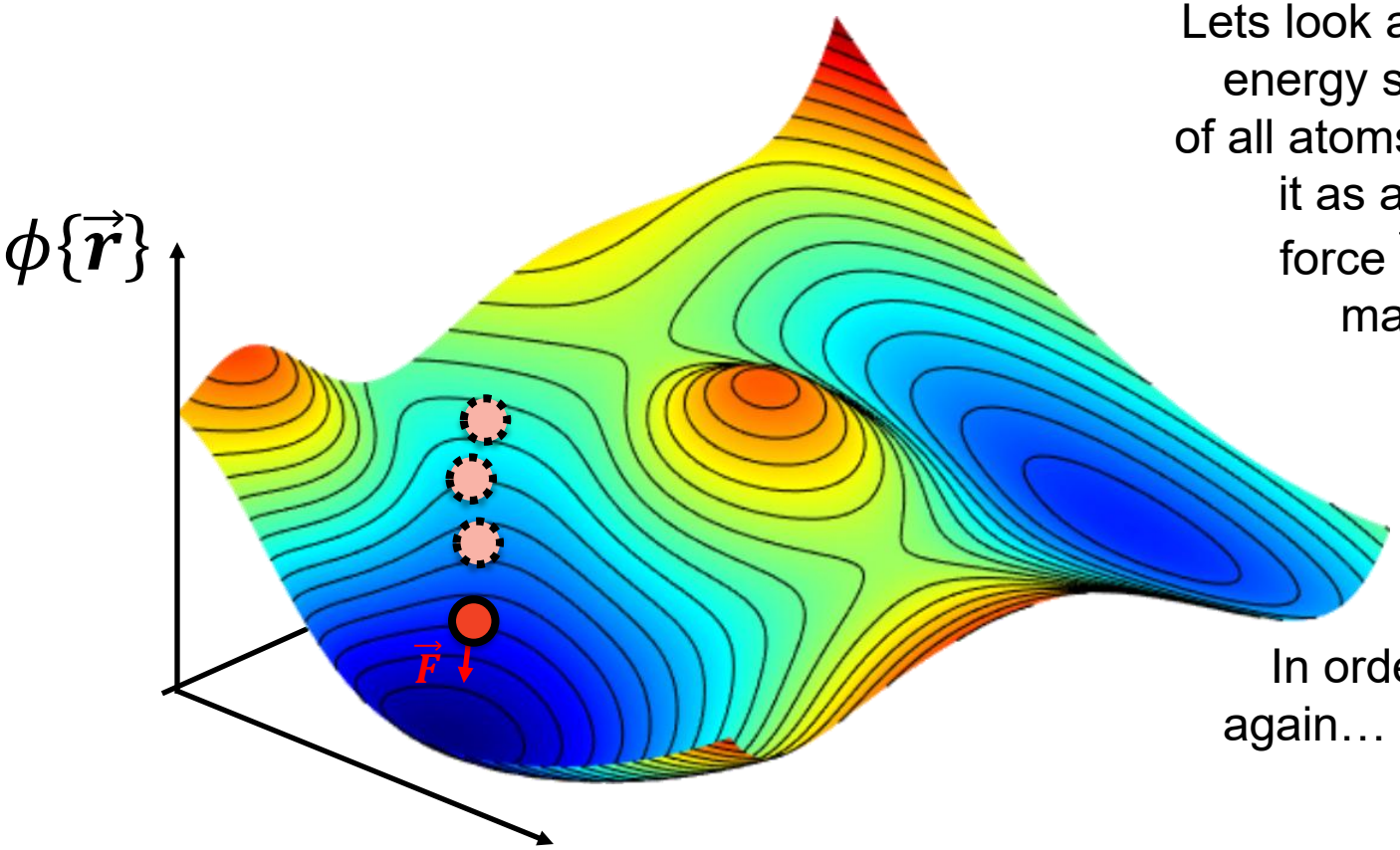
Local minimization: steepest-descent algorithm

Lets consider a configuration of N atoms at coordinates $\{\vec{r}_i\}$.

$$\vec{F}(k) = \sum_{j \neq k} [-\nabla_k \phi(r_{kj})]$$

If a configuration corresponds to a minimum of potential energy → equilibrium: $\vec{F}(k) = 0 \ \ \forall k$

For a generic configuration (such as the initial configuration of the MD simulation) the atoms are subject to non-zero forces. Forces are always directed opposite to the potential energy gradient so that they move atoms toward the (closer) energy minimum.



Lets look at one atom sitting at some point $\vec{r}_k$ on the potential energy surface (PES) determined by the (relative) positions of all atoms (it should be a $3N$ hypersurface but here we draw it as a 2D surface for simplicity). The atom is subject to a force $\vec{F}(k)$ directed toward the closest minimum and with magnitude proportional to how steep is the PES at $\vec{r}_k$.

Lets move "a bit" the atom along the direction of $\vec{F}$ (and do the same for all other atoms): we reach a point that is closer to the energy minimum!

In order to reach the minimum we just have to repeat this again… and again…

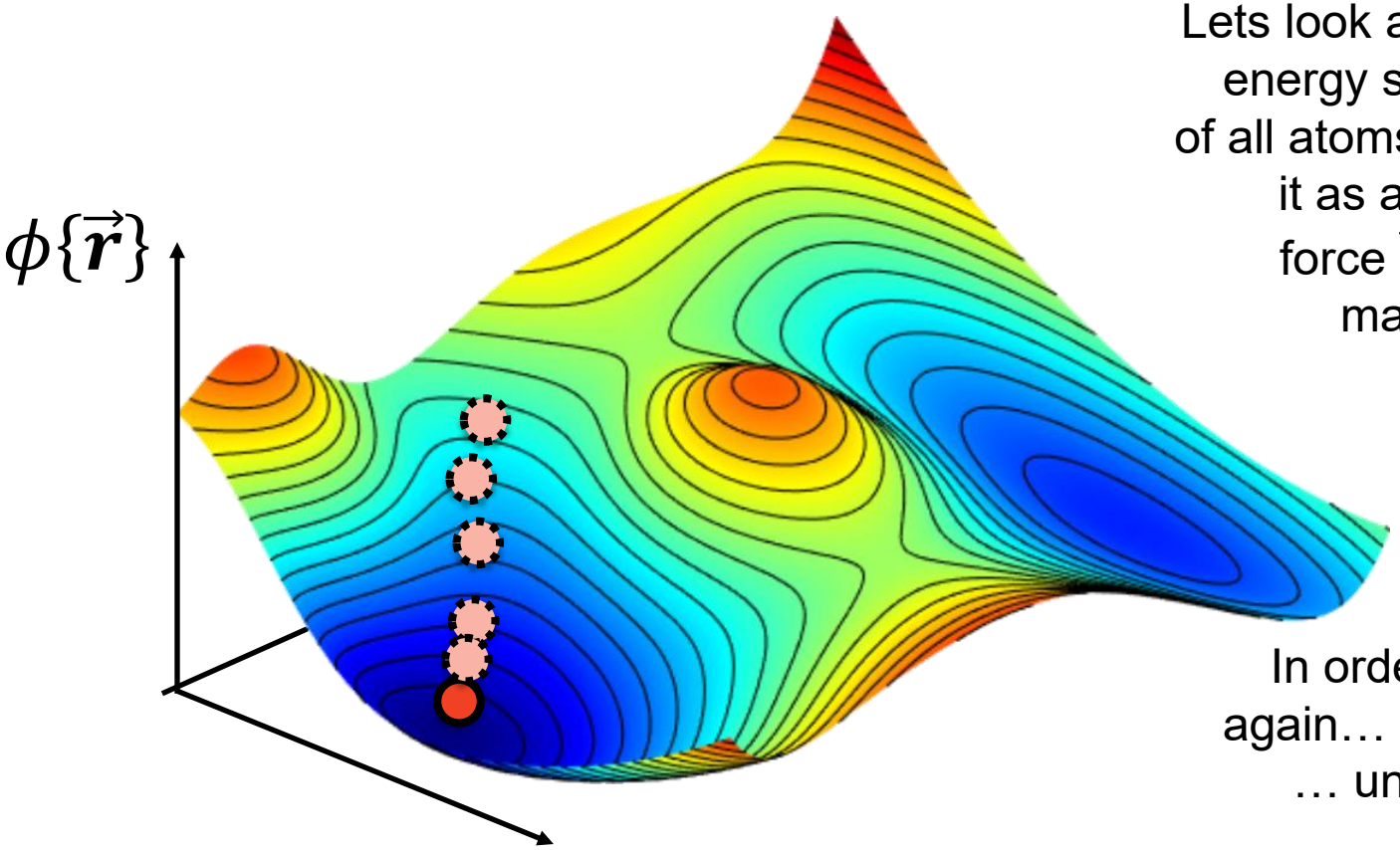
Local minimization: steepest-descent algorithm

Lets consider a configuration of N atoms at coordinates $\{\vec{r}_i\}$.

$$\vec{F}(k) = \sum_{j \neq k} [-\nabla_k \phi(r_{kj})]$$

If a configuration corresponds to a minimum of potential energy → equilibrium: $\vec{F}(k) = 0 \ \ \forall k$

For a generic configuration (such as the initial configuration of the MD simulation) the atoms are subject to non-zero forces. Forces are always directed opposite to the potential energy gradient so that they move atoms toward the (closer) energy minimum.



Lets look at one atom sitting at some point $\vec{r}_k$ on the potential energy surface (PES) determined by the (relative) positions of all atoms (it should be a $3N$ hypersurface but here we draw it as a 2D surface for simplicity). The atom is subject to a force $\vec{F}(k)$ directed toward the closest minimum and with magnitude proportional to how steep is the PES at $\vec{r}_k$.

Lets move "a bit" the atom along the direction of $\vec{F}$ (and do the same for all other atoms): we reach a point that is closer to the energy minimum!

In order to reach the minimum we just have to repeat this again… and again…
… until we have arrived "close enough to the minimum".

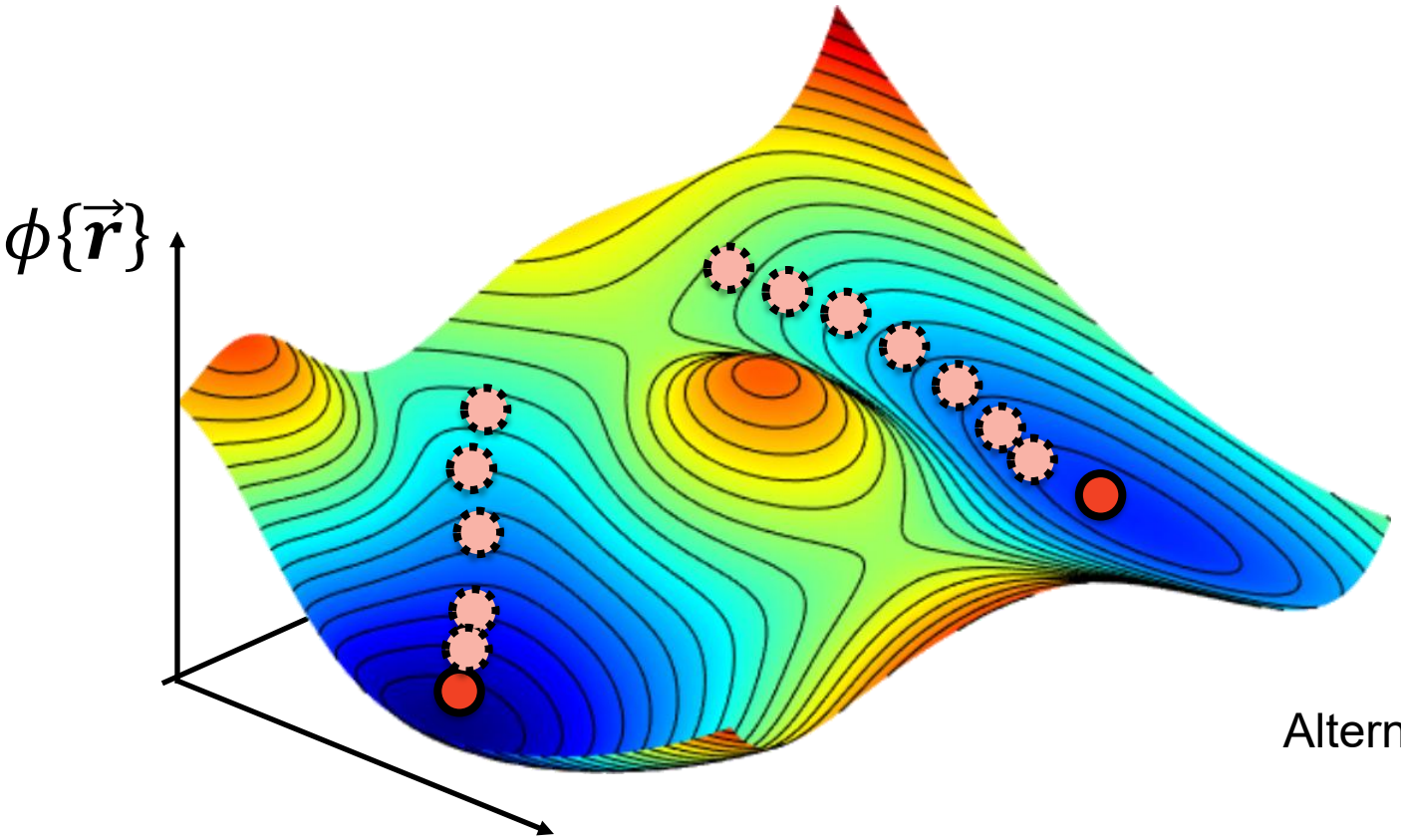
Local minimization: steepest-descent algorithm

Lets consider a configuration of N atoms at coordinates $\{\vec{r}_i\}$.

$$\vec{F}(k) = \sum_{j \neq k} [-\nabla_k \phi(r_{kj})]$$

If a configuration corresponds to a minimum of potential energy → equilibrium: $\vec{F}(k) = 0 \ \ \forall k$

For a generic configuration (such as the initial configuration of the MD simulation) the atoms are subject to non-zero forces. Forces are always directed opposite to the potential energy gradient so that they move atoms toward the (closer) energy minimum.



We only converge toward the closer local minimum!!!
To find other minima change the initial configuration

Alternative approaches for multi-dimensional minimization, e.g. conjugate gradient…

Local minimization: steepest-descent algorithm

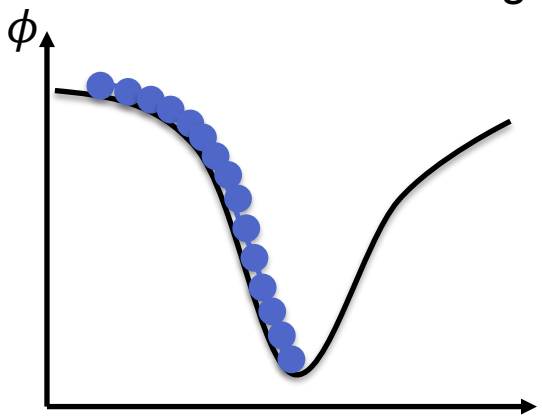
Step size

How much we move the atoms? A "bit"…

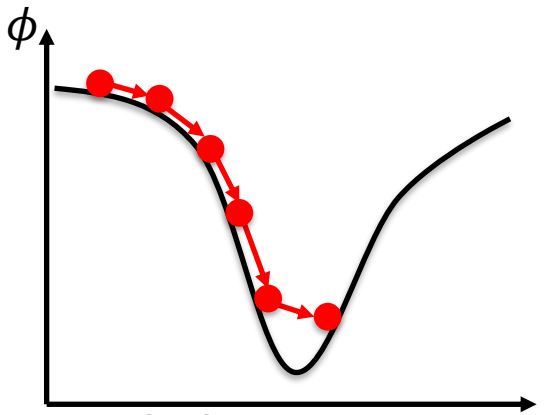
… surely proportionally to $\vec{F}$

$$\delta\vec{r}(k) = C \cdot \vec{F}(k)$$

- Constant C: how large?



Small C: possibly huge number of tiny steps



Large C: if not crashing we could not converge or bounch back and forth around the minimum

- Adaptive C

Convergency criterion

When we do consider that we are "close enough to the minimum" to stop?

$$\max(F) \leq F_{tol}$$

F_{tol} is the threshold value for which we consider that a force is negligible ($F_{tol} \sim 10^{-5}$ eV/Å)

$$\max(F) = \max_{k=1:N} \left(\sqrt{F_x^2(\boldsymbol{k}) + F_y^2(\boldsymbol{k}) + F_z^2(\boldsymbol{k})} \right)$$

Local minimization: steepest-descent algorithm



We construct an algorithm that evolves the configuration following only the forces until convergence is reached.

```
# initial/given configuration pos
Csteep=0.005
ftoll=1.e-3

for iStep in range(0,maxStep)      # we don't want to run the loop forever if not converging

    # compute neighbors lists
    forces = calcForces(pos, nNbrs, whichNbrs)

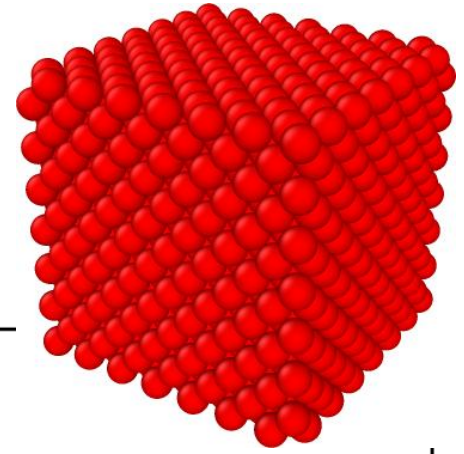
    maxf = np.max( np.sqrt(forces[:,0]**2 + forces[:,1]**2 + forces[:,2]**2) )
    if(maxf < ftoll):
       break        # converged! We are done and exit the loop
    else:
       pos = pos + Csteep * forces
```

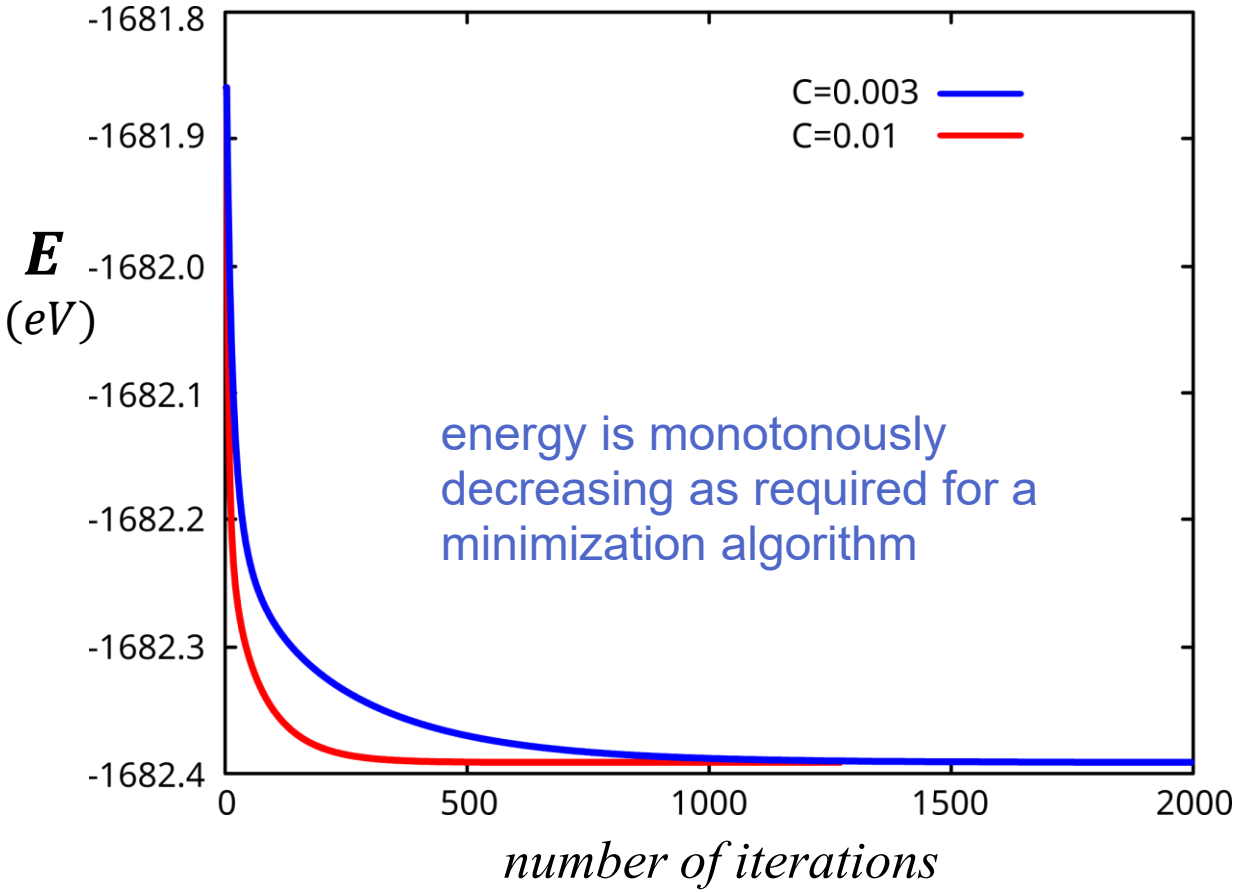
Is it working?

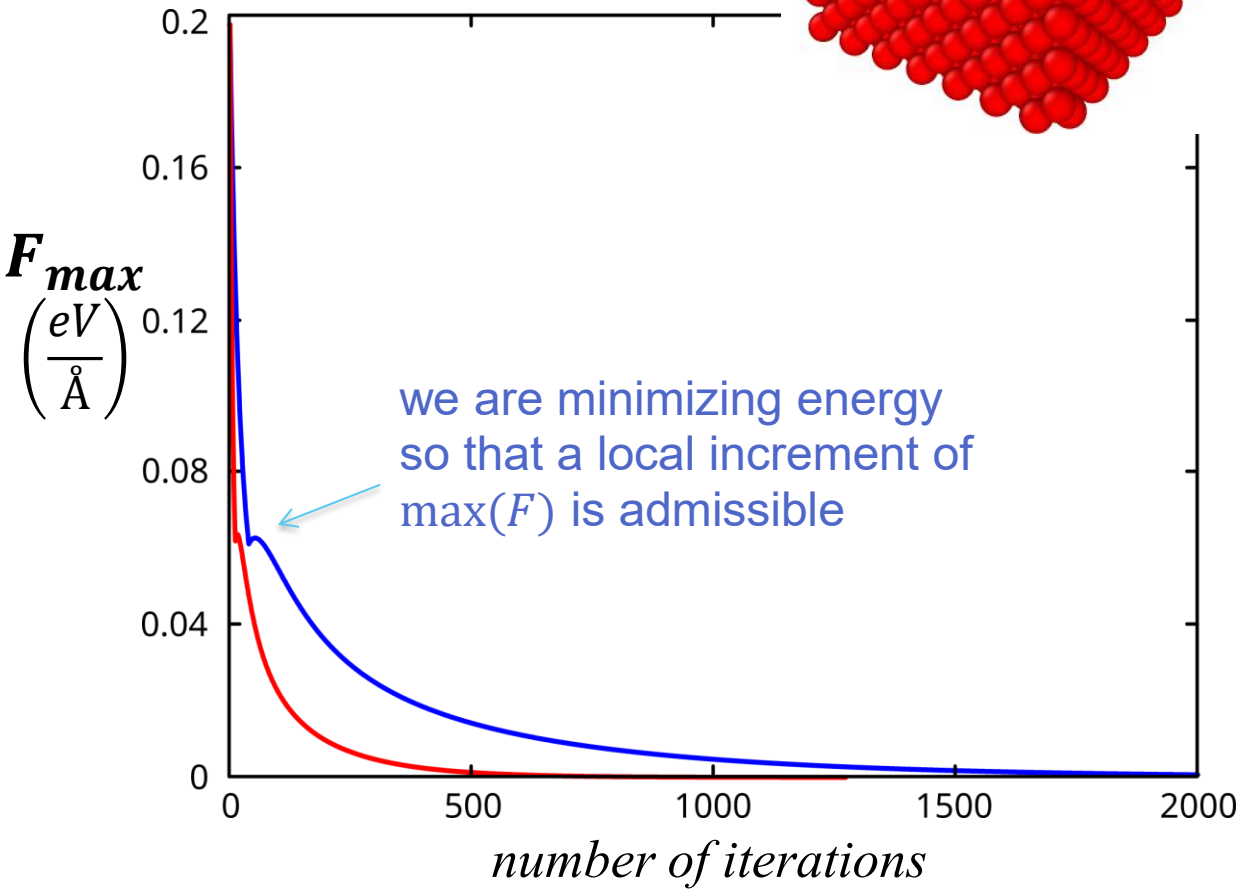
- Compute the (potential) energy and check it is always decreasing
- Check that also the force is decreasing (local increment are possible)

Steepest-descent on the 864 atoms cell

Here below an example of the steepest procedure for a fcc Ag cluster of 864 atoms. During the process atomic coordinates are very slightly changing.







Simulating a bulk: periodic boundary conditions

From a finite number of atoms to bulk-like structures

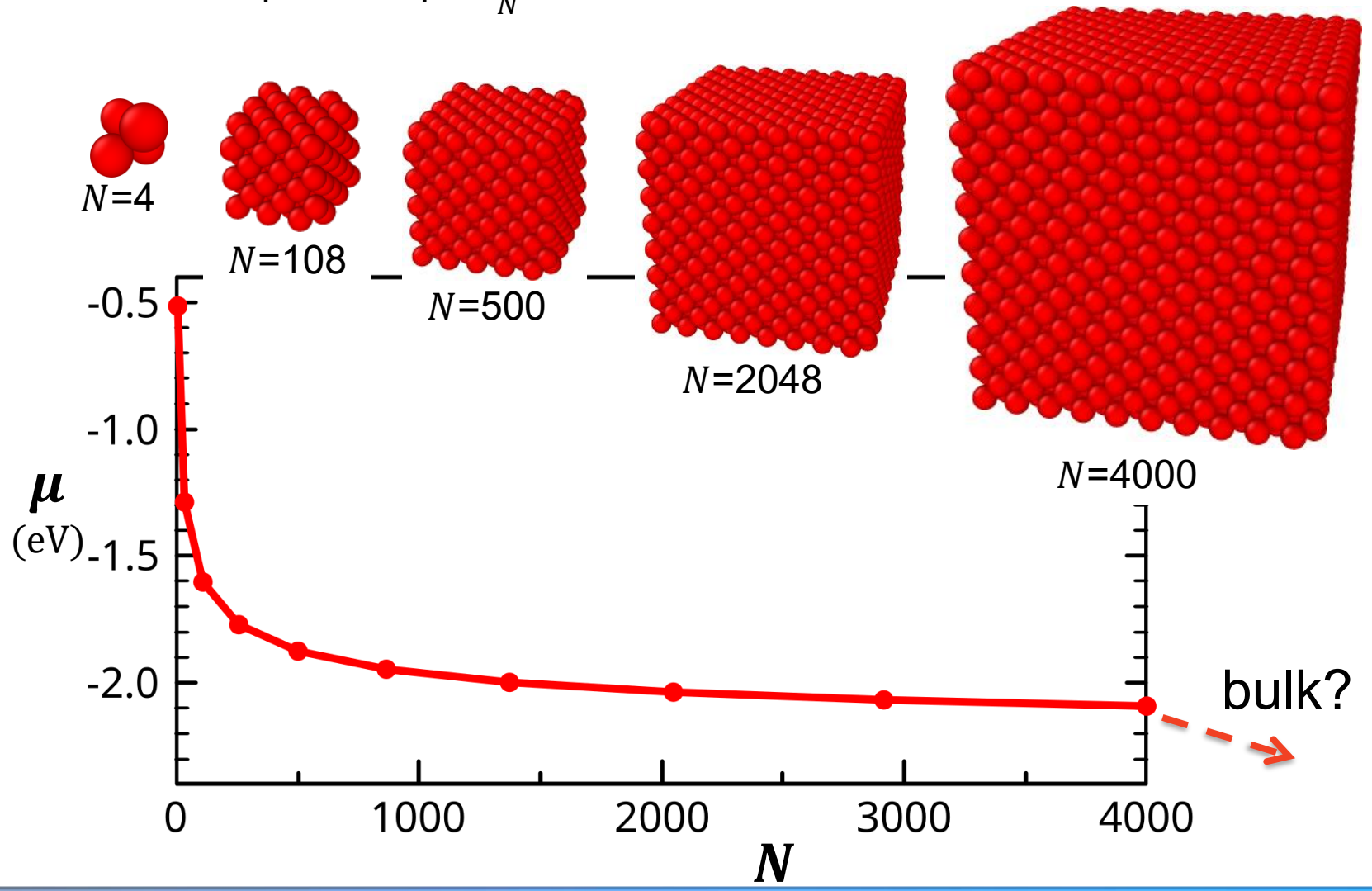
What if we want to study an “infinite” bulk? Can we take it as a very large cluster?

Let us look to the energy per atom, i.e. the chemical potential $\mu = \frac{E}{N}$ for clusters of different size (after minimization).

Small clusters have several undercoordinated atoms due to the large surface-to-volume S/V ratio:

$$\left.\begin{aligned} V &\propto N \\ S &\propto N^{2/3} \end{aligned}\right\} \quad \boxed{\frac{S}{V} \propto N^{-1/3}}$$

At any finite cell size surface effects are to be considered

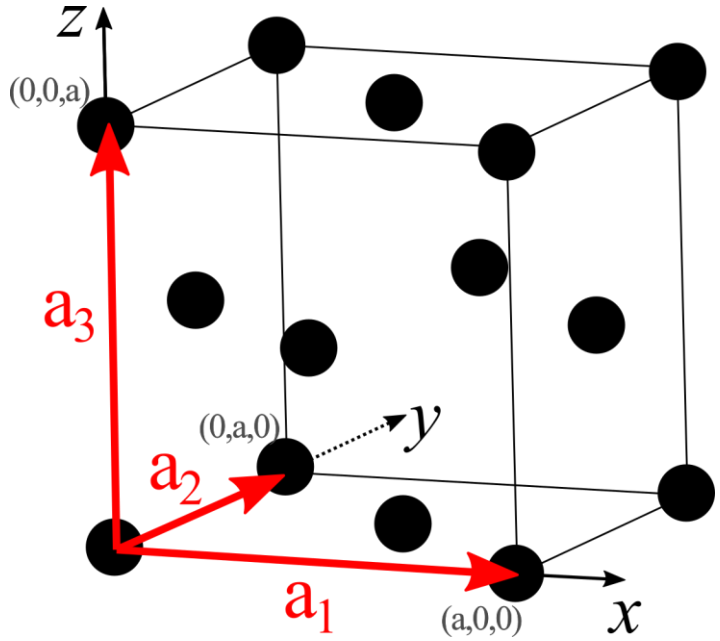


From a finite number of atoms to bulk-like structures

What if we want to study an “infinite” bulk? Can we take it as a very large cluster?

What is the energy for the perfect bulk fcc crystal?

Lets start from the conventional unit cell of fcc lattice.



From a finite number of atoms to bulk-like structures

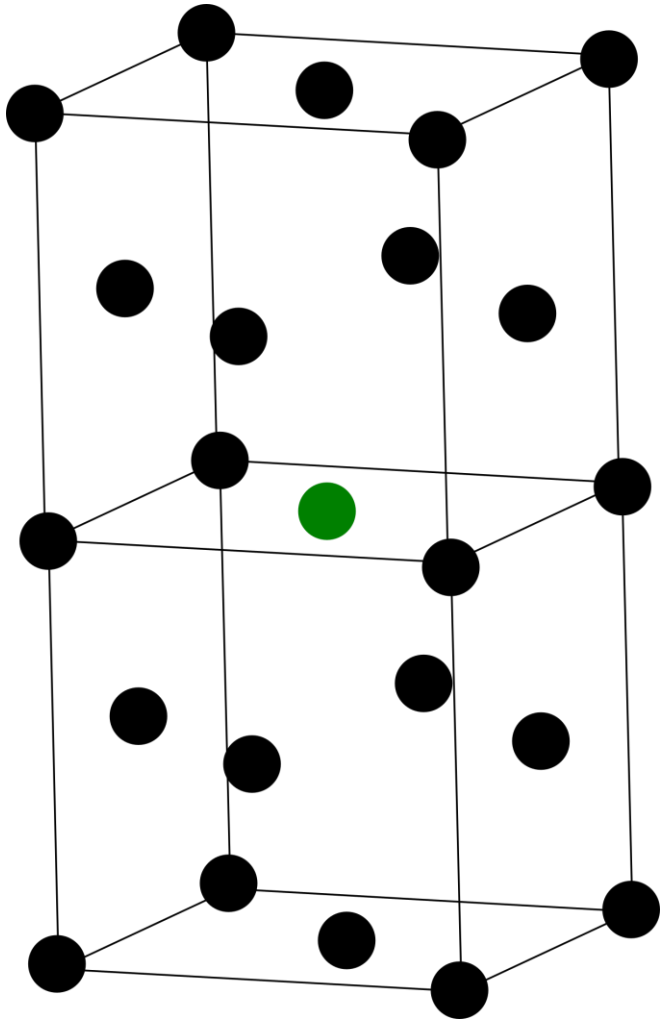
What if we want to study an “infinite” bulk? Can we take it as a very large cluster?

What is the energy for the perfect bulk fcc crystal?

Lets start from the conventional unit cell of fcc lattice.

Given our cutoff radius r_c, both 1st and 2nd neighbors contribute to the potential energy (LJ).

To find them it is convenient to look at two units on top of each other and look at the central atom.



From a finite number of atoms to bulk-like structures

What if we want to study an “infinite” bulk? Can we take it as a very large cluster?

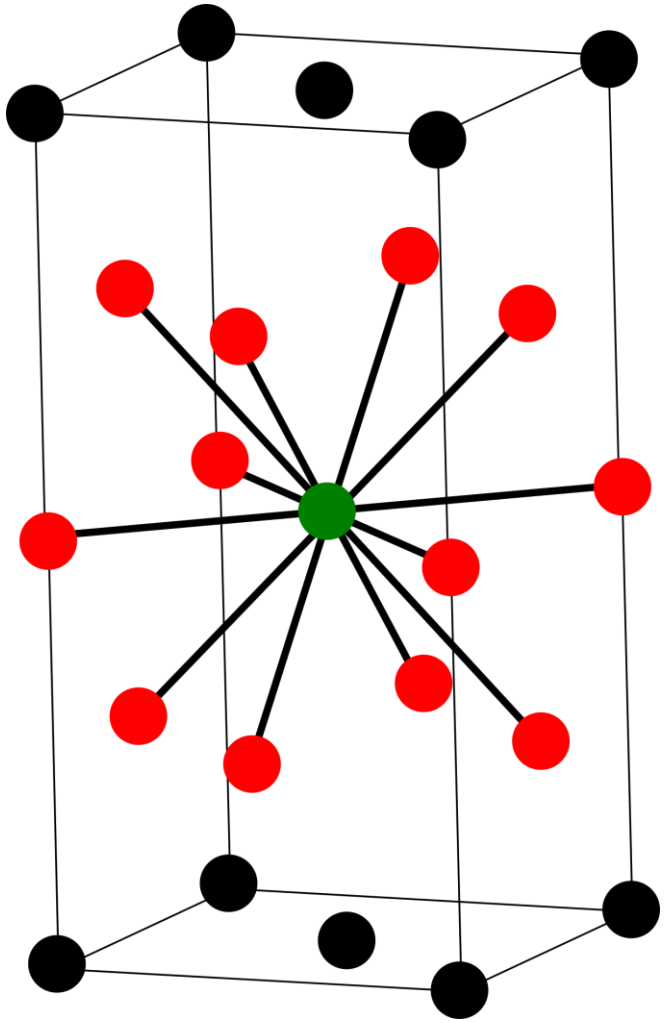
What is the energy for the perfect bulk fcc crystal?

Lets start from the conventional unit cell of fcc lattice.

Given our cutoff radius r_c, both 1st and 2nd neighbors contribute to the potential energy (LJ).

To find them it is convenient to look at two units on top of each other and look at the central atom.

– 12 first neighbors (along <110> directions): $r_1 = \frac{\sqrt{2}}{2} a$



From a finite number of atoms to bulk-like structures

What if we want to study an “infinite” bulk? Can we take it as a very large cluster?

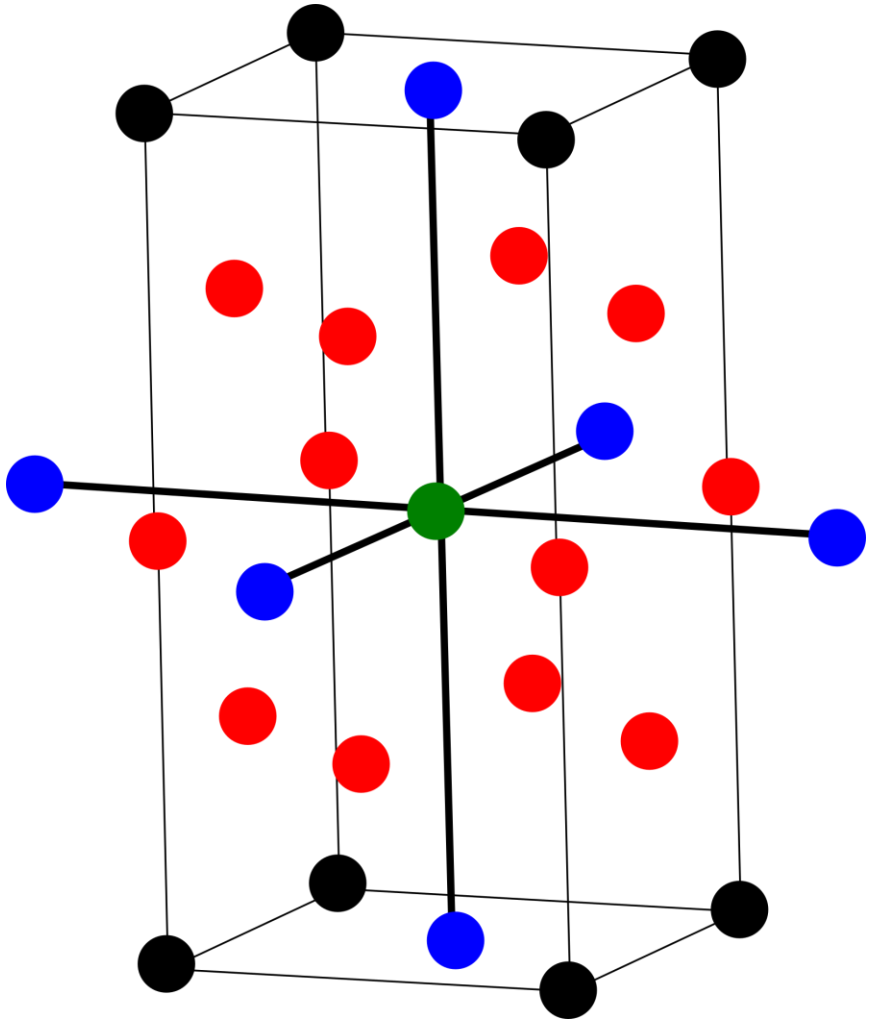
What is the energy for the perfect bulk fcc crystal?

Lets start from the conventional unit cell of fcc lattice.

Given our cutoff radius r_c, both 1st and 2nd neighbors contribute to the potential energy (LJ).

To find them it is convenient to look at two units on top of each other and look at the central atom.

- 12 first neighbors (along <110> directions): $r_1 = \frac{\sqrt{2}}{2} a$
- 6 second neighbors (along <100> directions): $r_2 = a$



From a finite number of atoms to bulk-like structures

What if we want to study an “infinite” bulk? Can we take it as a very large cluster?

What is the energy for the perfect bulk fcc crystal?

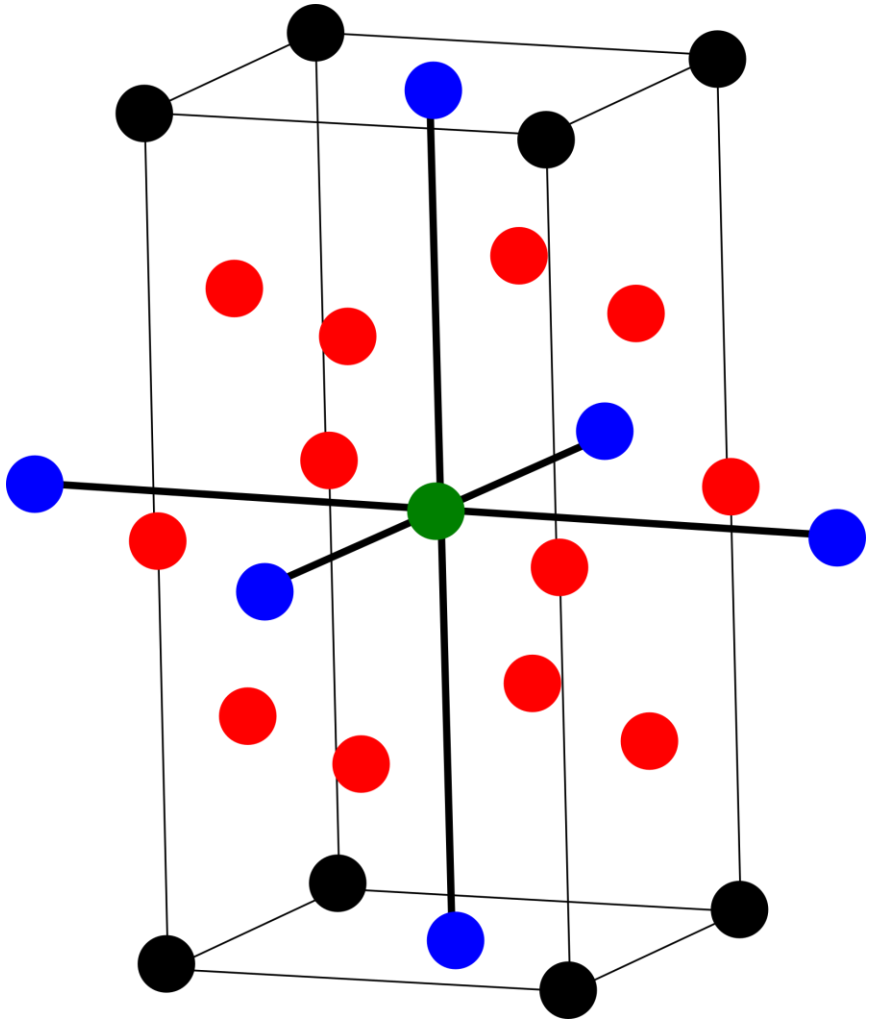
Lets start from the conventional unit cell of fcc lattice.

Given our cutoff radius r_c, both 1st and 2nd neighbors contribute to the potential energy (LJ).

To find them it is convenient to look at two units on top of each other and look at the central atom.

- 12 first neighbors (along <110> directions): $r_1 = \frac{\sqrt{2}}{2} a$
- 6 second neighbors (along <100> directions): $r_2 = a$

$$\mu_{bulk} = \frac{1}{2}\left[12\,\phi_{LJ}(r_1) + 6\,\phi_{LJ}(r_2)\right] = -2.32\ eV$$

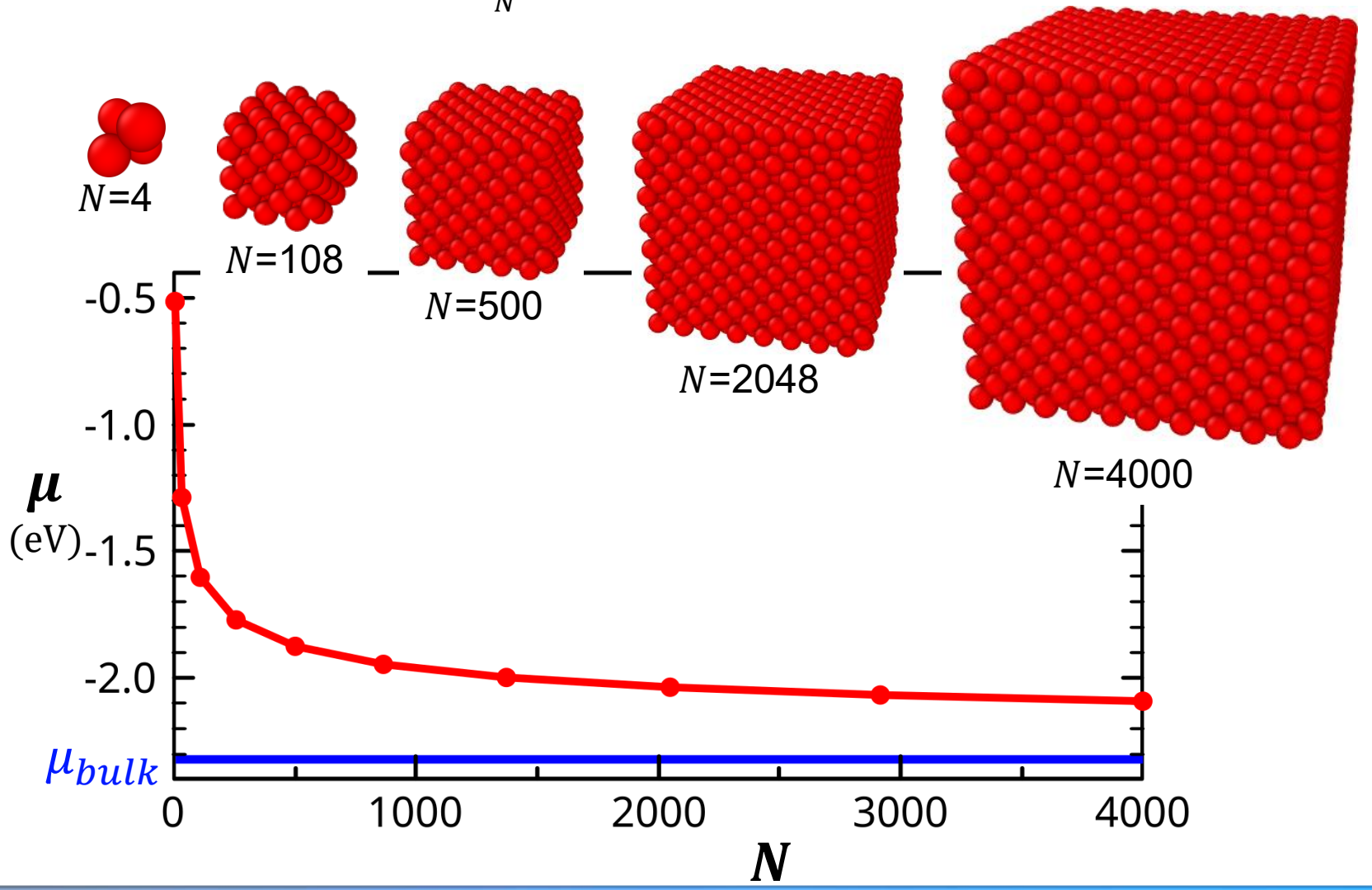


From a finite number of atoms to bulk-like structures

What if we want to study an “infinite” bulk? Can we take it as a very large cluster?

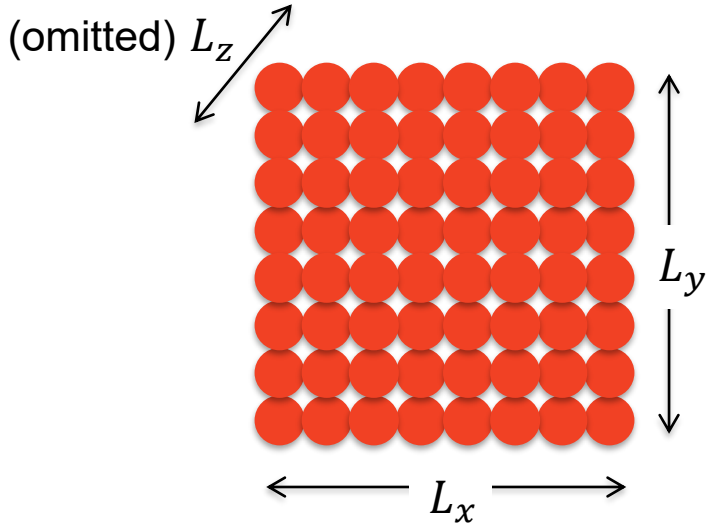
Let us look to the energy per atom, i.e. the chemical potential $\mu = \frac{E}{N}$ for clusters of different size (after minimization).

Huge clusters are needed to match the real bulk-like crystal...



Periodic boundary conditions

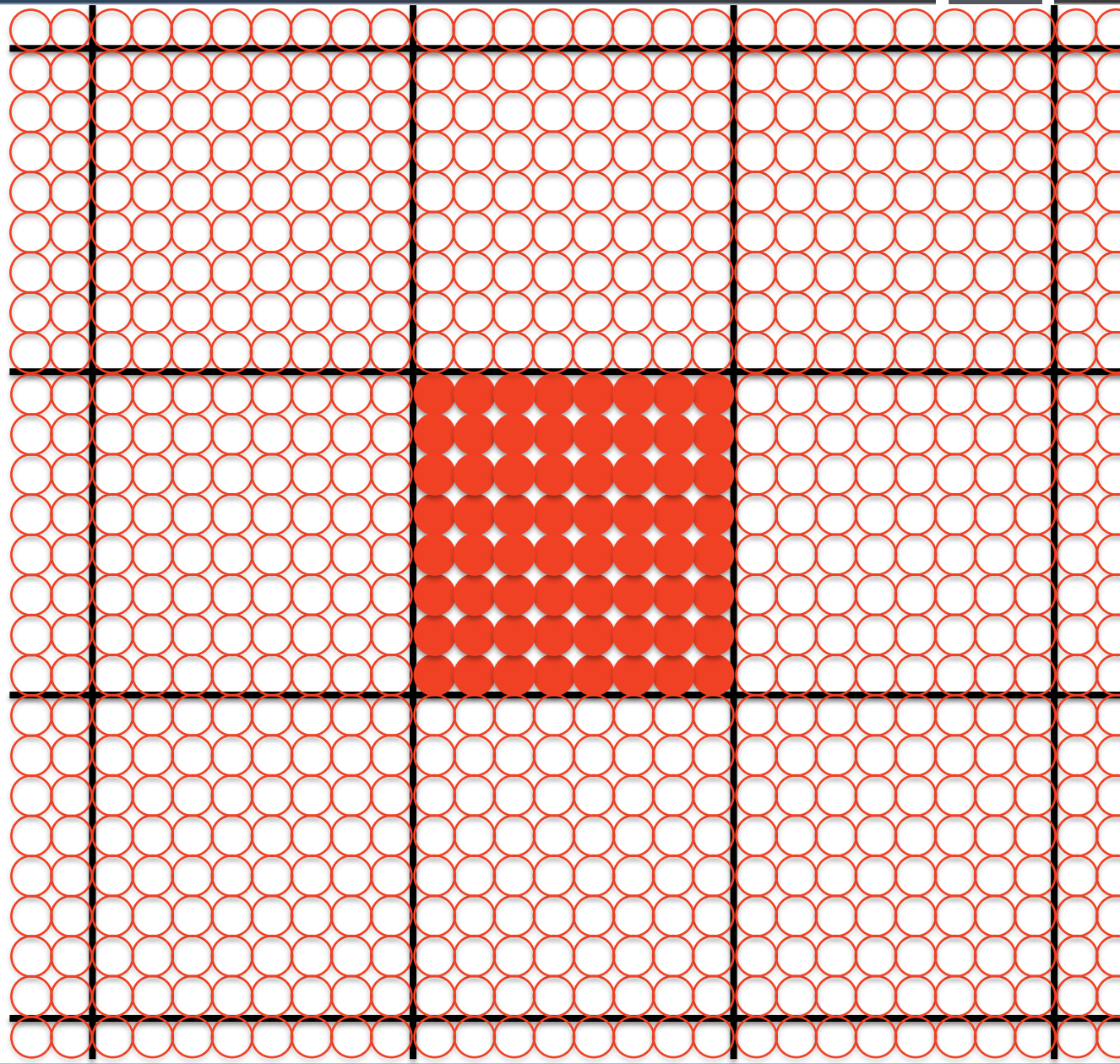
Lets assume that our simulation cell is a representative portion of a infinitely large bulk so that what happens inside it is the same, on average, to what happens in any other region of the same volume $(L_x \times L_y \times L_z)$.



Periodic boundary conditions

Lets assume that our simulation cell is a representative portion of a infinitely large bulk so that what happens inside it is the same, on average, to what happens in any other region of the same volume $(L_x \times L_y \times L_z)$.

→ we can fill the entire bulk by repeating the simulation cell as a periodic unit in a lattice

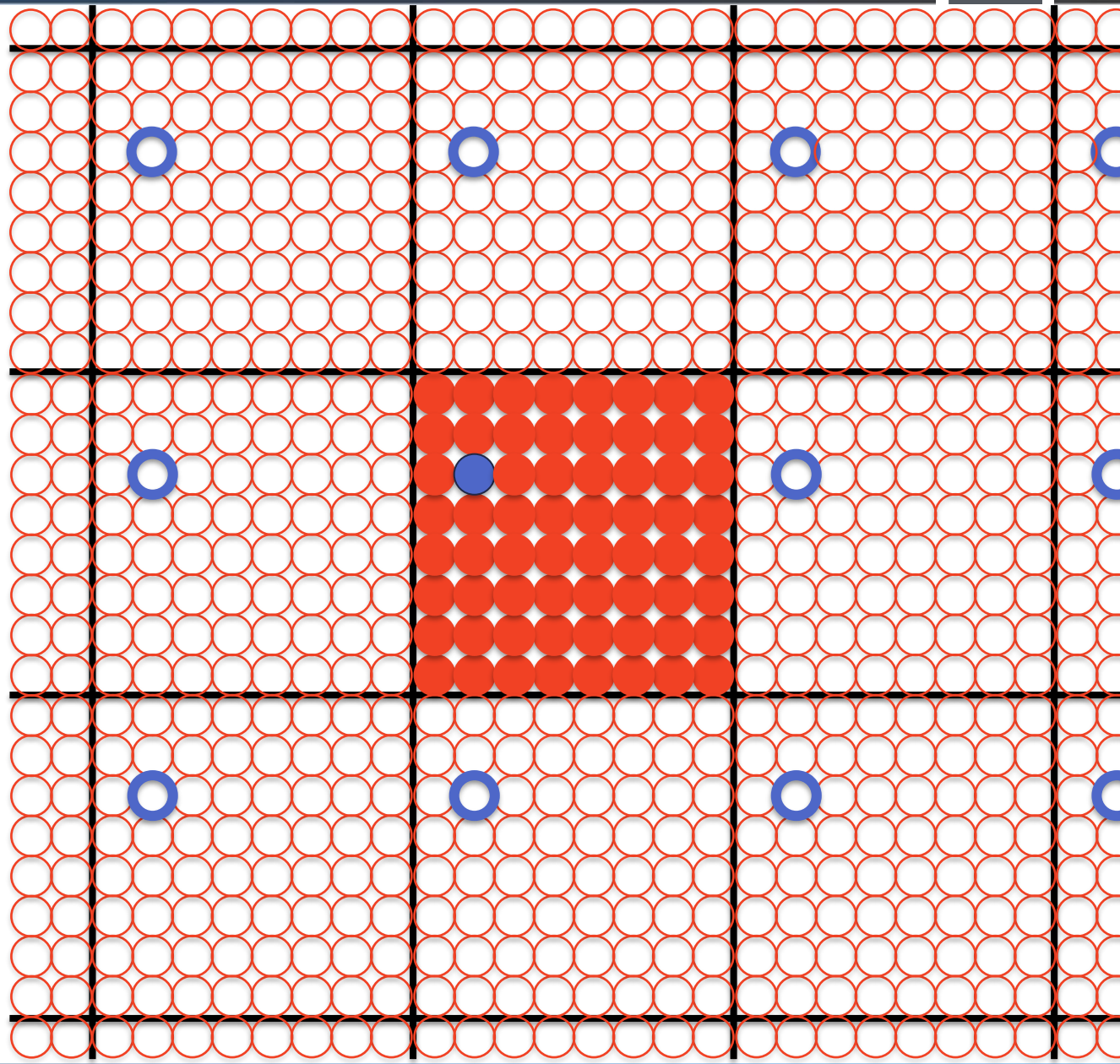


Periodic boundary conditions

Lets assume that our simulation cell is a representative portion of a infinitely large bulk so that what happens inside it is the same, on average, to what happens in any other region of the same volume $(L_x \times L_y \times L_z)$.

→ we can fill the entire bulk by repeating the simulation cell as a periodic unit in a lattice

$$(x, y, z) \rightarrow (x + n_x L_x, y + n_y L_y, z + n_z L_z)$$



Periodic boundary conditions

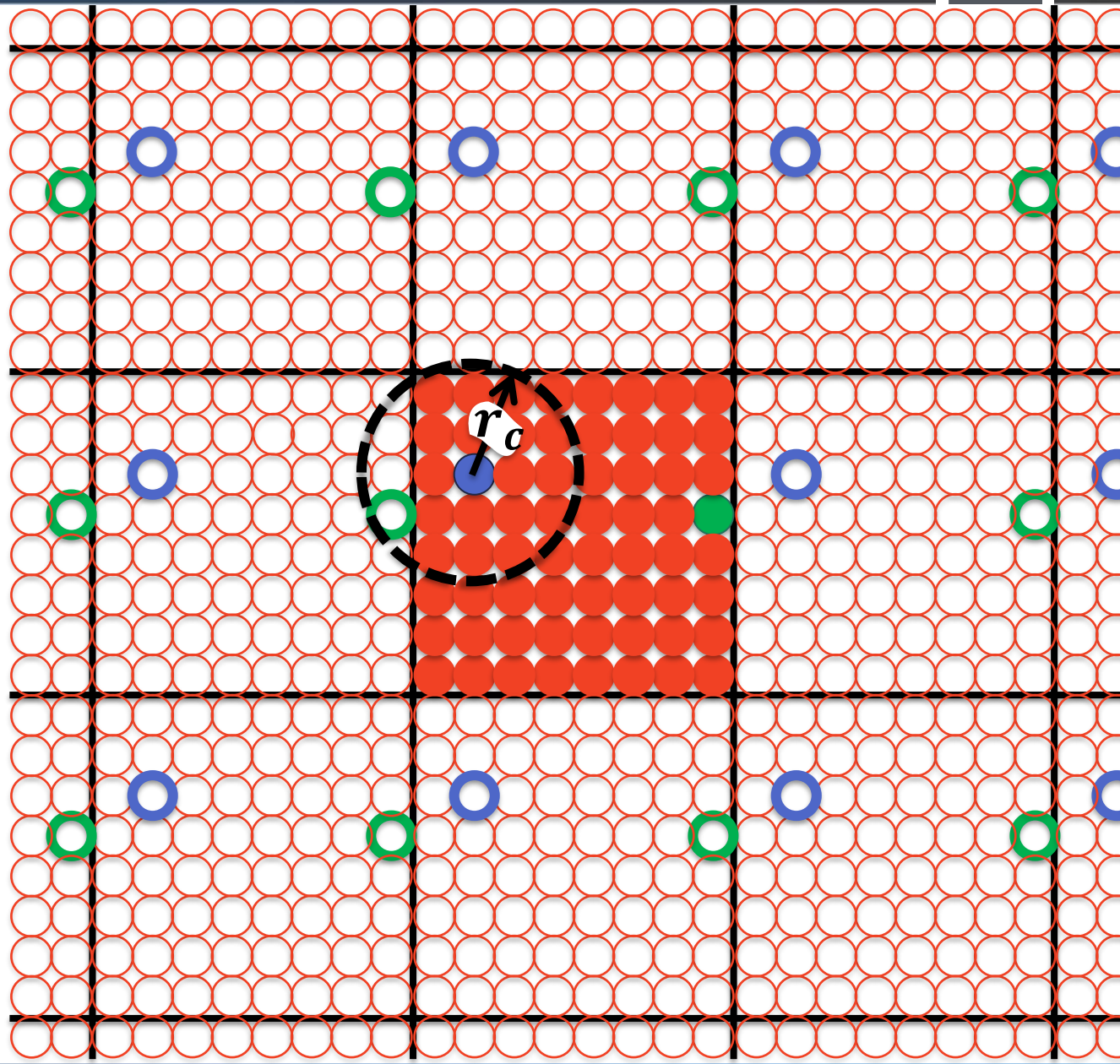
Lets assume that our simulation cell is a representative portion of a infinitely large bulk so that what happens inside it is the same, on average, to what happens in any other region of the same volume $(L_x \times L_y \times L_z)$.

→ we can fill the entire bulk by repeating the simulation cell as a periodic unit in a lattice

$$(x, y, z) \rightarrow (x + n_x L_x, y + n_y L_y, z + n_z L_z)$$

Atoms that are far apart in the original simulation cell, i.e. $r_{kj} > r_c$, may actually interact with each other trough their images!

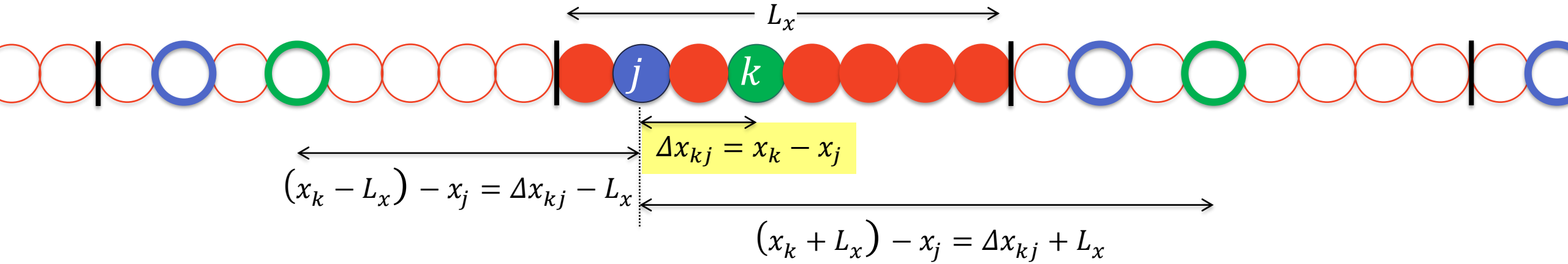
→ the actual r_{kj}^{pbc} is the minimum of all distances between the atom j and all images of atom k.



Periodic boundary conditions: interatomic distances

r_{kj}^{pbc} is the minimum of all distances between the atom j and all images of atom k.

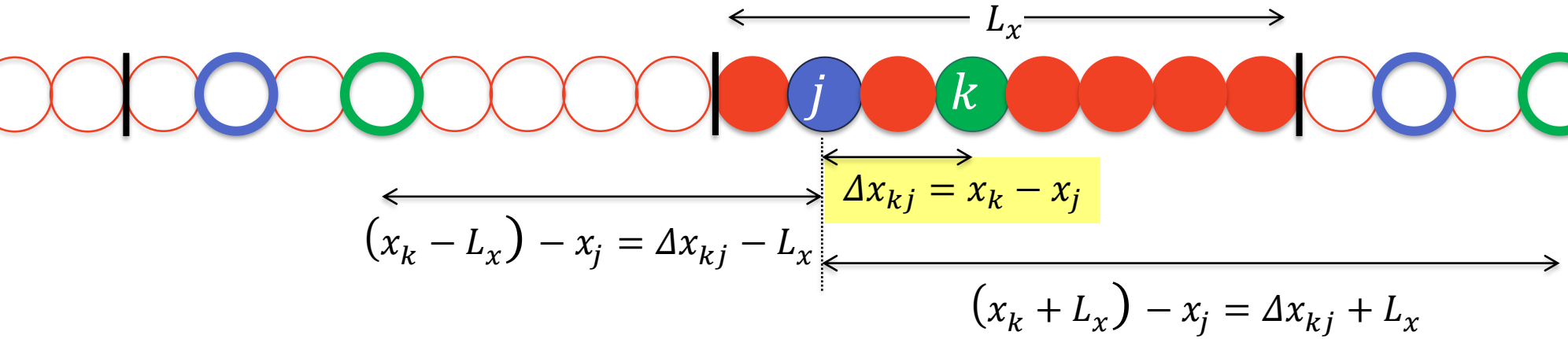
How do we compute it? Let us first look to a 1D atomic chain



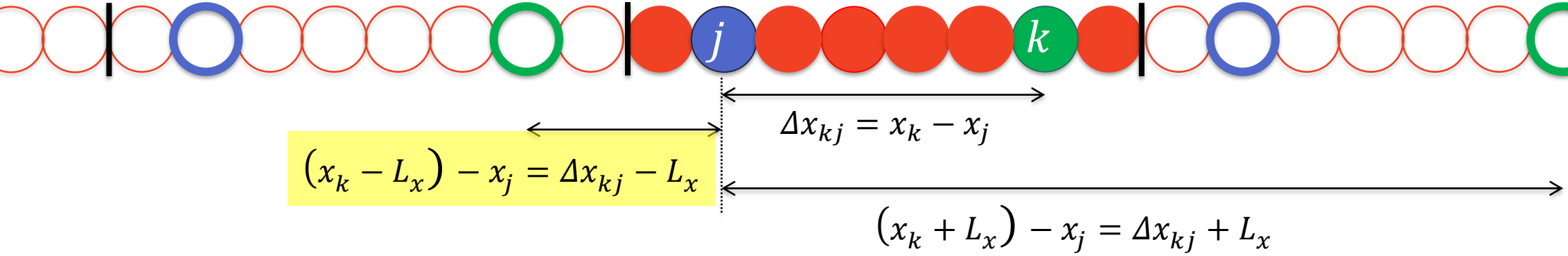
Periodic boundary conditions: interatomic distances

r_{kj}^{pbc} is the minimum of all distances between the atom j and all images of atom k.

How do we compute it? Let us first look to a 1D atomic chain



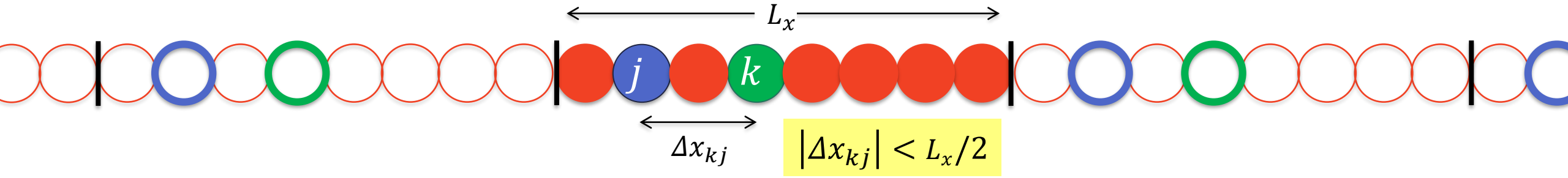
Another case:

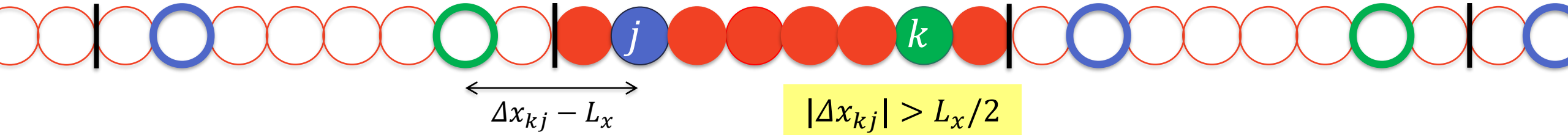


Periodic boundary conditions: interatomic distances

r_{kj}^{pbc} is the minimum of all distances between the atom j and all images of atom k.

How do we compute it? Let us first look to a 1D atomic chain



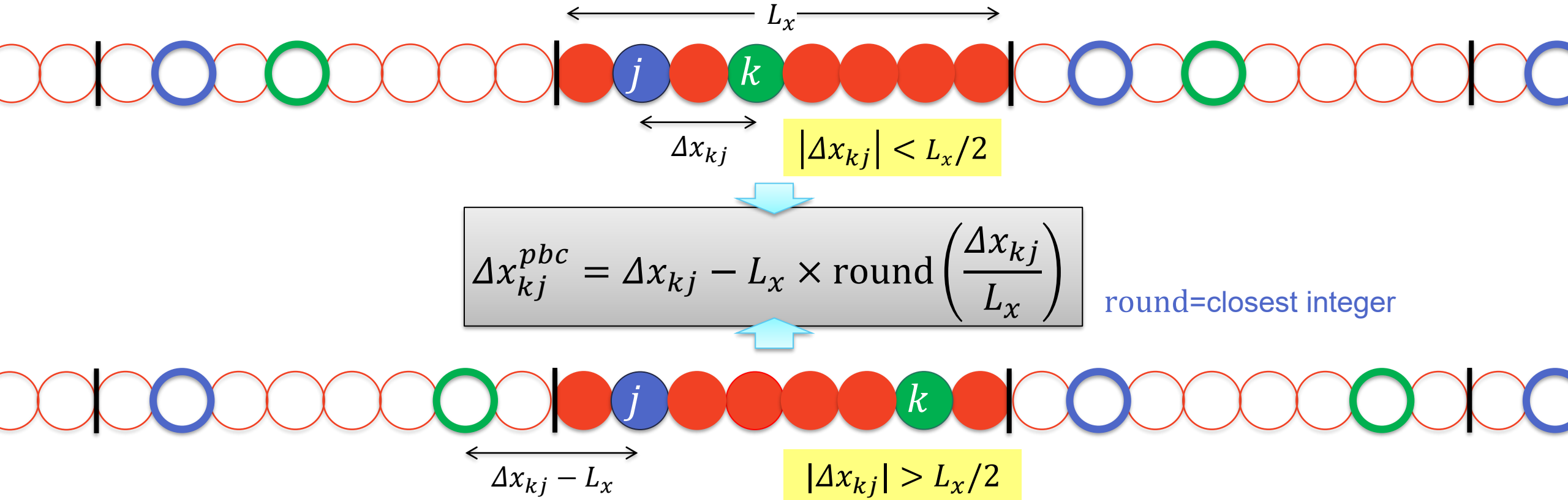


same situation also if $x_k < x_j$

Periodic boundary conditions: interatomic distances

r_{kj}^{pbc} is the minimum of all distances between the atom j and all images of atom k.

How do we compute it? Let us first look to a 1D atomic chain



$$\Delta x_{kj}^{pbc} = \Delta x_{kj} - L_x \times \text{round}\left(\frac{\Delta x_{kj}}{L_x}\right)$$

round=closest integer

same situation also if $x_k < x_j$

Periodic boundary conditions: interatomic distances

r_{kj}^{pbc} is the minimum of all distances between the atom j and all images of atom k.

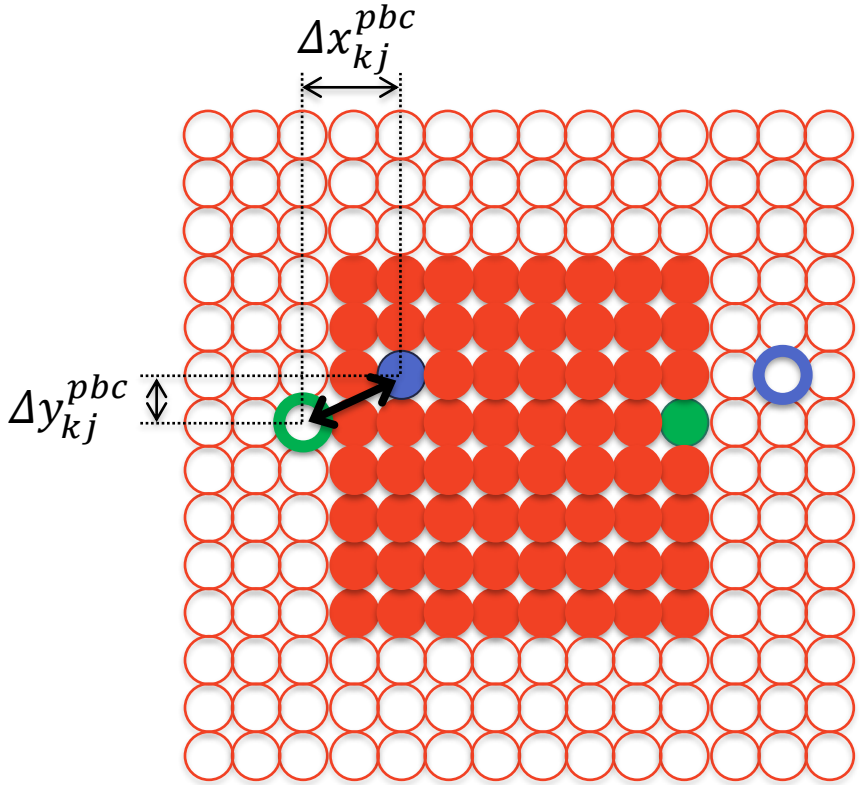
The result for the 1D case can be applied to each component of the 3D r_{kj}^{pbc} distance, so that:

$$\Delta x_{kj}^{pbc} = \Delta x_{kj} - L_x \times \text{round}\left(\frac{\Delta x_{kj}}{L_x}\right)$$

$$\Delta y_{kj}^{pbc} = \Delta y_{kj} - L_y \times \text{round}\left(\frac{\Delta y_{kj}}{L_y}\right)$$

$$\Delta z_{kj}^{pbc} = \Delta z_{kj} - L_z \times \text{round}\left(\frac{\Delta z_{kj}}{L_z}\right)$$

$$r_{kj}^{pbc} = \sqrt{\Delta x_{kj}^{pbc\,2} + \Delta y_{kj}^{pbc\,2} + \Delta z_{kj}^{pbc\,2}}$$



Periodic boundary conditions: interatomic distances

r_{kj}^{pbc} is the minimum of all distances between the atom j and all images of atom k.

The result for the 1D case can be applied to each component of the 3D r_{kj}^{pbc} distance, so that:

$$\Delta x_{kj}^{pbc} = \Delta x_{kj} - L_x \times \text{round}\left(\frac{\Delta x_{kj}}{L_x}\right)$$

$$\Delta y_{kj}^{pbc} = \Delta y_{kj} - L_y \times \text{round}\left(\frac{\Delta y_{kj}}{L_y}\right)$$

$$\Delta z_{kj}^{pbc} = \Delta z_{kj} - L_z \times \text{round}\left(\frac{\Delta z_{kj}}{L_z}\right)$$

$$r_{kj}^{pbc} = \sqrt{\Delta x_{kj}^{pbc\,2} + \Delta y_{kj}^{pbc\,2} + \Delta z_{kj}^{pbc\,2}}$$

In order to implement PBC it is then sufficient to update the evaluation of all distance components `dx,dy,dz` within the code:

```
dx=x(k)-x(j)
dx=dx-Lx*np.round(dx/Lx)
```

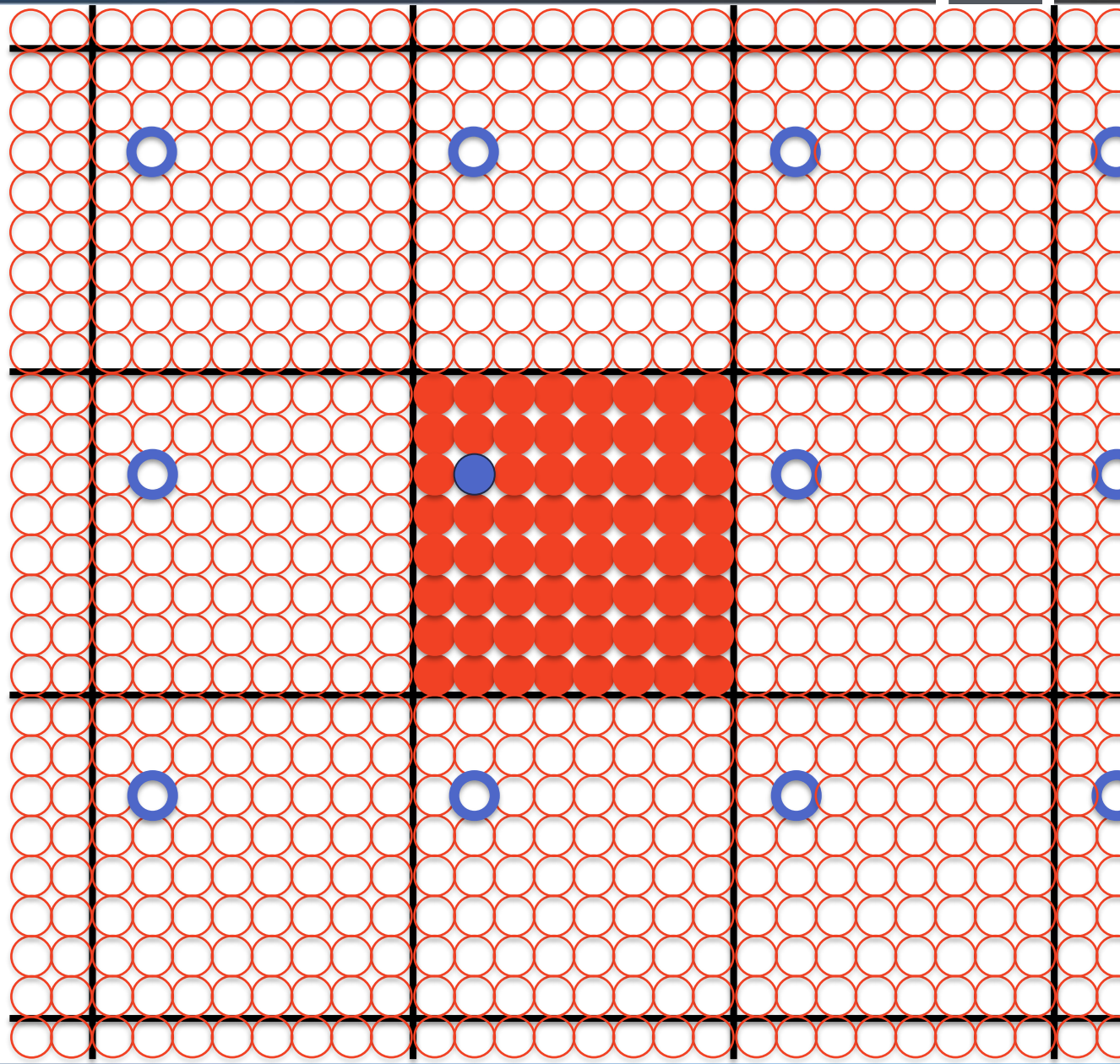
Functions involving computing distances

- assign neighbor list
- calculation of energy
- calculation of forces

Periodic boundary conditions is not the same as real bulk!

If we look at the ordered state of a crystal lattice, applying pbc returns the perfect bulk!

- infinite system → no rotations
- reset $\vec{v}_{\mathrm{CM}} = 0$ → no translations
- lattice positions correspond to the energy minimum
  → no change by steepest-descent
  → energy equipartition
- the volume of the cell is fixed
  → no thermal expansion (atoms can't increase their spacing as they would press against those in adjacent replicas)

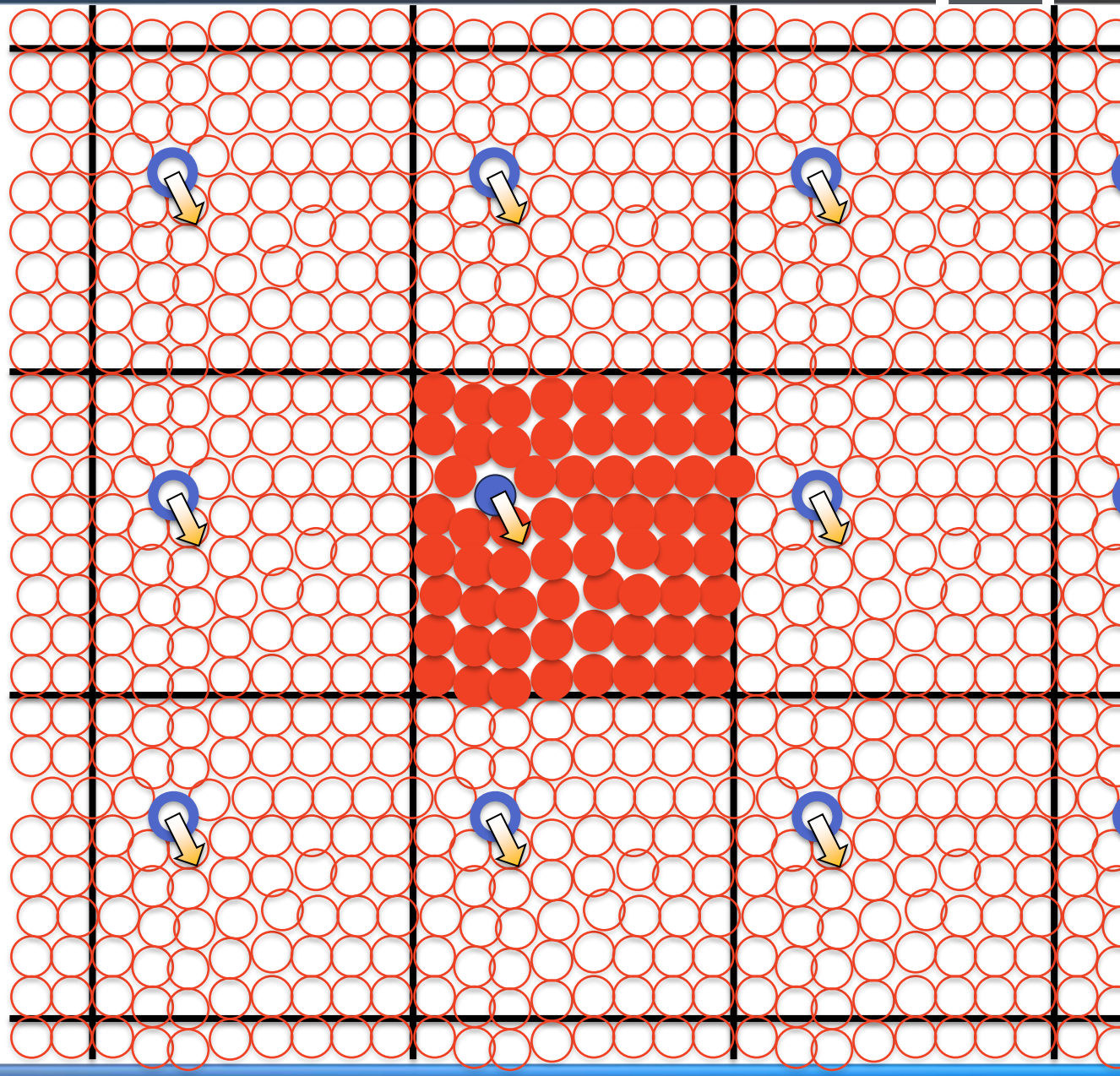


Periodic boundary conditions is not the same as real bulk!

If we look at the ordered state of a crystal lattice, applying pbc returns the perfect bulk!

but

1. All images of each atom behave exactly in the same way from one system replica to the other: full correlation beyond L_x, L_y, L_z



Periodic boundary conditions is not the same as real bulk!

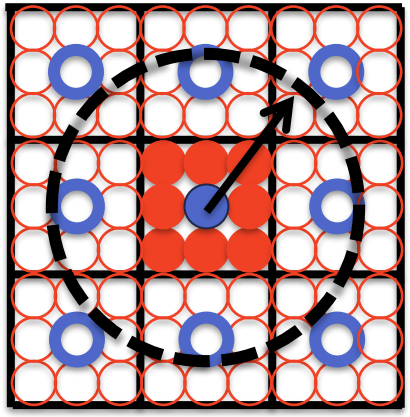
If we look at the ordered state of a crystal lattice, applying pbc returns the perfect bulk!

but

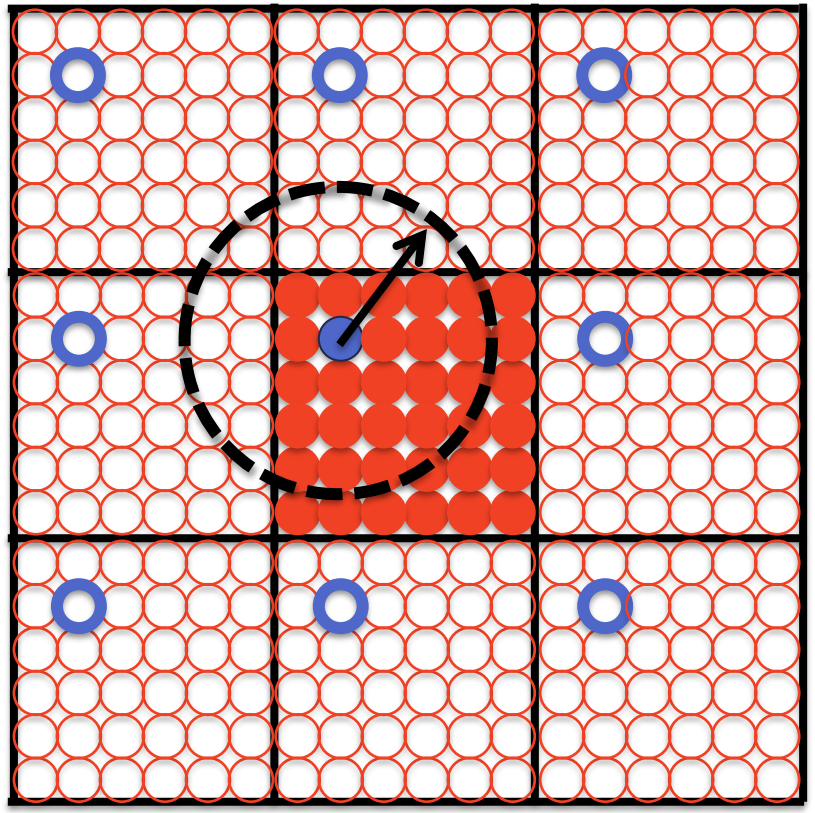
1. All images of each atom behave exactly in the same way from one system replica to the other: full correlation beyond L_x, L_y, L_z
2. Possible self-interactions: atoms can interact with their own image if the cutoff is larger than L_x, L_y, L_z
   → minimum cell size to avoid this:

$$L_{min} > 2r_c$$

minimum image convention: each atom can at most interact with only one image (including the original configuration) of other atoms



$L < 2r_c$ → NO



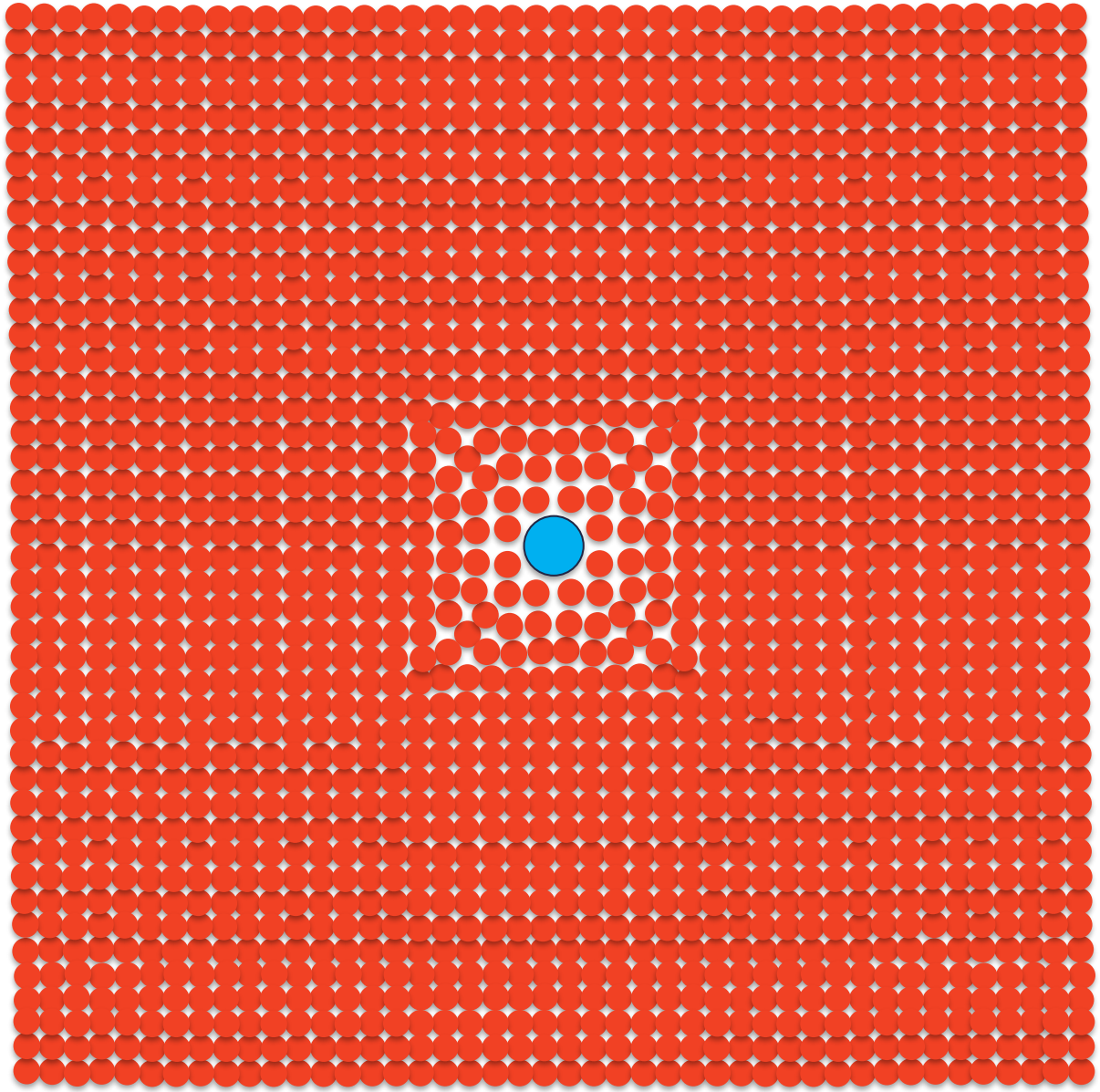
$L > 2r_c$ → OK

Periodic boundary conditions is not the same as real bulk!

If we look at the ordered state of a crystal lattice, applying pbc returns the perfect bulk!

but

1. All images of each atom behave exactly in the same way from one system replica to the other: full correlation beyond L_x, L_y, L_z
2. Possible self-interactions: atoms can interact with their own image if the cutoff is larger than L_x, L_y, L_z
   → minimum cell size to avoid this: $L_{min} > 2r_c$
3. Long-range effects are badly reproduced unless taking large enough cells
   - Inhomogeneities/point defects (vacancies, interstitials, inclusions)



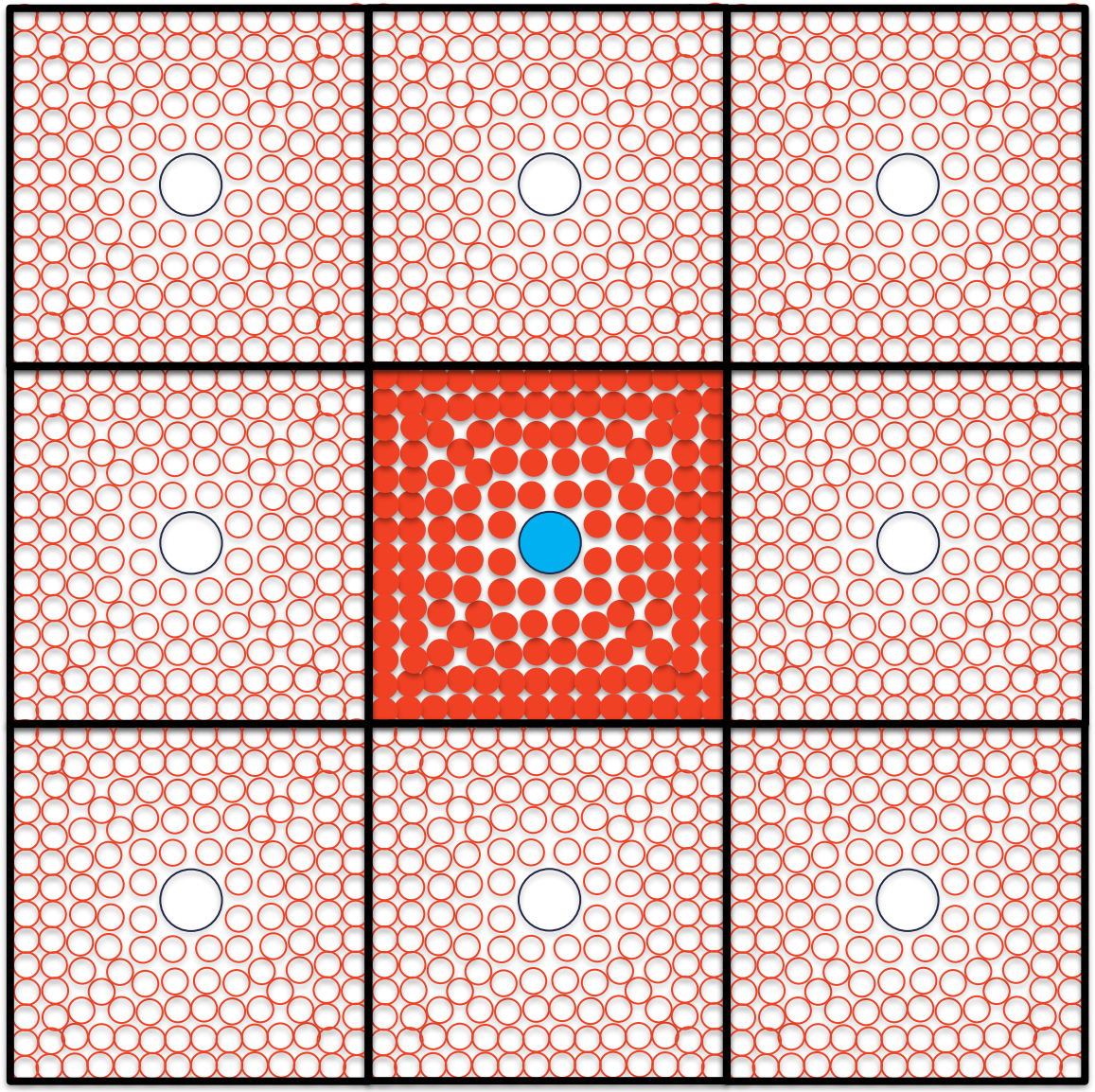
Periodic boundary conditions is not the same as real bulk!

If we look at the ordered state of a crystal lattice, applying pbc returns the perfect bulk!

but

1. All images of each atom behave exactly in the same way from one system replica to the other: full correlation beyond L_x, L_y, L_z
2. Possible self-interactions: atoms can interact with their own image if the cutoff is larger than L_x, L_y, L_z
   → minimum cell size to avoid this: $L_{min} > 2r_c$
3. Long-range effects are badly reproduced unless taking large enough cells
   - Inhomogeneities/point defects (vacancies, interstitials, inclusions)

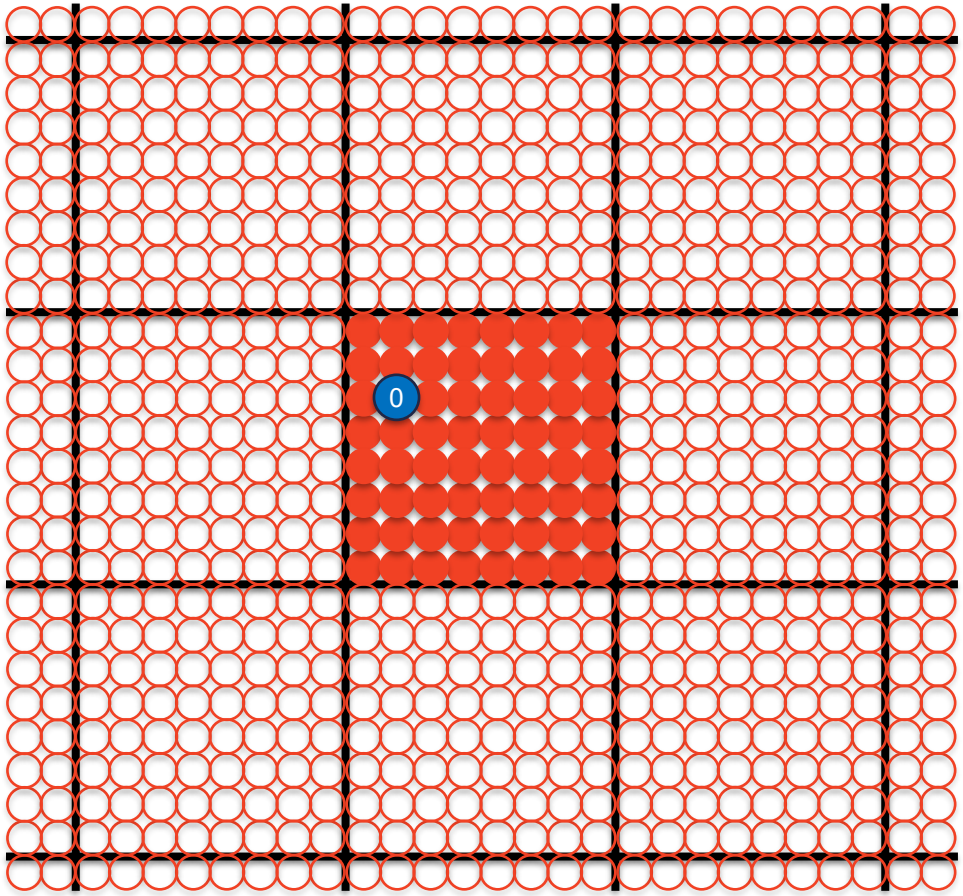
We will never simulate an isolated defect but a perfectly ordered array of defects of density $\frac{1}{L_x L_y L_z}$



Periodic boundary conditions and dynamics

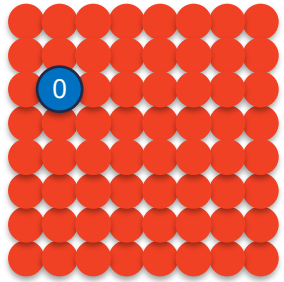
Let us consider the simulation of a particle ● diffusing on a surface (or a defect moving in a bulk, ...).
How do we trace the trajectory?

a) follow the unbounded (x, y, z) particle's coordinates while using pbc to evaluate interatomic distances



b) follow the motion of the particle and of its images inside the simulation cell only. Pbc are used both to evaluate distances and to trace the $(x^{pbc}, y^{pbc}, z^{pbc})$ coordinates within the cell $(0,0,0) \rightarrow (L_x, L_y, L_z)$.

floor=lower integer



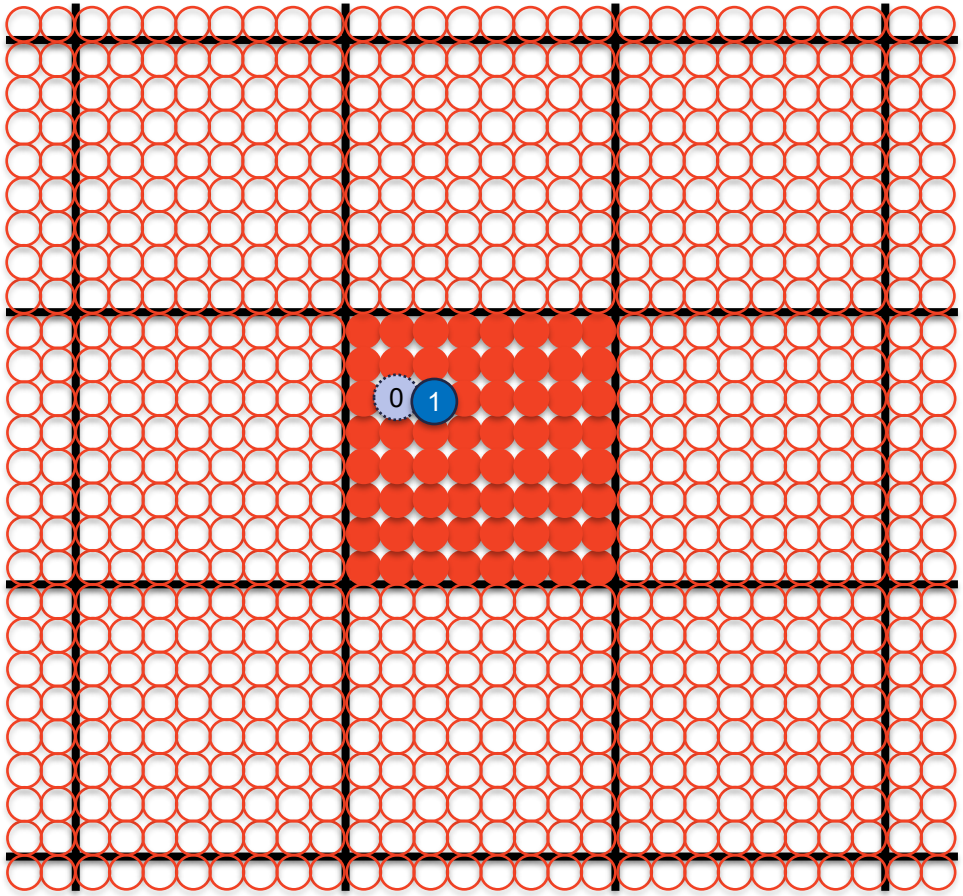
$$x^{pbc} = x - L_x \times \text{floor}\left(\frac{x}{L_x}\right)$$
$$y^{pbc} = y - L_y \times \text{floor}\left(\frac{y}{L_y}\right)$$
$$z^{pbc} = z - L_z \times \text{floor}\left(\frac{z}{L_z}\right)$$

By reducing the view within the simulation cell we better see which are the atoms interacting with the diffusing particle...
... but we lose the measure of the actual traveled distance.
Choose the representation depending on the analysis!

Periodic boundary conditions and dynamics

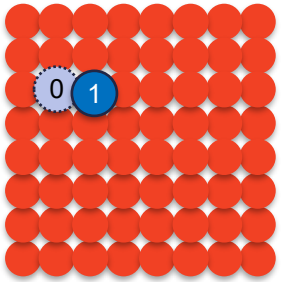
Let us consider the simulation of a particle ● diffusing on a surface (or a defect moving in a bulk, ...).
How do we trace the trajectory?

a) follow the unbounded (x, y, z) particle's coordinates while using pbc to evaluate interatomic distances



b) follow the motion of the particle and of its images inside the simulation cell only. Pbc are used both to evaluate distances and to trace the $(x^{pbc}, y^{pbc}, z^{pbc})$ coordinates within the cell $(0,0,0) \rightarrow (L_x, L_y, L_z)$.

floor=lower integer



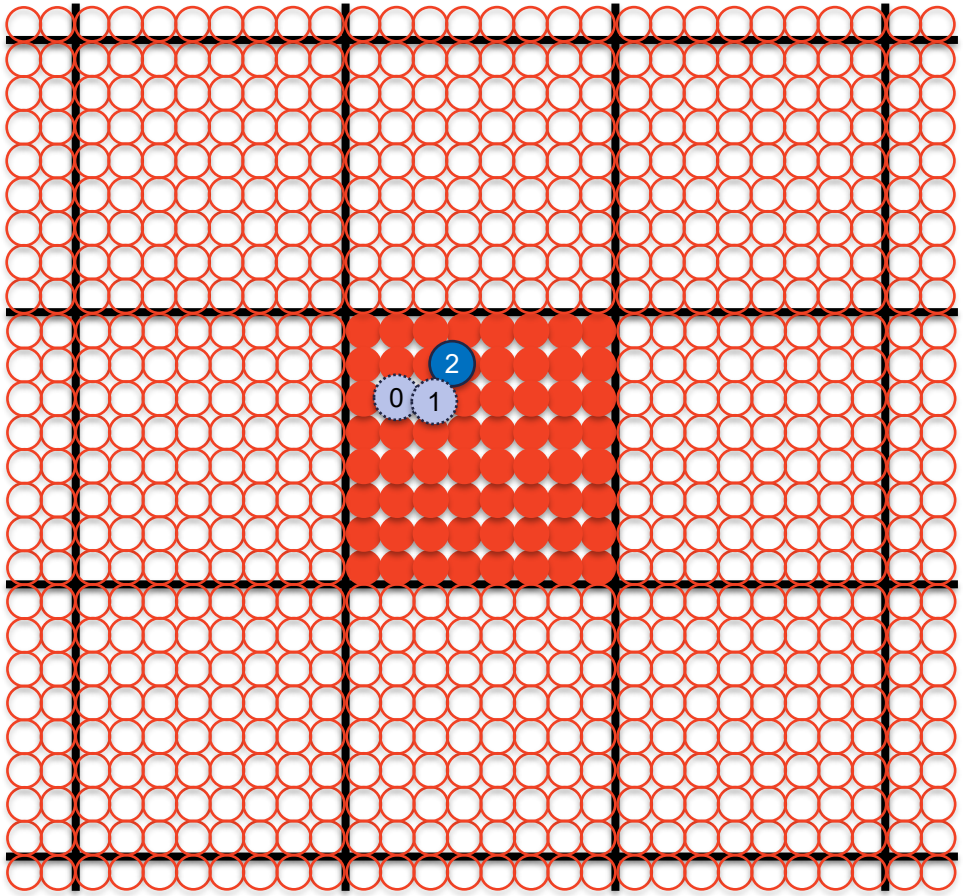
$$x^{pbc} = x - L_x \times \text{floor}\left(\frac{x}{L_x}\right)$$
$$y^{pbc} = y - L_y \times \text{floor}\left(\frac{y}{L_y}\right)$$
$$z^{pbc} = z - L_z \times \text{floor}\left(\frac{z}{L_z}\right)$$

By reducing the view within the simulation cell we better see which are the atoms interacting with the diffusing particle...
... but we lose the measure of the actual traveled distance.
Choose the representation depending on the analysis!

Periodic boundary conditions and dynamics

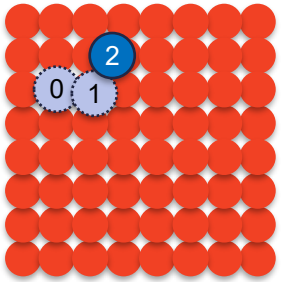
Let us consider the simulation of a particle ● diffusing on a surface (or a defect moving in a bulk, ...).
How do we trace the trajectory?

a) follow the unbounded (x, y, z) particle's coordinates while using pbc to evaluate interatomic distances



b) follow the motion of the particle and of its images inside the simulation cell only. Pbc are used both to evaluate distances and to trace the $(x^{pbc}, y^{pbc}, z^{pbc})$ coordinates within the cell $(0,0,0) \rightarrow (L_x, L_y, L_z)$.

floor=lower integer



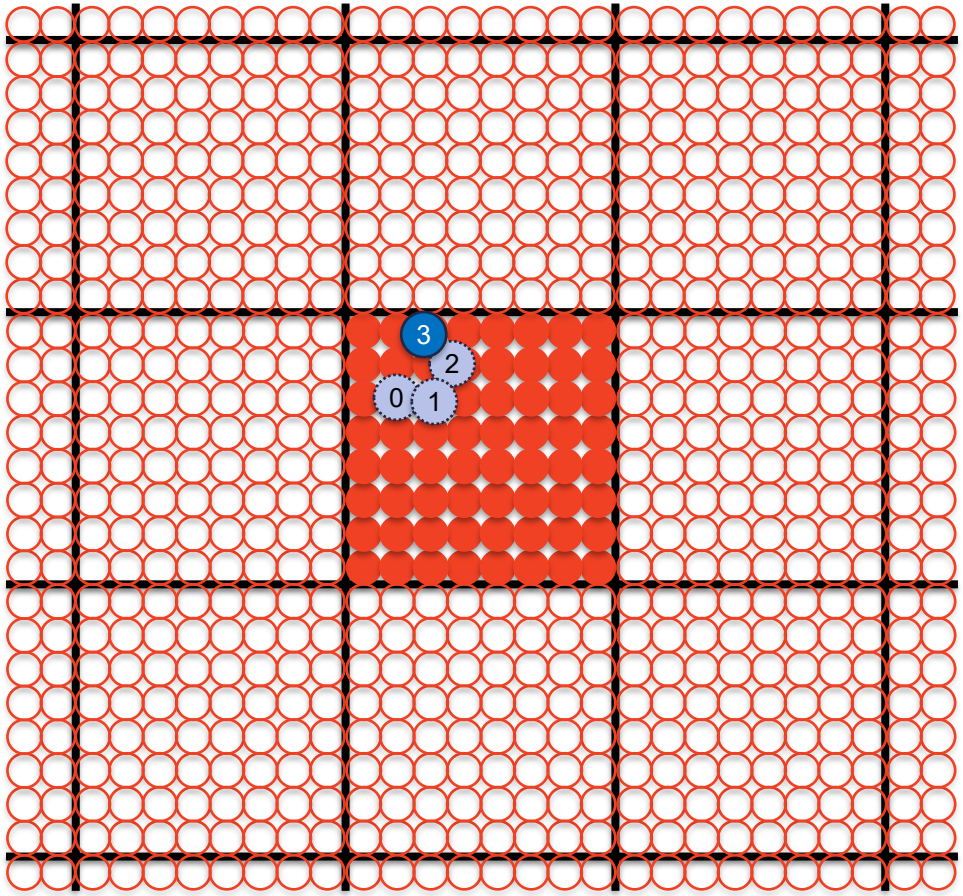
$$x^{pbc} = x - L_x \times \text{floor}\left(\frac{x}{L_x}\right)$$
$$y^{pbc} = y - L_y \times \text{floor}\left(\frac{y}{L_y}\right)$$
$$z^{pbc} = z - L_z \times \text{floor}\left(\frac{z}{L_z}\right)$$

By reducing the view within the simulation cell we better see which are the atoms interacting with the diffusing particle...
... but we lose the measure of the actual traveled distance.
Choose the representation depending on the analysis!

Periodic boundary conditions and dynamics

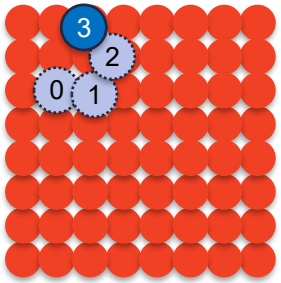
Let us consider the simulation of a particle ● diffusing on a surface (or a defect moving in a bulk, ...).
How do we trace the trajectory?

a) follow the unbounded (x, y, z) particle's coordinates while using pbc to evaluate interatomic distances



b) follow the motion of the particle and of its images inside the simulation cell only. Pbc are used both to evaluate distances and to trace the $(x^{pbc}, y^{pbc}, z^{pbc})$ coordinates within the cell $(0,0,0) \rightarrow (L_x, L_y, L_z)$.

floor=lower integer



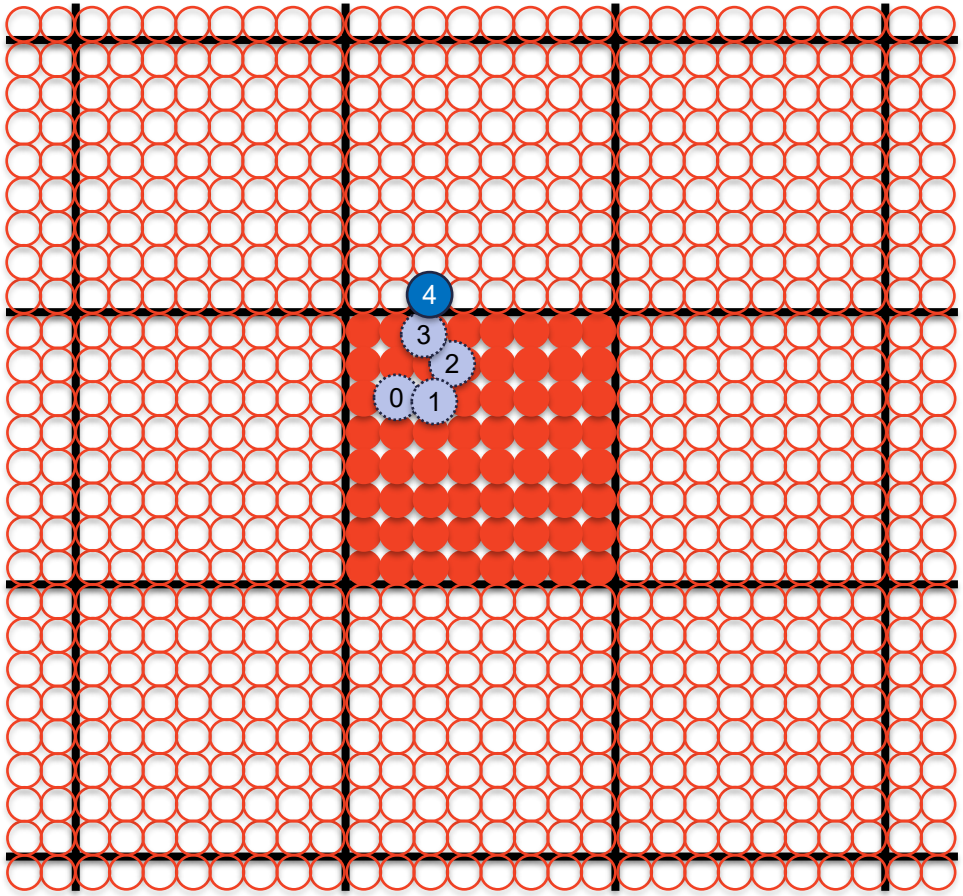
$$x^{pbc} = x - L_x \times \text{floor}\left(\frac{x}{L_x}\right)$$
$$y^{pbc} = y - L_y \times \text{floor}\left(\frac{y}{L_y}\right)$$
$$z^{pbc} = z - L_z \times \text{floor}\left(\frac{z}{L_z}\right)$$

By reducing the view within the simulation cell we better see which are the atoms interacting with the diffusing particle...
... but we lose the measure of the actual traveled distance.
Choose the representation depending on the analysis!

Periodic boundary conditions and dynamics

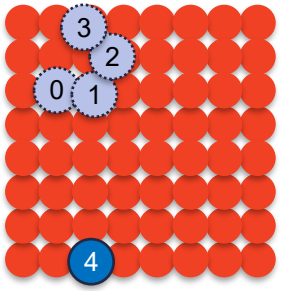
Let us consider the simulation of a particle ● diffusing on a surface (or a defect moving in a bulk, ...).
How do we trace the trajectory?

a) follow the unbounded (x, y, z) particle's coordinates while using pbc to evaluate interatomic distances



b) follow the motion of the particle and of its images inside the simulation cell only. Pbc are used both to evaluate distances and to trace the $(x^{pbc}, y^{pbc}, z^{pbc})$ coordinates within the cell $(0,0,0) \rightarrow (L_x, L_y, L_z)$.

floor=lower integer



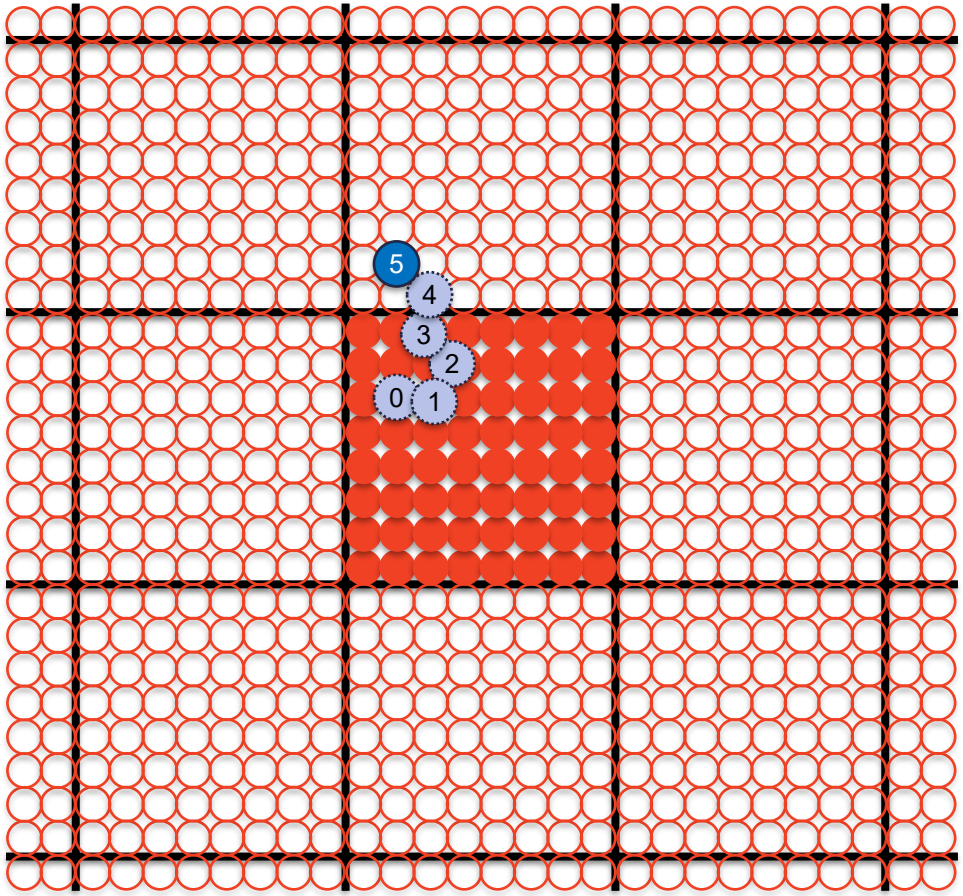
$$x^{pbc} = x - L_x \times \text{floor}\left(\frac{x}{L_x}\right)$$
$$y^{pbc} = y - L_y \times \text{floor}\left(\frac{y}{L_y}\right)$$
$$z^{pbc} = z - L_z \times \text{floor}\left(\frac{z}{L_z}\right)$$

By reducing the view within the simulation cell we better see which are the atoms interacting with the diffusing particle...
... but we lose the measure of the actual traveled distance.
Choose the representation depending on the analysis!

Periodic boundary conditions and dynamics

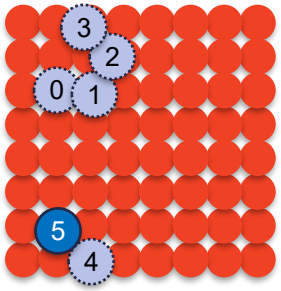
Let us consider the simulation of a particle ● diffusing on a surface (or a defect moving in a bulk, ...).
How do we trace the trajectory?

a) follow the unbounded (x, y, z) particle's coordinates while using pbc to evaluate interatomic distances



b) follow the motion of the particle and of its images inside the simulation cell only. Pbc are used both to evaluate distances and to trace the $(x^{pbc}, y^{pbc}, z^{pbc})$ coordinates within the cell $(0,0,0) \rightarrow (L_x, L_y, L_z)$.

floor=lower integer



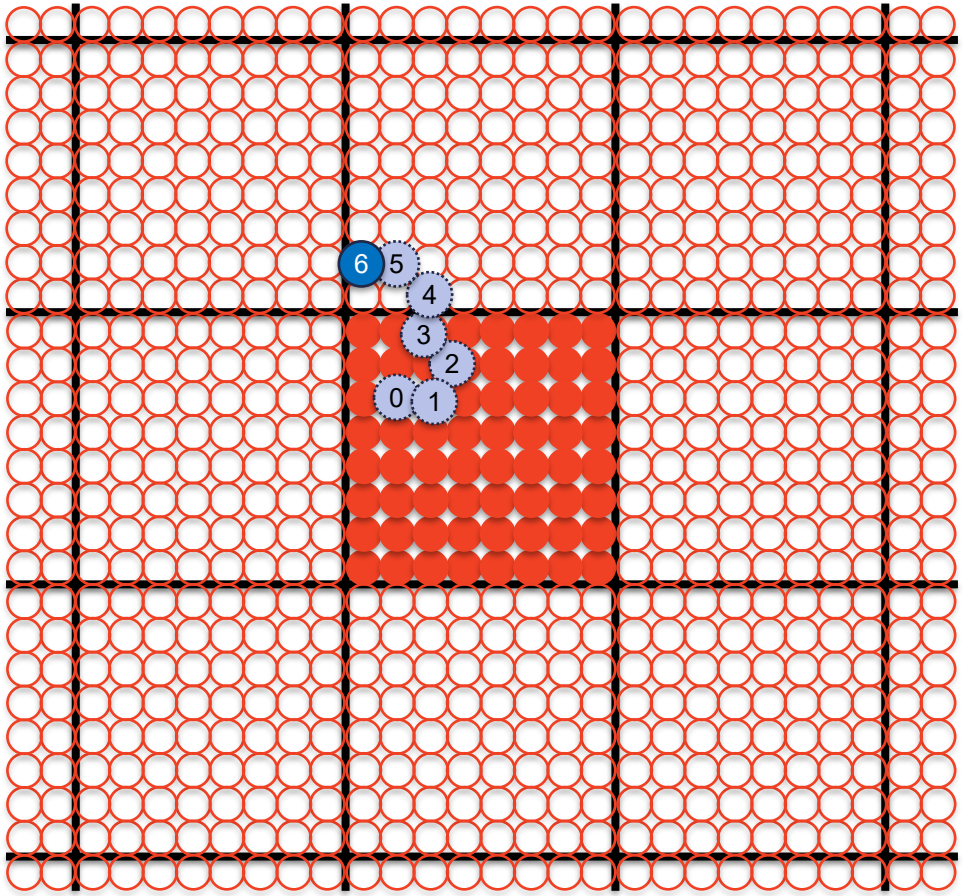
$$x^{pbc} = x - L_x \times \text{floor}\left(\frac{x}{L_x}\right)$$
$$y^{pbc} = y - L_y \times \text{floor}\left(\frac{y}{L_y}\right)$$
$$z^{pbc} = z - L_z \times \text{floor}\left(\frac{z}{L_z}\right)$$

By reducing the view within the simulation cell we better see which are the atoms interacting with the diffusing particle...
... but we lose the measure of the actual traveled distance.
Choose the representation depending on the analysis!

Periodic boundary conditions and dynamics

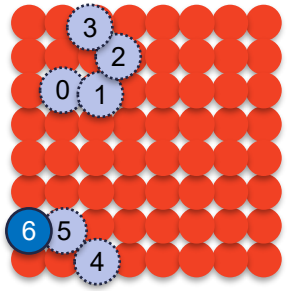
Let us consider the simulation of a particle ● diffusing on a surface (or a defect moving in a bulk, ...).
How do we trace the trajectory?

a) follow the unbounded (x, y, z) particle's coordinates while using pbc to evaluate interatomic distances



b) follow the motion of the particle and of its images inside the simulation cell only. Pbc are used both to evaluate distances and to trace the $(x^{pbc}, y^{pbc}, z^{pbc})$ coordinates within the cell $(0,0,0) \rightarrow (L_x, L_y, L_z)$.

floor=lower integer



$$x^{pbc} = x - L_x \times \text{floor}\left(\frac{x}{L_x}\right)$$

$$y^{pbc} = y - L_y \times \text{floor}\left(\frac{y}{L_y}\right)$$

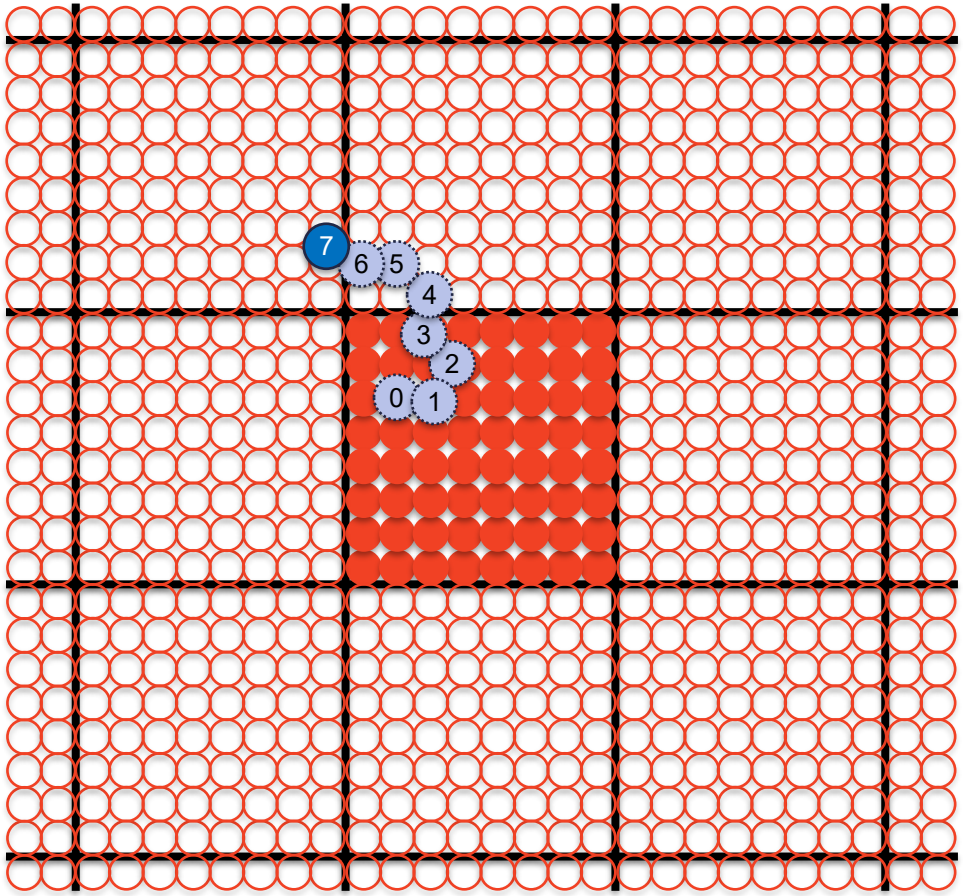
$$z^{pbc} = z - L_z \times \text{floor}\left(\frac{z}{L_z}\right)$$

By reducing the view within the simulation cell we better see which are the atoms interacting with the diffusing particle...
... but we lose the measure of the actual traveled distance.
Choose the representation depending on the analysis!

Periodic boundary conditions and dynamics

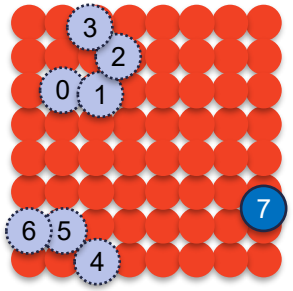
Let us consider the simulation of a particle ● diffusing on a surface (or a defect moving in a bulk, ...).
How do we trace the trajectory?

a) follow the unbounded (x, y, z) particle's coordinates while using pbc to evaluate interatomic distances



b) follow the motion of the particle and of its images inside the simulation cell only. Pbc are used both to evaluate distances and to trace the $(x^{pbc}, y^{pbc}, z^{pbc})$ coordinates within the cell $(0,0,0) \rightarrow (L_x, L_y, L_z)$.

floor=lower integer



$$x^{pbc} = x - L_x \times \text{floor}\left(\frac{x}{L_x}\right)$$

$$y^{pbc} = y - L_y \times \text{floor}\left(\frac{y}{L_y}\right)$$

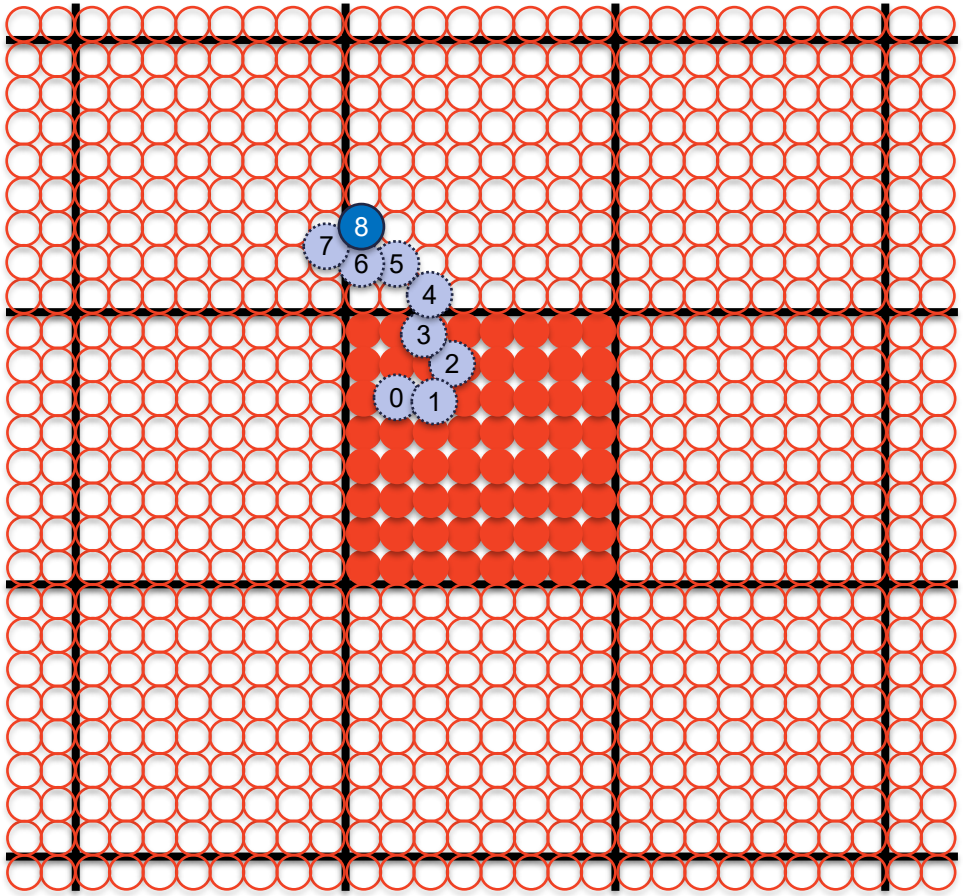
$$z^{pbc} = z - L_z \times \text{floor}\left(\frac{z}{L_z}\right)$$

By reducing the view within the simulation cell we better see which are the atoms interacting with the diffusing particle...
... but we lose the measure of the actual traveled distance.
Choose the representation depending on the analysis!

Periodic boundary conditions and dynamics

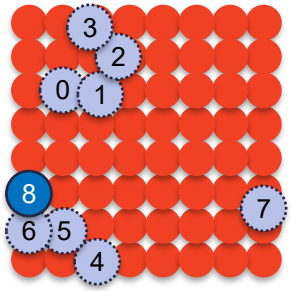
Let us consider the simulation of a particle ● diffusing on a surface (or a defect moving in a bulk, ...).
How do we trace the trajectory?

a) follow the unbounded (x, y, z) particle's coordinates while using pbc to evaluate interatomic distances



b) follow the motion of the particle and of its images inside the simulation cell only. Pbc are used both to evaluate distances and to trace the $(x^{pbc}, y^{pbc}, z^{pbc})$ coordinates within the cell $(0,0,0) \rightarrow (L_x, L_y, L_z)$.

floor=lower integer



$$x^{pbc} = x - L_x \times \text{floor}\left(\frac{x}{L_x}\right)$$

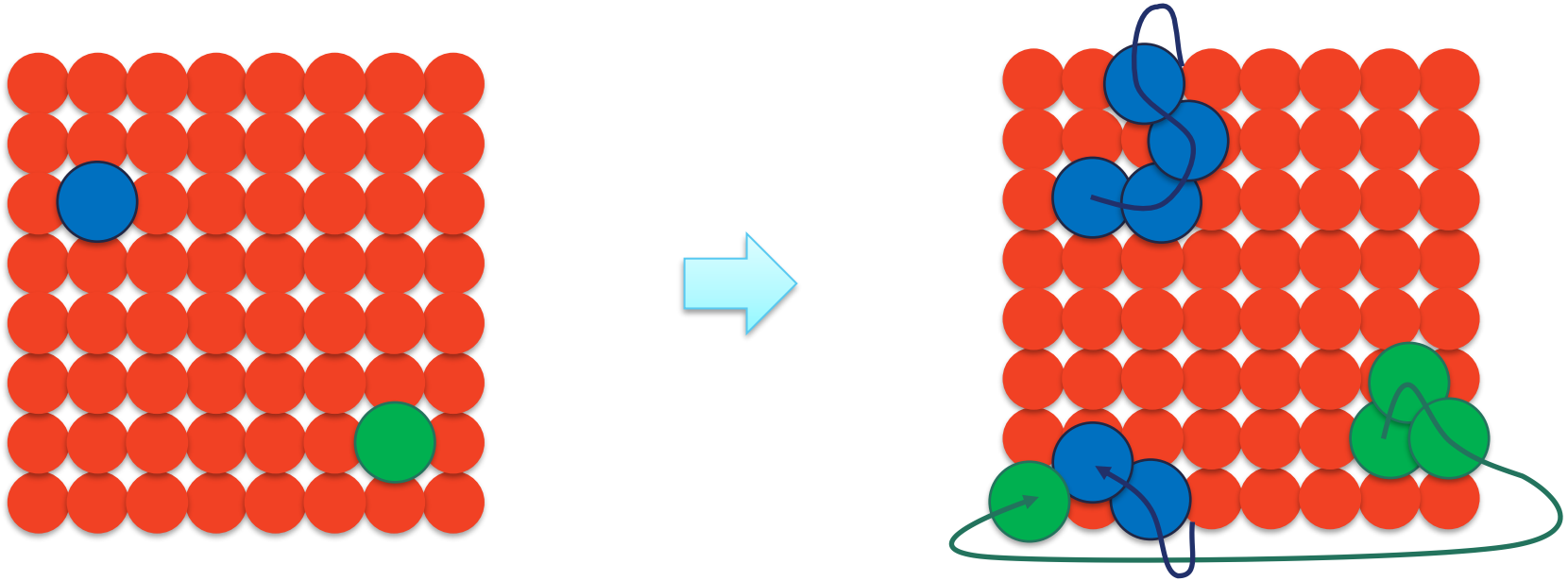
$$y^{pbc} = y - L_y \times \text{floor}\left(\frac{y}{L_y}\right)$$

$$z^{pbc} = z - L_z \times \text{floor}\left(\frac{z}{L_z}\right)$$

By reducing the view within the simulation cell we better see which are the atoms interacting with the diffusing particle...
... but we lose the measure of the actual traveled distance.
Choose the representation depending on the analysis!

Periodic boundary conditions: interacting particles

Now we add a second particle in the simulation cell and study when the two particles meet with each other, e.g. to form a surface cluster. Similar dynamics for vacancy-interstitial annihilation or dislocation reactions.



The finiteness + pbc of the simulation cell enhance the interaction probability with respect to the real system! The two particles cannot go farther than the cell size and if they “exit” from one side of the cell they are thrown in again from the other side.

Once again, large enough cells compared to the characteristic diffusion length are needed.

Periodic boundary conditions and trajectories

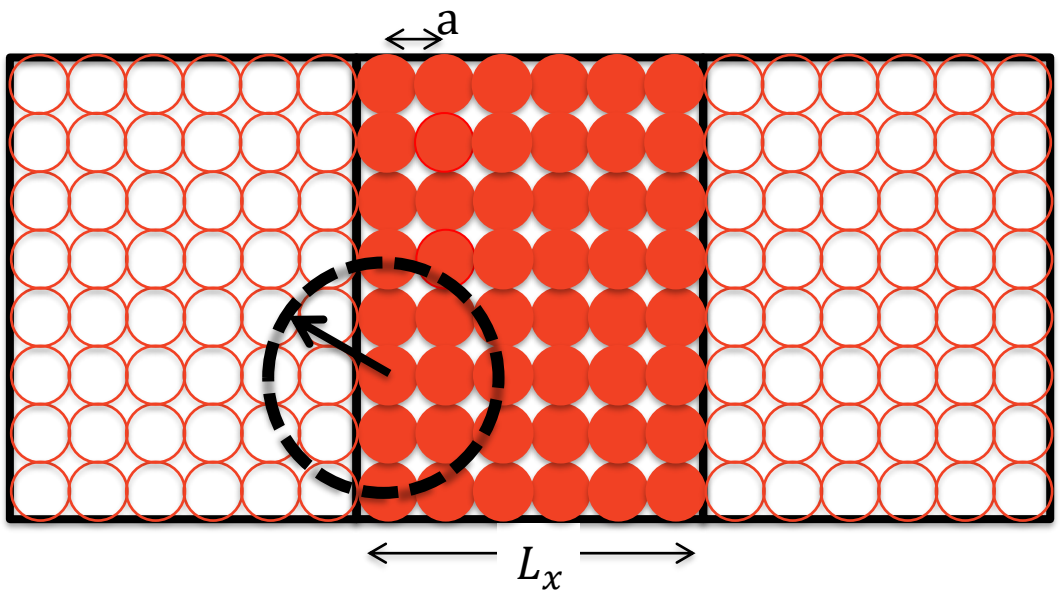
The choice of using or not pbc depends on the physical system under investigation:

- bulk-like → pbc
- cluster/isolated particles → no pbc

A comprehensive MD code should include both options. How?

a) (not convenient) implement a `if` condition to choose among the two definitions of interatomic distances with or without pbc

b) just use the pbc implementation but control the boundary interactions by changing L_x, L_y, L_z



In order to reproduce a proper bulk we take the simulation cell as large as a N-cell bulk unit

$$L_x = N\mathrm{a}$$
$$\downarrow$$
pbc along the x axis

Periodic boundary conditions and trajectories

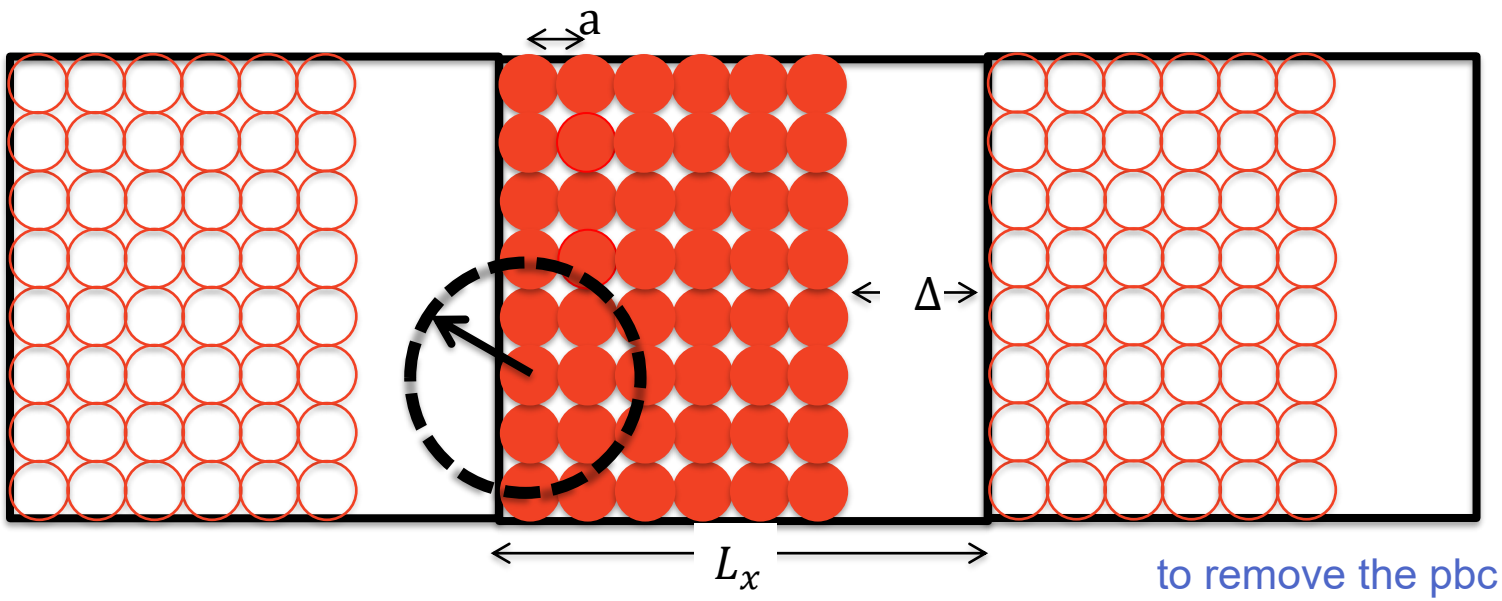
The choice of using or not pbc depends on the physical system under investigation:

- bulk-like → pbc
- cluster/isolated particles → no pbc

A comprehensive MD code should include both options. How?

a) (not convenient) implement a `if` condition to choose among the two definitions of interatomic distances with or without pbc

b) just use the pbc implementation but control the boundary interactions by changing L_x, L_y, L_z



Lets add a vacuum region so that atoms on the boundary do not interact with any image on the adjacent replica

$$L_x = Na + \Delta \quad (\Delta > r_c)$$
$$\downarrow$$
free surfaces along the x axis

to remove the pbc in one direction simply enlarge the cell by $\Delta \gg r_c$

Coding hints

Code efficiency: vector operations

Python can efficiently process operations on all components of arrays e.g. for Verlet loop:

👎

```
for i in range(0,natoms):
   for j in range(0,3):
      pos[i,j]=pos[i,j]+vel[i,j]*dt+0.5*(force[i,j]/mass)*dt**2
```

can be simply replaced by

👍

```
pos = pos + vel*dt + 0.5*force/mass*dt**2
```

This is executed faster with respect to scrolling each array component so it is better to code like-this.

Code efficiency: vector operations



However, using vector operations implies some overhead. It is convenient only for long enough arrays. E.g. when computing the distance between two atoms `i` and `j`



```
dx=pos[i,0]-pos[j,0]
dy=pos[i,1]-pos[j,1]
dz=pos[i,2]-pos[j,2]
d2=dx*dx+dy*dy+dz*dz
dis=d2**0.5
```

is still faster than

```
dxyz=pos[i,:]-pos[j,:]
dis=np.sqrt( sum(dxyz**2) )
```



- Basic arithmetic operations on 3 scalar number

- Array allocation
- Call to numpy functions for basic operations

Code efficiency: interpretation vs. compilation



```
from numba import jit

@jit
def calc_nbrs(natoms,pos,rc):
    nnbrs=np.zeros(natoms,int)
    whichnbr = np.zeros((natoms,natoms),int)
    for i in range(0,natoms):
        for j in range(0,natoms):
            dx=pos[i,0]-pos[j,0]
            dy=pos[i,1]-pos[j,1]
            dz=pos[i,2]-pos[j,2]
            d2=dx*dx+dy*dy+dz*dz
            dis=d2**0.5
            if i != j and dis<(rc):
                whichnbr[i,nnbrs[i]]=j
                nnbrs[i]=nnbrs[i]+1

    return nnbrs, whichnbr

@jit                    @jit
def calc_energy(…):     def calc_forces(…):
```

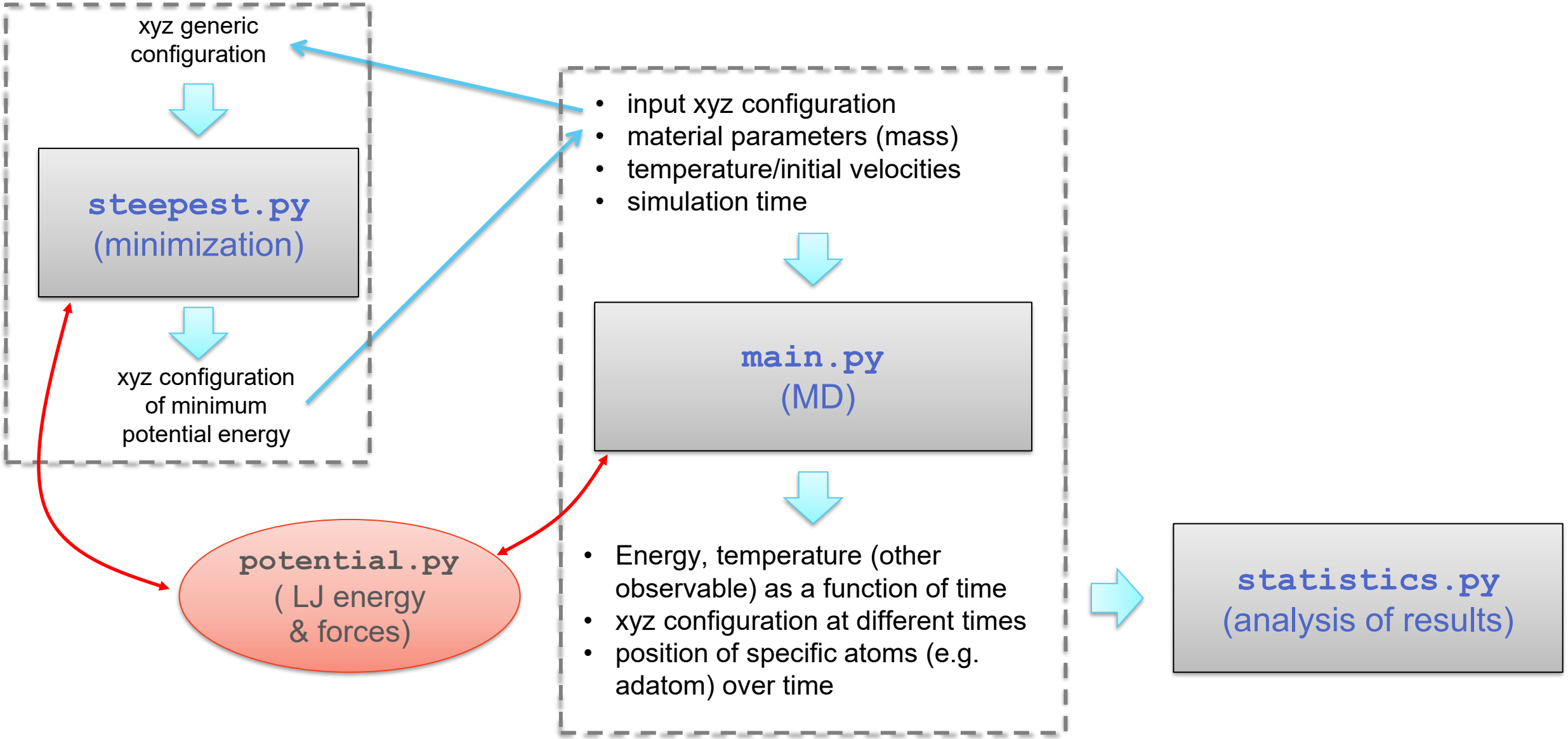
Python is an interpreted language so that instructions are converted into executable code only line-by-line. This is very costly and hence return slow computations especially when repeating operation over explicit loops.

Compiled languages like C/C++ or Fortran could handle these much faster!



Just In Time compilation (from module `numba`) automatically converts into compiled code a function starting with the decorator `@jit` → significant speed up of execution of functions with repeating operations

Wrap-up


xyz generic configuration
steepest.py (minimization)
xyz configuration of minimum potential energy
potential.py (LJ energy & forces)
input xyz configuration
material parameters (mass)
temperature/initial velocities
simulation time
main.py (MD)
Energy, temperature (other observable) as a function of time
xyz configuration at different times
position of specific atoms (e.g. adatom) over time
statistics.py (analysis of results)